

SQL va MySQL — Noldan boshlovchilar uchun amaliy kitob

 [Butun kitobni PDF formatda yuklab olish →](#)

Bu kitob **hech qachon dasturlash qilmagan** odam ham tushunadigan tilda yozilgan. Har bir bobda: sodda nazariya → tayyor misollar → **20 ta masala** (o'zingiz yechasiz). Jami 25 bob, 500 masala.

 Har bob **SVG diagrammalar** bilan boyitilgan — JOIN, GROUP BY, indeks, tranzaksiya, normalizatsiya kabi tushunchalar ko'z bilan ko'rib o'rganiladi.

Qoida: SQL o'qib o'rganilmaydi — **YOZIB** o'rganiladi. Har bir masalani kompyuterda o'zingiz tering. Xato chiqsa — bu yaxshi, xatodan o'rganasiz.

Qanday o'qish kerak

1. Boblarni **tartib bilan** o'qing (01 → 02 → ...). Har biri oldingisiga tayanadi.
2. 3-bobdagi **4 ta amaliyot bazasini** (kutubxona, do'kon, klinika, taksi) albatta o'rnating — butun kitob masalalari shularga asoslangan.
3. Har bob oxiridagi **20 ta masalani** o'zingiz yeching — kod misollarini ko'chirib qo'yish bilan SQL o'rganilmaydi.
4. Diagrammalar tushunchani tezroq singdiradi — ularga e'tibor bering.

Talab

Kerak	Daraja
Kompyuter (Windows / macOS / Linux)	Shart
MySQL 8.0 (2-bobda o'rnatamiz)	Shart
Oldingi dasturlash tajribasi	Shart emas

I qism — Tanishuv

#	Bob	Nima o'rganasiz
01	Ma'lumotlar bazasi nima?	Ma'lumotlar bazasi nima va u daftar yoki Excel'dan nimasi bilan kuchli ekanini, jadval/qator/ustun/id tushunchalarini, SQL (til) bilan MySQL (dastur) farqini va klient-server modelini kutubxona misolida o'rganamiz. Bob oxirida kompyutersiz yechiladigan 20 ta qog'oz masalasi bor.
02	MySQL'ni o'rnatish va ishga tushirish	MySQL'ning server-klient arxitekturasini tushunib, uni Windows (Laragon), macOS (brew) va Linux (apt) tizimlariga o'rnatamiz, terminal hamda DBeaver orqali ulanib, birinchi SQL buyruqlarimizni bajarishni o'rganamiz.
03	Amaliyot bazalarini tayyorlash (4 ta tizim)	Butun kitob davomida ishlatiladigan 4 ta amaliyot bazasini (kutubxona, do'kon, klinika, taksi) yaratamiz, jadvallar orasidagi bog'lanishlarni ER-diagrammalarda ko'rib, skriptlarni bajarib tekshirishni o'rganamiz.

II qism — Asoslar

#	Bob	Nima o'rganasiz
04	CREATE — baza va jadval yaratish	CREATE DATABASE, USE va CREATE TABLE bilan birinchi baza va jadvalimizni quramiz; PRIMARY KEY, AUTO_INCREMENT, NOT NULL, UNIQUE, DEFAULT qoidalarini hamda SHOW/DESCRIBE/DROP yordamchi buyruqlarini o'rganamiz.
05	Ma'lumot turlari	Sonlar (INT, DECIMAL, FLOAT), matnlar (CHAR, VARCHAR, TEXT), sana-vaqt va maxsus turlar (ENUM, BOOLEAN, JSON) bilan tanishamiz; har bir ustunga to'g'ri tur tanlashni va pul nima uchun faqat DECIMAL'da saqlanishini o'rganamiz.
06	INSERT — ma'lumot	Jadvalga yangi qator qo'shishni o'rganamiz: INSERT INTO anatomiyasi, ko'p qatorli kiritish,

#	Bob	Nima o'rganasiz
	kiritish	AUTO_INCREMENT va DEFAULT qanday ishlashi, NOW() va LAST_INSERT_ID() hamda INSERT xatolarini o'qib tushunish.
07	SELECT — ma'lumot o'qish	SQL'ning eng ko'p ishlatiladigan buyrug'i — SELECT bilan jadvaldan kerakli ustunlarni tanlab o'qishni, alias (AS) berishni, SELECT ichida hisob-kitob qilishni va DISTINCT bilan takrorlarni olib tashlashni o'rganamiz.
08	WHERE — filtrlash	Qatorlarni shart bo'yicha filtrlashni o'rganamiz: taqqoslash operatorlari, AND/OR/NOT mantiqiy bog'lovchilari va qavslar, BETWEEN va IN, LIKE naqshlari hamda NULL bilan to'g'ri ishlash (IS NULL).
09	ORDER BY, LIMIT, DISTINCT	Natijani ORDER BY bilan saralashni, LIMIT/OFFSET bilan kesishni ("eng katta N ta" qolipi va sahifalash) hamda DISTINCT bilan takror qiymatlarni olib tashlashni o'rganamiz.
10	Built-in funksiyalar (matn, son, sana)	MySQL'ning tayyor funksiyalarini o'rganamiz: matn (UPPER, CONCAT, SUBSTRING), son (ROUND, CEIL, FLOOR), sana (NOW, DATEDIFF, DATE_FORMAT, TIMESTAMPDIF), shartli qiymat (IF, CASE) va NULL bilan ishlash (IFNULL, COALESCE) — hammasi hayotiy misollarda.

III qism — Kuchli qurollar

#	Bob	Nima o'rganasiz
11	Aggregate funksiyalar va GROUP BY	COUNT, SUM, AVG, MIN, MAX bilan ko'p qatordan bitta xulosa chiqarishni, "har bir ... bo'yicha" savollariga GROUP BY bilan javob berishni va guruhlarini HAVING bilan filtrlashni o'rganamiz. COUNT(*) va COUNT(ustun) orasidagi NULL farqi hamda query bajarilish tartibini ham ko'rib chiqamiz.
12	JOIN — jadvallarni	Bo'lingan ma'lumotni qaytadan yig'ishni — JOIN'ni o'rganamiz: ON sharti, INNER va LEFT JOIN farqi,

#	Bob	Nima o'rganasiz
	bog'lash	anti-join qolipi, LEFT JOIN + WHERE tuzog'i, 3-4 jadvalli zanjirlar va JOIN + GROUP BY bilan real hisobotlar.
13	UNION — natijalarni birlashtirish	Bir nechta SELECT natijasini bitta ro'yxatga "tagma-tag" ulaydigan UNION va UNION ALL'ni, ustun yetishmaganda NULL bilan to'ldirishni, UNION'da ORDER BY/LIMIT qoidalarini va JOIN bilan farqini o'rganamiz.
14	Subquery — query ichida query	Query ichiga yana bitta query joylashni o'rganamiz: skalyar, IN-ro'yxat va correlated (EXISTS) subquery turlari, mashhur NOT IN + NULL tuzog'idan qochish hamda FROM ichidagi hosila jadval bilan ikki bosqichli hisob-kitoblar.
15	CTE — WITH bilan ishlash	WITH yordamida murakkab query'ni nomli, yuqoridan pastga o'qiladigan bosqichlarga bo'lishni, bir nechta CTE'ni zanjir qilishni va recursive CTE bilan jadvalsiz sonlar, kalendar hamda daraxt strukturalar hosil qilishni o'rganamiz.
16	Window funksiyalar	GROUP BY'dan farqli, qatorlarni yo'qotmasdan guruh statistikasini hisoblashni o'rganamiz: OVER va PARTITION BY, ROW_NUMBER/RANK/DENSE_RANK bilan raqamlash, LAG/LEAD, yig'ilib boruvchi summa va LAST_VALUE'dagi frame tuzog'i.

IV qism — Ma'lumotni boshqarish

#	Bob	Nima o'rganasiz
17	UPDATE va DELETE — chuqur	Ma'lumotni o'zgartirish (UPDATE) va o'chirish (DELETE)ni xavfsiz ishlash odati bilan o'rganamiz: "avval SELECT" protokoli, SQL_SAFE_UPDATES kamari, NULL tuzog'i, soft delete yondashuvi va DELETE/TRUNCATE/DROP farqlari.
18	ALTER, Constraint, Foreign Key	Tayyor jadvalni ALTER TABLE bilan buzmasdan o'zgartirishni, bazaning o'zi qo'riqlaydigan qoidalar — NOT NULL, UNIQUE, DEFAULT, CHECK

#	Bob	Nima o'rganasiz
		constraint'larini va jadvallararo bog'lanish kafolati FOREIGN KEY'ni o'rganamiz; ota qator o'chirilganda nima bo'lishini ON DELETE (RESTRICT, CASCADE, SET NULL) bilan boshqaramiz.
19	Tranzaksiyalar	Tranzaksiya — "yo hammasi, yo hech narsa" tamoyilini o'rganamiz: START TRANSACTION, COMMIT va ROLLBACK, autocommit rejimi, parallel ishlashda FOR UPDATE qulfi, deadlock holati va ACID xossalari.
20	Normalizatsiya — to'g'ri schema	Bitta katta jadvalning kasalliklarini (takrorlash, yangilash/o'chirish/kiritish anomaliyalari), normalizatsiyaning 3 sodda qoidasini (1NF, 2NF, 3NF), bog'lanish turlarini (1:1, 1:N, N:M) va qachon ataylab takrorlash (snapshot, denormalizatsiya) to'g'ri ekanini o'rganamiz.

V qism — Professional daraja

#	Bob	Nima o'rganasiz
21	Indekslar	Indeksni kitob oxiridagi alfavit ko'rsatkich o'xshatishi bilan tushunamiz: 1 million qatorli jadvalda full scan va indeksli qidiruv farqini o'z qo'limiz bilan o'lchaymiz, B-tree daraxti, indeks ishlamaydigan tuzoqlar, kompozit indeks (chap prefiks qoidasi) va indeksning narxini o'rganamiz.
22	VIEW, Stored Procedure, Trigger, Event	Takrorlanadigan query'larni VIEW qilib nomlab qo'yishni, parametr qabul qiladigan stored procedure yozish va CALL bilan chaqirishni, jadvaldagi o'zgarishlarga avtomatik javob beradigan trigger hamda jadval bo'yicha o'z-o'zidan ishlaydigan EVENT'larni o'rganamiz.
23	EXPLAIN va optimizatsiya	Sekin query'ning sababini EXPLAIN bilan ochishni o'rganamiz: type/rows/key ustunlarini o'qish, const'dan ALL'gacha shkala, va to'liq

#	Bob	Nima o'rganasiz
		optimizatsiya sikli — sekin query → EXPLAIN → indeks → qayta o'lchash.
24	Xavfsizlik va administratsiya	Har ilovaga alohida foydalanuvchi yaratish va GRANT bilan minimal ruxsat berishni, SQL injection hujumi qanday ishlashini va prepared statement bilan himoyalanishni, mysqldump bilan backup/restore qilishni o'rganamiz.
25	Yakuniy loyihalar	O'rganganlarimizni jamlab 3 ta to'liq tizimni noldan quramiz: talablar → schema → CREATE → ma'lumotlar → hisobot query'lari. Oxirida kitobdan keyingi yo'l xaritasi: PostgreSQL, ORM va backend tomon.

Muallif

Oqil Imomnazarov — ioqil.uz · [Telegram](#) · [YouTube](#)

Kitob bepul tarqatiladi (CC BY-NC-SA 4.0). Savdo qilish taqiqlanadi.

01 — Ma'lumotlar bazasi nima?

 [README](#) · [Keyingi: 02 — MySQL'ni o'rnatish va ishga tushirish](#) 

Bu bobda: ma'lumotlar bazasi nima va u oddiy daftar yoki Excel'dan nimasi bilan kuchli ekanini, jadval, qator, ustun va id tushunchalarini, SQL (til) bilan MySQL (dastur) o'rtasidagi farqni hamda klient-server modeli qanday ishlashini o'rganamiz.

Hayotiy misol bilan boshlaymiz

Tasavvur qiling: siz mahalladagi kichik kutubxonada ishlaysiz. Sizda daftar bor, unga yozasiz:

```
Aziz Karimov — "O'tkan kunlar" kitobini oldi — 5-iyun  
Malika Tosheva — "Mehrobdan chayon" oldi — 6-iyun  
Aziz Karimov — kitobni qaytardi — 9-iyun
```

100 ta o'quvchi bo'lsa — daftar yetadi. 10 000 ta bo'lsa-chi? Savollar tug'iladi:

- "Aziz qaytarmagan kitoblari bormi?" — butun daftarni varaqlaysiz
- "Eng ko'p o'qilgan kitob qaysi?" — bir kunlik ish
- Ikki xodim bir vaqtda yozsa — daftar bitta, navbat kutadi

Ma'lumotlar bazasi (database) — mana shu daftarning kompyuterdagi aqlli versiyasi. U:

- Millionlab yozuv ichidan keraklisini **soniyaning ulushida** topadi
 - Bir vaqtda minglab odamga xizmat qiladi — hech kim navbat kutmaydi
 - Ma'lumotni yo'qotib qo'ymaslik uchun maxsus himoyaga ega: svet o'chib qolsa ham, saqlangan yozuv joyida qoladi
-

Excel yetmaydimi?

"Jadval kerak bo'lsa, Excel bor-ku!" — to'g'ri savol. Kichik ro'yxatlar (50–100 qator) uchun Excel juda qulay. Lekin jiddiy ish boshlanishi bilan farq sezilib qoladi:

Mezon	Excel	Ma'lumotlar bazasi
Hajm	million qator atrofida qiynala boshlaydi	yuz millionlab qator — muammosiz
Birga ishlash	fayl ochiq bo'lsa, boshqalar kutadi	minglab odam bir vaqtda yozadi va o'qiydi
Qidiruv	qator ko'paygani sari sekinlashadi	maxsus tuzilmalar tufayli soniyada topadi
Xatodan himoya	istalgan katakka istalgan narsa yoziladi	qoida o'rnatasiz: "yil ustuniga faqat son"

Qisqasi: Excel — shaxsiy daftarcha, ma'lumotlar bazasi — butun kutubxonaning tartibli arxivi.

Jadval, qator, ustun

Baza ichida ma'lumot **jadval** (table) ko'rinishida saqlanadi — Excel'dagi varaq kabi:

JADVAL: kitoblar

id	nomi	muallif	yil
1	O'tkan kunlar	A. Qodiriy	1925
2	Mehrobdan chayon	A. Qodiriy	1928
3	Sariq devni minib	X. To'xtaboyev	1968

← USTUNLAR (column)
← QATOR (row)
← QATOR
← QATOR

- **Ustun (column)** — ma'lumot turi: nomi, muallif, yil. Har bir ustunning o'z nomi va o'z "qoidasi" bo'ladi (masalan, yil ustuniga faqat son yoziladi)
- **Qator (row)** — bitta yozuv: bitta kitob haqidagi hamma ma'lumot

- **id** — har bir qatorning takrorlanmas raqami (pasport raqami kabi). Ismlar takrorlanishi mumkin (ikki xil Aziz Karimov bo'lishi mumkin), lekin id — hech qachon. Odatda uni baza 1 dan boshlab o'zi avtomatik beradi

Jadval anatomiyasi — kitoblar jadvali

Ustun (column) — ma'lumot turi

id	nomi	muallif	yil
1	O'tkan kunlar	A. Qodiriy	1925
2	Mehrobdan chayon	A. Qodiriy	1928
3	Sariq devni minib	X. To'xtaboyev	1968

id — takrorlanmas raqam
(pasport raqami kabi)

Qator (row)
bitta yozuv

Ustun — ma'lumot turi · Qator — bitta yozuv · id — qatorning takrorlanmas raqami

Bitta bazada odatda bir nechta jadval bo'ladi: kutubxona bazasida mualliflar , kitoblar , azolar , ijaralar — har biri o'z mavzusidagi ma'lumotni saqlaydi. Aynan shu to'rt jadvalli kutubxona bazasini 3-bobda o'z qo'lingiz bilan qurasiz.

SQL nima?

SQL (Structured Query Language — "tuzilgan so'rovlar tili"; "es-kyu-el" yoki "sikvel" deb o'qiladi) — bazaga buyruq berish **tili**. Inglizchaga juda o'xshaydi:

```
SELECT nomi FROM kitoblar WHERE yil > 1950;
```

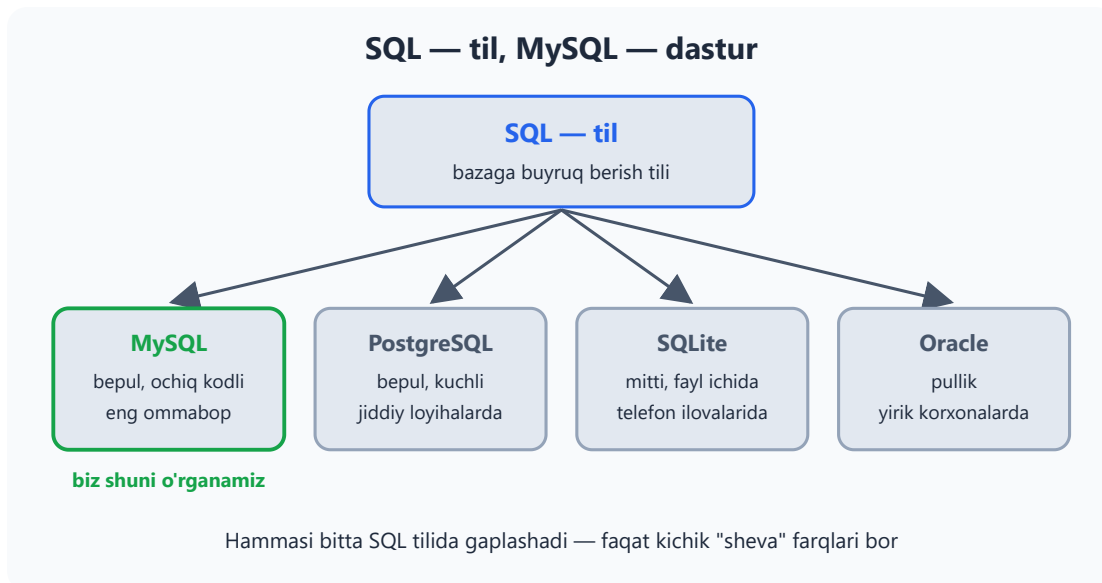
So'zma-so'z: "TANLA nomi[ni] kitoblar[dan] QAYERDA yil > 1950". Tarjimasini: "1950-yildan keyin chiqqan kitoblar nomini ko'rsat".

SQL faqat o'qish uchun emas: ma'lumot **qo'shish** (`INSERT`), **o'zgartirish** (`UPDATE`) va **o'chirish** (`DELETE`) buyruqlari ham bor — hammasini keyingi boblarda birma-bir o'rganamiz.

MySQL nima?

- **SQL** — til (o'zbek tili kabi)
- **MySQL** — shu tilni tushunadigan dastur (o'zbek tilida gaplashadigan odam kabi)

Bunday dasturlarning rasmiy nomi — **MBBT** (ma'lumotlar bazasini boshqarish tizimi, inglizcha DBMS). Boshqa "odamlar" ham bor: PostgreSQL, SQLite, Oracle. Hammasi SQL tilida gaplashadi — faqat kichik "sheva" farqlari bilan. MySQL — dunyodagi eng ommabop ochiq kodli (bepul) baza, biz shuni o'rganamiz.

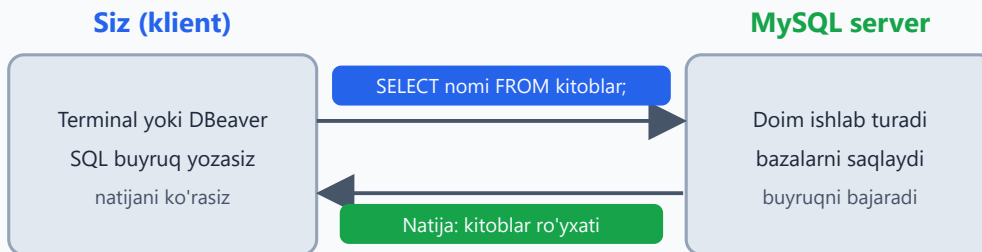


MySQL qanday ishlaydi: klient va server

MySQL kompyuterda **server** bo'lib ishlaydi — doim yoniq turadigan, buyruq kutadigan dastur. Siz esa **klient** orqali (qora oyna — terminal, yoki DBeaver kabi grafik dastur) unga SQL buyruq yuborasiz:

1. Siz buyruq yozasiz: `SELECT nomi FROM kitoblar;`
2. MySQL server buyruqni qabul qilib, bajaradi
3. Natijani jadval ko'rinishida sizga qaytaradi

MySQL qanday ishlaydi: buyruq va natija



1) Buyruq yozasiz → 2) Server bajaradi → 3) Natija qaytadi
Server sizning kompyuteringizda ham, uzoqdagi mashinada ham bo'lishi mumkin

"Server" so'zidan cho'chimang: u sizning oddiy noutbusingizda ham bimalol ishlaydi. Keyingi bobda aynan shuni qilamiz — MySQL'ni o'rnatib, birinchi buyruqlarimizni beramiz.

1-bob masalalari (kompyutersiz, qog'ozda)

1. O'z hayotingizdan 3 ta "jadval bo'lishi mumkin" narsani toping (masalan: telefon kontaktlari)
2. "Telefon kontaktlari" jadvalining ustunlarini yozing (kamida 4 ta)
3. O'sha jadvalga 3 ta qator (misol ma'lumot) yozing
4. Maktab uchun "o'quvchilar" jadvali ustunlarini o'ylab toping
5. "Dorixona" tizimida qanday jadvallar bo'ladi? (kamida 3 ta jadval nomi)
6. "Taksi ilovasi"da qanday jadvallar bo'ladi? (kamida 4 ta)
7. Nega har bir qatorga `id` kerak? O'z so'zingiz bilan tushuntiring
8. Ikki xil odamning ismi bir xil bo'lsa (2 ta Aziz Karimov), baza ularni qanday farqlaydi?
9. Excel va ma'lumotlar bazasining 3 ta farqini yozing
10. `SELECT nomi FROM kitoblar` — bu buyruq nima qiladi? Taxmin qiling
11. `SELECT * FROM kitoblar WHERE yil = 1925` — bu nima qiladi?
12. "Bemorlar" jadvali uchun 6 ta ustun o'ylab toping
13. Kutubxonada "kim qaysi kitobni olgan"ni saqlash uchun qanday jadval kerak? Ustunlarini yozing

14. Bitta muallifning 5 ta kitobi bo'lsa, muallif ismini har safar yozish yaxshimi? Nega?
15. "Online do'kon"da buyurtma jadvalining ustunlarini o'ylab toping
16. Qaysi ma'lumotni jadvalda saqlash NOTO'G'RI: a) mijoz telefoni b) mijozning bugungi kayfiyati c) buyurtma summasi — javobingizni asoslang
17. SQL va MySQL farqini bir gapda yozing
18. O'zingiz bilgan biror ilovani (Telegram, Click, Uzun) oling — unda qanday jadvallar bo'lishi mumkin? 5 ta yozing
19. "Davomat" (kim qaysi kuni kelgan) jadvalini chizing — ustunlar + 3 qator misol
20. Nega katta kompaniyalar Excel emas, baza ishlatadi? 2 ta sabab yozing

02 — MySQL'ni o'rnatish va ishga tushirish

◀ Oldingi: 01 — Ma'lumotlar bazasi nima? · 🏠 README · Keyingi: 03 — Amaliyot bazalarini tayyorlash (4 ta tizim) ▶

Bu bobda: MySQL aslida qanday ishlashini (doim ishlab turadigan server va unga ulanadigan klientlar), uni Windows, macOS va Linux'ga o'rnatishni, terminal hamda DBeaver orqali ulanishni o'rganamiz va birinchi SQL buyruqlarimizni bajarib ko'ramiz.

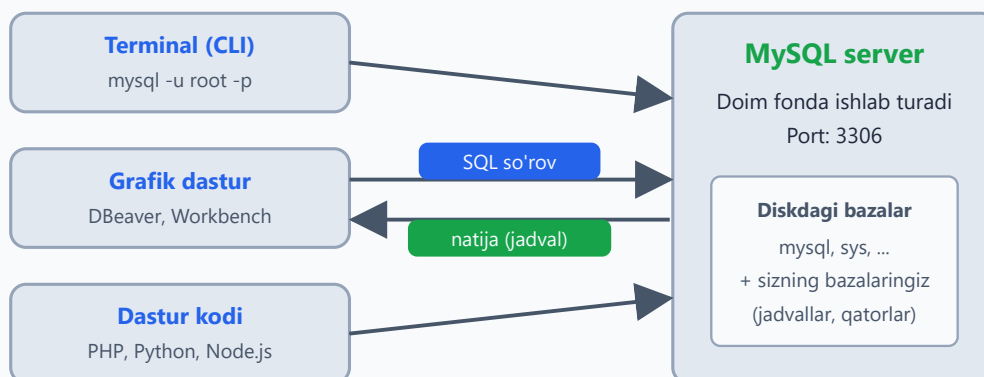
Avval tushunib olaylik: MySQL — bu server

MySQL'ni o'rnatganingizda kompyuteringizga oddiy dastur emas, **server** o'rnatiladi. Server — fonda jim ishlab turadigan dastur: siz uni Word'dek ochib o'tirmaysiz, u doim "navbatchilikda" turadi va kim ulansa, o'shaning so'roviga javob beradi.

Buni call-markazga o'xshatish mumkin: operator doim ish joyida o'tiradi, kim telefon qilsa — javob beradi. MySQL server ham xuddi shunday: u kompyuterda **3306-port**da (serverning "telefon raqami") kutib turadi.

Klient — server bilan gaplashadigan har qanday dastur. Terminal (`mysql` buyrug'i), DBeaver kabi grafik dastur yoki o'zingiz yozgan PHP/Python kod — hammasi klient. Server bitta, klientlar esa istalgancha — hatto bir vaqtning o'zida bir nechta klient ulanib ishlayverishi mumkin.

MySQL arxitekturası: bitta server, ko'p klient



Server bitta — klientlar ko'p, hammasi bir vaqtda ulanishi mumkin

Eng oson yo'l — har bir OS uchun

Windows: [Laragon](#) dasturini yuklab o'rnatish (Full versiyasi). Ichida MySQL tayyor keladi. O'rnatib, "Start All" tugmasini bosish — MySQL server ishga tushadi. Bo'ldi.

✦ Rasmiy yo'l ham bor: [mysql.com](#) saytidan "MySQL Installer for Windows" yuklab o'rnatish. Lekin boshlovchiga Laragon ancha oson — keyinroq xohlasangiz rasmiy usulga o'tib olasiz.

macOS: Terminal'da:

```
brew install mysql
brew services start mysql
```

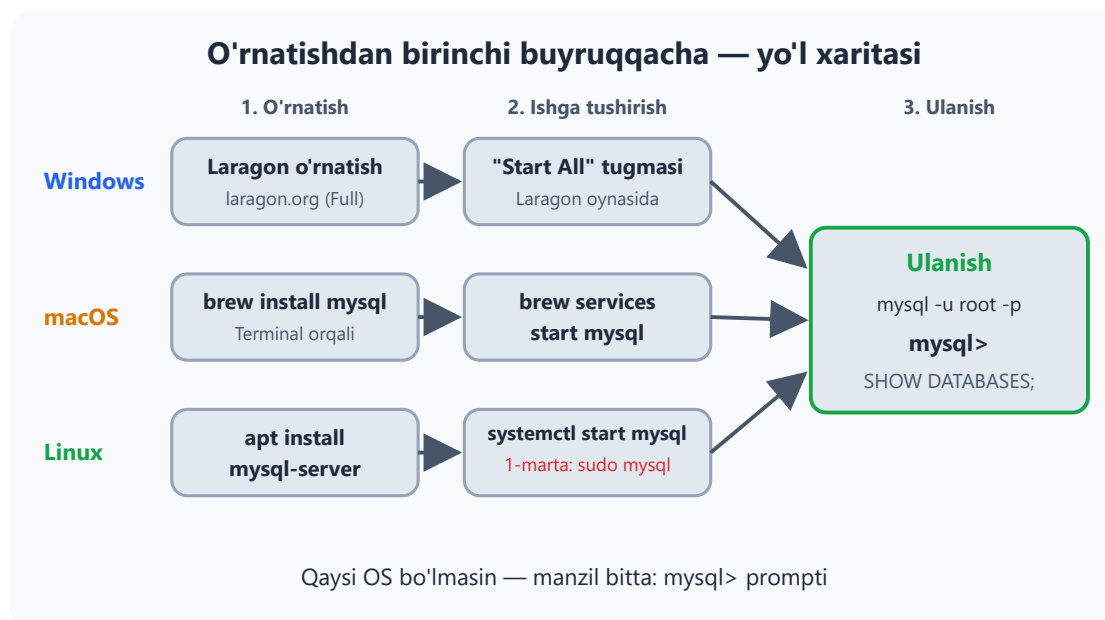
Birinchisi MySQL'ni o'rnatadi, ikkinchisi serverni ishga tushiradi (va kompyuter qayta yonganda ham avtomatik ishga tushadigan qilib qo'yadi). Homebrew o'rnatgan MySQL'da `root` paroli boshida bo'sh bo'ladi.

Linux (Ubuntu):

```
sudo apt update
sudo apt install mysql-server
sudo systemctl start mysql
```

Server ishlayotganini tekshirish: `sudo systemctl status mysql` — yashil **active (running)** yozuvi chiqsa, hammasi joyida.

⚠️ Ubuntu'da bitta nozik joy bor: yangi o'rnatilgan MySQL'da `root` foydalanuvchisi parol bilan emas, tizim foydalanuvchisi orqali kiradi. Shuning uchun birinchi marta `mysql -u root -p` emas, `sudo mysql` deb ulaning. Keyinchalik xohlasangiz `root` 'ga parol qo'yib olasiz (foydalanuvchilar va parollarni 24-bobda batafsil ko'ramiz).



Bazaga ulanish — 2 usul

1-usul: qora oyna (terminal/CMD):

```
mysql -u root -p
```

`mysql` — klient dasturining nomi, `-u root` — "root nomli foydalanuvchi bilan kiranman", `-p` — "parolni so'ra" degani. Laragon'da parol bo'sh (shunchaki Enter bosing). Manzil ko'rsatmadik, chunki klient standart bo'yicha o'z kompyuteringizdagi (`localhost`) serverga, 3306-portga ulanadi.

Muvaffaqiyatli ulansangiz, shunday ko'rinadi:

```
mysql>
```

Bu — "men tayyorman, buyruq bering" degani. Endi siz to'g'ridan-to'g'ri MySQL server bilan gaplashyapsiz.

2-usul: grafik dastur (tavsiya — boshlovchiga osonroq): - **DBeaver** (bepul, hamma OS) — dbeaver.io - **HeidiSQL** (bepul, Windows) - **TablePlus** (chiroyli, macOS/Windows) - **MySQL Workbench** (rasmiy, bepul) — mysql.com

DBeaver'da: New Connection → MySQL → host: `localhost`, user: `root`, parol → Test Connection → Finish.

💡 DBeaver "Public Key Retrieval is not allowed" degan xato bersa, qo'rqmang — bu MySQL 8 bilan tez-tez uchraydi. Ulanish sozlamalarida **Driver properties** bo'limini ochib, `allowPublicKeyRetrieval` ni `TRUE` qiling — ishlab ketadi.

Esda tuting: grafik dastur ham, terminal ham — bitta serverga ulanadigan klientlar, xolos. Qaysi biri qulay bo'lsa, o'shanisini ishlating; kitobdagi misollarni ikkalasida ham bajarsa bo'ladi.

Birinchi buyruqlar

```
SHOW DATABASES;
```

Natija — serverdagi mavjud bazalar ro'yxati:

```
+-----+  
| Database |  
+-----+  
| information_schema |  
| mysql |  
| performance_schema |  
| sys |  
+-----+
```

Bular MySQL'ning o'z xizmat bazalari — server o'zi haqidagi ma'lumotlarni shu yerda saqlaydi, tegmaymiz. O'z bazalarimizni keyingi bobdan boshlab yaratamiz.

```
SELECT NOW();           -- hozirgi sana va vaqt  
SELECT 2 + 2;           -- ha, kalkulyator sifatida ham ishlaydi :)  
SELECT VERSION();       -- MySQL versiyasi (8.0 yoki undan yangi  
bo'lishi kerak)
```

Muhim qoidalar: - Har bir buyruq **nuqta-vergul** ; bilan tugaydi. Unutsangiz, MySQL "buyruq hali davom etyapti" deb kutib turadi — ; terib Enter bossangiz bajariladi - SQL buyruqlari katta-kichik harfga **befarq**: `select` = `SELECT` . Lekin odat bo'yicha buyruqlar KATTA, jadval/ustun nomlari kichik yoziladi — kod o'qishga oson bo'ladi - Izoh (komment) yozish mumkin: `-- bu izoh` (ikki defisdan keyin bo'sh joy bo'lishi shart!) yoki `# bu ham izoh` - Xato yozsangiz — qo'rqmang, MySQL xato xabarini ko'rsatadi, hech narsa buzilmaydi

Tez-tez uchraydigan muammolar

Xato xabari	Sababi	Yechimi
<code>Access denied for user 'root'</code>	Parol noto'g'ri	Laragon'da parol bo'sh; Ubuntu'da <code>sudo mysql</code> deb kiring
<code>Can't connect to MySQL server (2003)</code>	Server ishga tushmagan	Laragon'da "Start All"; Linux'da <code>sudo systemctl start mysql</code>
<code>ERROR 1064 ... your SQL syntax</code>	Buyruqda imlo xatosi	Buyruqni diqqat bilan qayta tering (13-masaladagi kabi)
<code>Public Key Retrieval is not allowed</code>	DBeaver + MySQL 8	Driver properties'da <code>allowPublicKeyRetrieval = TRUE</code>

2-bob masalalari

1. MySQL'ni o'z kompyuteringizga o'rnatish (Laragon yoki boshqa usul)
2. Terminal orqali ulanib, `mysql>` promptini ko'ring
3. DBeaver (yoki HeidiSQL) o'rnatib, ulanish yarating
4. `SHOW DATABASES;` buyrug'ini bajaring — nechta baza bor?
5. `SELECT VERSION();` — versiyangiz qaysi?
6. `SELECT NOW();` — natijani yozib oling
7. `SELECT 7 * 8;` bajaring
8. `SELECT 100 / 3;` — natija qanday chiqdi? Kasr qismiga e'tibor bering
9. `select now();` — kichik harflarda yozing. Ishladimi? Xulosa qiling

10. `SELECT NOW()` — nuqta-vergulsiz yozing. Nima bo'ldi? (terminal kutib turadi — `;` terib Enter bosing)
11. `SELECT 'Salom, SQL!';` bajaring — matn qanday chiqdi?
12. `SELECT 'Mening ismim', 'Aziz';` — ikki ustunli natija oling
13. Ataylab xato yozing: `SELEC NOW();` — xato xabarini o'qing va tushuning
14. `SELECT 10 % 3;` — `%` nima qilarkan? (qoldiq)
15. `SELECT (5 + 3) * 2;` — qavslar ishlaydimi?
16. `SHOW WARNINGS;` buyrug'ini sinab ko'ring
17. `SELECT DATABASE();` — natija nega NULL? O'ylang (hali hech qaysi bazani tanlamadik)
18. Terminal'dan chiqing: `exit` yoki `quit`. Qayta kiring
19. DBeaver'da SQL Editor oching (Ctrl+J) va `SELECT NOW();` ni shu yerdan bajaring
20. `SELECT USER();` — qaysi foydalanuvchi sifatida ulangansiz?

03 — Amaliyot bazalarini tayyorlash (4 ta tizim)

◀ Oldingi: 02 — MySQL'ni o'rnatish va ishga tushirish · 🏠 README · Keyingi: 04 — CREATE — baza va jadval yaratish ▶

Bu bobda: butun kitob davomida ishlatiladigan 4 ta amaliyot bazasini (kutubxona, do'kon, klinika, taksi) yaratib olamiz, har birining jadvallarini va ular orasidagi bog'lanishlarni ER-diagrammalarda ko'rib chiqamiz, skriptlarni bajarib natijani tekshiramiz.

Butun kitob davomida 4 ta real tizim ustida mashq qilamiz. Nega aynan to'rtta? Chunki har xil soha — har xil savol: kutubxonada "kim kitobni qaytarmadi?", do'konda "qaysi mahsulot ko'p sotildi?", klinikada "qaysi shifokor ko'proq daromad keltirdi?", taksida "kimning reytingi yuqori?". Bitta baza bilan tez zerikasiz, to'rttasi bilan esa SQL hayotdagi haqiqiy savollarga javob berishini his qilasiz.

Hozir shu 4 ta skriptni bajarib qo'ying — keyingi boblardagi 400+ masala shularga asoslangan.

Skriptni qanday bajaraman?

- **DBeaver/HeidiSQL'da:** SQL Editor oching, skriptni to'liqligicha nusxalab qo'ying va butun skriptni bajaring (DBeaver'da **Alt+X** — "Execute script"; oddiy Ctrl+Enter faqat kursor turgan bitta buyruqni bajaradi).
- **Terminalda:** `mysql -u root -p` bilan ulaning, skriptni nusxalab qo'ying va Enter bosing — buyruqlar ketma-ket bajariladi.

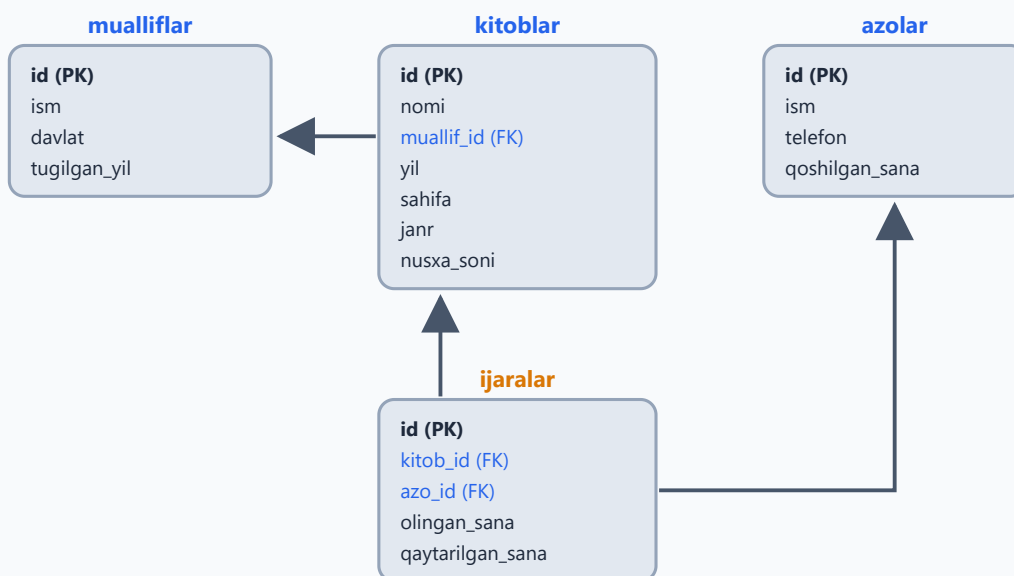
Boshlashdan oldin ikkita kichik izoh:

1. Ma'lumotlar ichida atayin apostrof ishlatmadik (masalan, `'O'zbekiston'` deb yozdik): SQL matnni `'...'` orasiga oladi, matn ichidagi apostrof esa uni chalkashtirib yuboradi. Buni hal qilish usullarini keyinroq ko'ramiz, hozircha shunday qabul qiling.
2. Jadvallar orasidagi bog'lanishni hozircha `FOREIGN KEY` bilan **e'lon qilmadik** — bu mavzu 18-bobda. Hozir bog'lanish "kelishuv" darajasida ishlaydi: masalan, `kitoblar.muallif_id` ustunida `mualliflar` jadvalidagi `id` qiymati turadi. Diagrammalardagi strelkalar aynan shu bog'lanishlarni ko'rsatadi.

Tizim 1: KUTUBXONA

Mahallangizdagi oddiy kutubxonani tasavvur qiling: kitoblar bor, har kitobning muallifi bor, a'zolar kitob olib ketib qaytaradi. Jami 4 jadval: `mualliflar`, `kitoblar`, `azolar` va bog'lovchi `ijaralar` — "kim qaysi kitobni qachon oldi" ro'yxati. `qaytarilgan_sana` ustuni `NULL` bo'lsa — kitob hali qaytarilmagan, kimdadir "yurgan" degani.

KUTUBXONA bazasi — 4 jadval (ER)



```
CREATE DATABASE kutubxona CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
USE kutubxona;
```

```
CREATE TABLE mualliflar (
  id INT AUTO_INCREMENT PRIMARY KEY,
  ism VARCHAR(100) NOT NULL,
  davlat VARCHAR(50),
  tugilgan_yil INT
);
```

```
CREATE TABLE kitoblar (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nomi VARCHAR(200) NOT NULL,
  muallif_id INT,
  yil INT,
  sahifa INT,
  janr VARCHAR(50),
  nusxa_soni INT DEFAULT 1
);
```



```

CREATE TABLE azolar (
    id INT AUTO_INCREMENT PRIMARY KEY,
    ism VARCHAR(100) NOT NULL,
    telefon VARCHAR(20),
    qoshilgan_sana DATE
);

CREATE TABLE ijaralar (
    id INT AUTO_INCREMENT PRIMARY KEY,
    kitob_id INT NOT NULL,
    azo_id INT NOT NULL,
    olingan_sana DATE NOT NULL,
    qaytarilgan_sana DATE NULL
);

INSERT INTO mualliflar (ism, davlat, tugilgan_yil) VALUES
('Abdulla Qodiriy', 'Ozbekiston', 1894),
('Xudoyberdi Toxtaboyev', 'Ozbekiston', 1932),
('Erkin Vohidov', 'Ozbekiston', 1936),
('Lev Tolstoy', 'Rossiya', 1828),
('Jack London', 'AQSH', 1876),
('Antoine de Saint-Exupery', 'Fransiya', 1900);

INSERT INTO kitoblar (nomi, muallif_id, yil, sahifa, janr, nusxa_soni)
VALUES
('Otkan kunlar', 1, 1925, 432, 'roman', 5),
('Mehrobdan chayon', 1, 1928, 380, 'roman', 3),
('Sariq devni minib', 2, 1968, 256, 'sarguzasht', 7),
('Sariq devning olimi', 2, 1973, 288, 'sarguzasht', 2),
('Ozbeqim', 3, 1981, 120, 'shariyat', 4),
('Urush va tinchlik', 4, 1869, 1225, 'roman', 2),
('Anna Karenina', 4, 1877, 864, 'roman', 3),
('Oq soyloq', 5, 1906, 320, 'sarguzasht', 6),
('Kichkina shahzoda', 6, 1943, 96, 'ertak', 10),
('Hayot chorrahalari', 3, 1975, 200, 'shariyat', 1);

INSERT INTO azolar (ism, telefon, qoshilgan_sana) VALUES
('Aziz Karimov', '+998901112233', '2025-01-15'),
('Malika Tosheva', '+998902223344', '2025-02-20'),
('Jasur Aliyev', '+998903334455', '2025-03-10'),
('Nilufar Rahimova', NULL, '2025-05-05'),
('Bobur Saidov', '+998905556677', '2026-01-12'),
('Zilola Ergasheva', '+998906667788', '2026-02-28');

INSERT INTO ijaralar (kitob_id, azo_id, olingan_sana, qaytarilgan_sana)
VALUES
(1, 1, '2026-05-01', '2026-05-15'),
(3, 1, '2026-05-20', NULL),
(9, 2, '2026-05-10', '2026-05-12'),
(6, 3, '2026-04-01', '2026-05-25'),
(3, 4, '2026-06-01', NULL),
(2, 2, '2026-06-03', NULL),

```

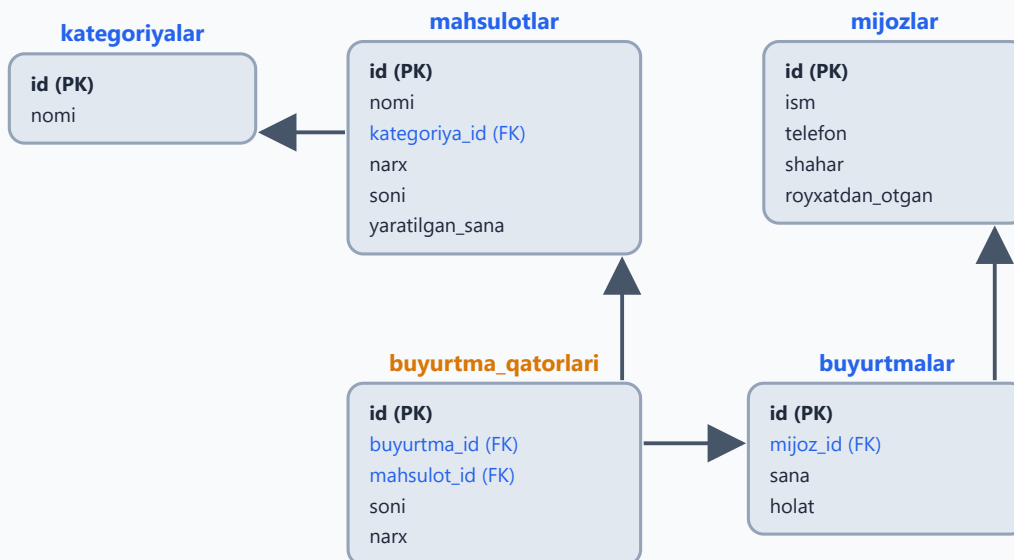
```
(8, 5, '2026-05-28', '2026-06-05'),  
(1, 6, '2026-06-07', NULL);
```

Tekshirib ko'ring: `SELECT COUNT(*) FROM kitoblar;` — 10 chiqishi kerak (`COUNT` qatorlarni sanaydi, uni 11-bobda batafsil o'rganamiz).

Tizim 2: ONLINE DO'KON

Eng katta tizimimiz — 5 jadval. Bu yerda muhim bir g'oya bor: bitta buyurtmada bir nechta mahsulot bo'lishi mumkin. Shuning uchun buyurtma ikkiga bo'lingan: `buyurtmalar` — "chek"ning o'zi (kim, qachon, qanday holatda), `buyurtma_qatorlari` — chekdagi har bir mahsulot qatori (nima, nechta, qancha narxda). Do'kondan chek olganingizda ham xuddi shunday: tepada sana va kassir, pastda mahsulotlar ro'yxati.

DO'KON bazasi — 5 jadval (ER)



PK — asosiy kalit · FK — tashqi kalit · to'q sariq nom — bog'lovchi jadval ("chek qatorlari")

```
CREATE DATABASE dokon CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;  
USE dokon;
```

```
CREATE TABLE kategoriyalar (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    nomi VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE mahsulotlar (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    kategoriya_id INT NOT NULL,  
    nomi VARCHAR(100) NOT NULL,  
    narx DECIMAL(10, 2) NOT NULL,  
    soni INT NOT NULL,  
    yaratilgan_sana DATE NOT NULL
```

```

        id INT AUTO_INCREMENT PRIMARY KEY,
        nomi VARCHAR(200) NOT NULL,
        kategoriya_id INT,
        narx DECIMAL(12,2) NOT NULL,
        soni INT NOT NULL DEFAULT 0,
        yaratilgan_sana DATE
    );

CREATE TABLE mijozlar (
    id INT AUTO_INCREMENT PRIMARY KEY,
    ism VARCHAR(100) NOT NULL,
    telefon VARCHAR(20) UNIQUE,
    shahar VARCHAR(50),
    royxatdan_otgan DATE
);

CREATE TABLE buyurtmalar (
    id INT AUTO_INCREMENT PRIMARY KEY,
    mijoz_id INT NOT NULL,
    sana DATETIME NOT NULL,
    holat ENUM('yangi','yo'lda','yetkazildi','bekor') DEFAULT 'yangi'
);

CREATE TABLE buyurtma_qatorlari (
    id INT AUTO_INCREMENT PRIMARY KEY,
    buyurtma_id INT NOT NULL,
    mahsulot_id INT NOT NULL,
    soni INT NOT NULL,
    narx DECIMAL(12,2) NOT NULL
);

INSERT INTO kategoriyalar (nomi) VALUES
('Telefonlar'), ('Noutbuklar'), ('Aksessuarlar'), ('Maishiy texnika');

INSERT INTO mahsulotlar (nomi, kategoriya_id, narx, soni,
    yaratilgan_sana) VALUES
('Samsung Galaxy A55', 1, 4200000, 15, '2026-01-10'),
('iPhone 15', 1, 11500000, 8, '2026-01-12'),
('Redmi Note 13', 1, 2800000, 25, '2026-02-01'),
('Lenovo IdeaPad 3', 2, 6500000, 10, '2026-01-20'),
('MacBook Air M2', 2, 16000000, 5, '2026-02-15'),
('Acer Aspire 5', 2, 7200000, 0, '2026-03-01'),
('Quloqchin AirPods', 3, 2300000, 30, '2026-01-05'),
('Telefon chexol', 3, 50000, 200, '2026-01-05'),
('Quvvatlagich 65W', 3, 180000, 45, '2026-02-10'),
('Artel muzlatgich', 4, 5400000, 7, '2026-03-15'),
('Samsung kir yuvish', 4, 4800000, 4, '2026-03-20'),
('Changyutgich Xiaomi', 4, 1900000, 12, '2026-04-01');

INSERT INTO mijozlar (ism, telefon, shahar, royxatdan_otgan) VALUES
('Davron Yusupov', '+998911234567', 'Toshkent', '2026-01-15'),
('Sevara Nazarova', '+998912345678', 'Samarqand', '2026-02-01'),
('Otabek Rashidov', '+998913456789', 'Toshkent', '2026-02-20'),
('Gulnoza Hamidova', '+998914567890', 'Buxoro', '2026-03-05'),

```

```
('Sherzod Tursunov', '+998915678901', 'Toshkent', '2026-04-10'),  
('Dilnoza Karimova', '+998916789012', 'Namangan', '2026-05-01');
```

```
INSERT INTO buyurtmalar (mijoz_id, sana, holat) VALUES
```

```
(1, '2026-05-01 10:30:00', 'yetkazildi'),  
(2, '2026-05-03 14:20:00', 'yetkazildi'),  
(1, '2026-05-15 09:00:00', 'yetkazildi'),  
(3, '2026-05-20 16:45:00', 'bekor'),  
(4, '2026-06-01 11:10:00', 'yolda'),  
(5, '2026-06-05 13:30:00', 'yangi'),  
(1, '2026-06-07 18:00:00', 'yangi'),  
(2, '2026-06-08 10:15:00', 'yolda');
```

```
INSERT INTO buyurtma_qatorlari (buyurtma_id, mahsulot_id, soni, narx)  
VALUES
```

```
(1, 3, 1, 2800000), (1, 8, 2, 50000),  
(2, 7, 1, 2300000),  
(3, 4, 1, 6500000), (3, 9, 1, 180000),  
(4, 2, 1, 11500000),  
(5, 10, 1, 5400000),  
(6, 1, 1, 4200000), (6, 8, 1, 50000), (6, 9, 1, 180000),  
(7, 12, 1, 1900000),  
(8, 5, 1, 16000000);
```

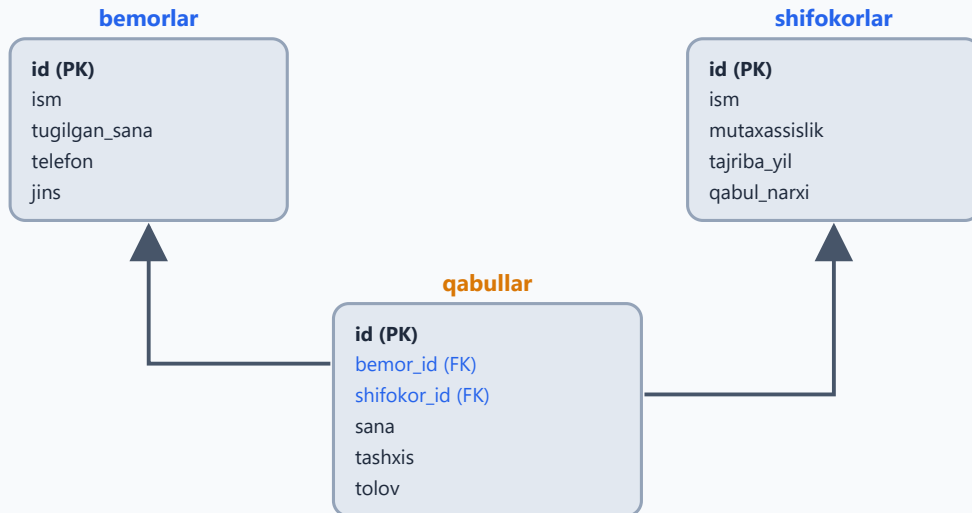
Tekshirib ko'ring: `SELECT COUNT(*) FROM mahsulotlar;` — 12 chiqishi kerak.

Tizim 3: KLINIKA

Klinikada 3 jadval: `shifokorlar`, `bemorlar` va ularni bog'lovchi `qabullar`. Bitta bemor turli shifokorlarga boradi, bitta shifokor ko'p bemorni qabul qiladi — bularning hammasi `qabullar` da yoziladi. E'tibor bering: `to'lov` har doim ham shifokorning `qabul_narxi` ga teng emas — qo'shimcha muolaja bo'lsa, to'lov oshadi (masalan, karies davosi 350 000 turibdi).

KLINIKA bazasi — 3 jadval (ER)

qabullar — bemor va shifokor orasidagi bog'lovchi jadval



PK — asosiy kalit · FK — tashqi kalit · bitta bemor ko'p qabulda, bitta shifokor ko'p qabulda

```
CREATE DATABASE klinika CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
USE klinika;
```

```
CREATE TABLE shifokorlar (
  id INT AUTO_INCREMENT PRIMARY KEY,
  ism VARCHAR(100) NOT NULL,
  mutaxassislik VARCHAR(100) NOT NULL,
  tajriba_yil INT,
  qabul_narxi DECIMAL(10,2)
);
```

```
CREATE TABLE bemorlar (
  id INT AUTO_INCREMENT PRIMARY KEY,
  ism VARCHAR(100) NOT NULL,
  tugilgan_sana DATE,
  telefon VARCHAR(20),
  jins ENUM('erkak','ayol')
);
```

```
CREATE TABLE qabullar (
  id INT AUTO_INCREMENT PRIMARY KEY,
  bemor_id INT NOT NULL,
  shifokor_id INT NOT NULL,
  sana DATETIME NOT NULL,
  tashxis VARCHAR(255),
  tolov DECIMAL(10,2)
);
```

```
INSERT INTO shifokorlar (ism, mutaxassislik, tajriba_yil, qabul_narxi)
VALUES
('Dr. Anvar Sodiqov', 'Terapevt', 15, 100000),
```

```

('Dr. Feruza Olimova', 'Kardiolog', 22, 200000),
('Dr. Rustam Nazarov', 'Stomatolog', 8, 150000),
('Dr. Madina Yoldasheva', 'Pediater', 12, 120000),
('Dr. Botir Hakimov', 'Nevrolog', 5, 180000);

INSERT INTO bemorlar (ism, tugilgan_sana, telefon, jins) VALUES
('Karim Abdullayev', '1965-04-12', '+998931112233', 'erkak'),
('Lola Mirzayeva', '1990-08-25', '+998932223344', 'ayol'),
('Sardor Umarov', '2018-01-30', '+998933334455', 'erkak'),
('Hulkar Tosheva', '1978-11-05', '+998934445566', 'ayol'),
('Akmal Rahmonov', '2001-06-17', NULL, 'erkak'),
('Zarina Qosimova', '1955-03-22', '+998936667788', 'ayol');

INSERT INTO qabullar (bemor_id, shifokor_id, sana, tashxis, tolov)
VALUES
(1, 2, '2026-05-10 09:00:00', 'Gipertoniya', 200000),
(2, 1, '2026-05-12 10:30:00', 'Gripp', 100000),
(3, 4, '2026-05-15 11:00:00', 'Angina', 120000),
(4, 3, '2026-05-20 14:00:00', 'Karies', 350000),
(1, 2, '2026-06-01 09:30:00', 'Nazorat korigi', 200000),
(5, 5, '2026-06-03 15:00:00', 'Migren', 180000),
(6, 2, '2026-06-05 10:00:00', 'Aritmiya', 250000),
(2, 3, '2026-06-08 16:30:00', 'Tish olish', 200000);

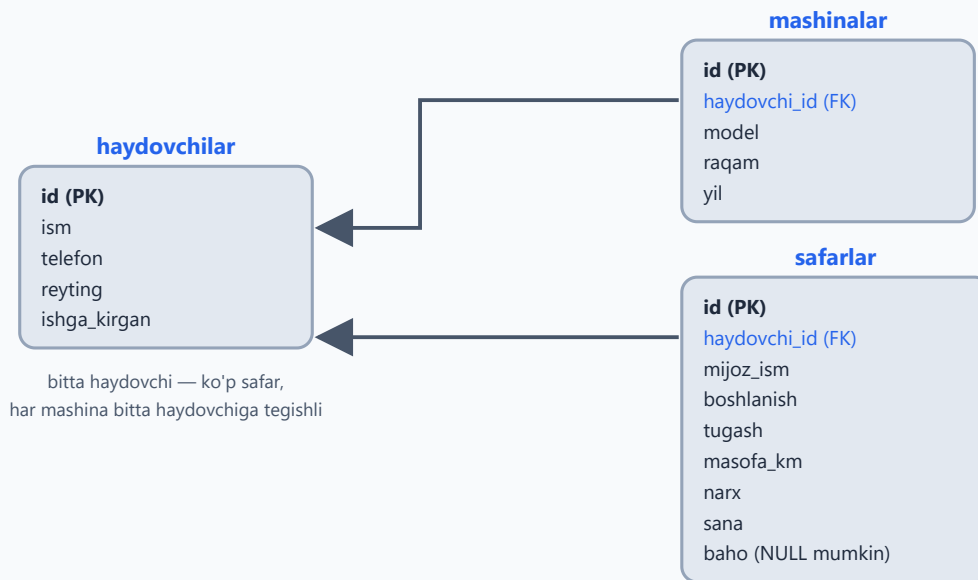
```

Tekshirib ko'ring: `SELECT COUNT(*) FROM qabullar;` — 8 chiqishi kerak.

Tizim 4: TAKSI

Taksi xizmati ham 3 jadval: `haydovchilar`, `mashinalar` va `safarlar` . Ikkita qiziq detalga e'tibor bering. Birinchisi: mijoz alohida jadval emas, `safarlar` da shunchaki `mijoz_ism` ustuni bor — kichik tizimlarda shunday soddalashtirish uchraydi (bu dizaynning kamchiligini 20-bobdagi normalizatsiya ko'zoynagi bilan o'zingiz topasiz). Ikkinchisi: `baho` ustuni `NULL` bo'lishi mumkin — hamma mijoz ham safardan keyin baho qo'yavermaydi, xuddi real ilovalardagidek.

TAKSI bazasi — 3 jadval (ER)



PK — asosiy kalit · FK — tashqi kalit · strelka: FK qaysi jadvalga ishora qilmoqda

```
CREATE DATABASE taksi CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;  
USE taksi;
```

```
CREATE TABLE haydovchilar (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  ism VARCHAR(100) NOT NULL,  
  telefon VARCHAR(20),  
  reyting DECIMAL(3,2) DEFAULT 5.00,  
  ishga_kirgan DATE  
);
```

```
CREATE TABLE mashinalar (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  haydovchi_id INT,  
  model VARCHAR(100),  
  raqam VARCHAR(15) UNIQUE,  
  yil INT  
);
```

```
CREATE TABLE safarlar (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  haydovchi_id INT NOT NULL,  
  mijoz_ism VARCHAR(100),  
  boshlanish VARCHAR(100),  
  tugash VARCHAR(100),  
  masofa_km DECIMAL(6,2),  
  narx DECIMAL(10,2),  
  sana DATETIME,  
  baho INT NULL  
);
```

```

INSERT INTO haydovchilar (ism, telefon, reyting, ishga_kirgan) VALUES
('Olim Toshev', '+998941112233', 4.85, '2024-03-01'),
('Sanjar Qodirov', '+998942223344', 4.95, '2023-07-15'),
('Murod Aliyev', '+998943334455', 4.20, '2025-01-10'),
('Farhod Berdiyev', '+998944445566', 4.70, '2025-06-20'),
('Ulugbek Sattorov', '+998945556677', 5.00, '2026-01-05');

INSERT INTO mashinalar (haydovchi_id, model, raqam, yil) VALUES
(1, 'Chevrolet Cobalt', '01A123BC', 2022),
(2, 'Chevrolet Gentra', '01B456DE', 2023),
(3, 'Chevrolet Spark', '01C789FG', 2019),
(4, 'Chevrolet Malibu', '01D012HI', 2024),
(5, 'BYD Song', '01E345JK', 2025);

INSERT INTO safarlar (haydovchi_id, mijoz_ism, boshlanish, tugash,
masofa_km, narx, sana, baho) VALUES
(1, 'Aziz', 'Chilonzor', 'Yunusobod', 14.5, 35000, '2026-06-01
08:30:00', 5),
(2, 'Malika', 'Sergeli', 'Markaz', 18.2, 42000, '2026-06-01 09:15:00',
5),
(1, 'Jasur', 'Yunusobod', 'Olmazor', 7.3, 20000, '2026-06-02 14:00:00',
4),
(3, 'Nodira', 'Markaz', 'Chilonzor', 9.8, 25000, '2026-06-03 18:45:00',
3),
(4, 'Bobur', 'Olmazor', 'Sergeli', 22.1, 50000, '2026-06-04 07:20:00',
5),
(2, 'Aziz', 'Markaz', 'Yashnobod', 11.0, 28000, '2026-06-05 12:30:00',
NULL),
(5, 'Kamola', 'Yunusobod', 'Markaz', 8.5, 30000, '2026-06-06 16:00:00',
5),
(1, 'Sherzod', 'Chilonzor', 'Sergeli', 19.7, 45000, '2026-06-07
19:10:00', 4),
(3, 'Dilshod', 'Yashnobod', 'Olmazor', 13.4, 32000, '2026-06-08
10:05:00', 2),
(2, 'Malika', 'Markaz', 'Chilonzor', 10.1, 26000, '2026-06-08
20:30:00', 5);

```

Tekshirib ko'ring: `SELECT COUNT(*) FROM safarlar;` — 10 chiqishi kerak.

Hammasi tayyor bo'lsa, oxirgi tekshiruv:

```
SHOW DATABASES;
```



Ro'yxatda `kutubxona`, `dokon`, `klinika`, `taksi` — to'rttasi ham turishi kerak.
 Tabriklaymiz: endi sizda mashq qilish uchun to'laqonli "poligon" bor!

3-bob masalalari

1. 4 ta skriptning hammasini bajaring (`kutubxona` , `dokon` , `klinika` , `taksi`)
2. `SHOW DATABASES;` — 4 ta yangi baza ko'rinyaptimi?
3. `USE kutubxona;` keyin `SHOW TABLES;` — nechta jadval bor?
4. Har bir bazaga kirib jadvallar ro'yxatini yozib oling
5. `DESCRIBE kitoblar;` — bu buyruq nima ko'rsatdi? (jadval tuzilishi)
6. `DESCRIBE` ni `dokon.mahsulotlar` uchun bajaring
7. `SELECT * FROM mualliflar;` — nechta muallif bor?
8. `SELECT * FROM kitoblar;` — natijani ko'zdan kechiring, ustunlarni tushunib oling
9. `SELECT COUNT(*) FROM kitoblar;` — nechta kitob? (COUNT keyinroq o'rganamiz, hozir shunchaki sinang)
10. `dokon` bazasida nechta mahsulot bor? Sanab ko'ring
11. `klinika.shifokorlar` jadvalida qaysi mutaxassisliklar bor?
12. `taksi.safarlar` da baho NULL bo'lgan qator bormi? Toping (ko'z bilan)
13. `ijaralar` jadvalida `qaytarilgan_sana` ustuni NULL bo'lishi nimani anglatadi? O'ylang
14. `buyurtma_qatorlari` dagi `narx` ustuni nega kerak — mahsulotda narx bor-ku? O'ylang (javob: 20-bobda)
15. `SHOW CREATE TABLE kitoblar;` — jadval qanday yaratilganini ko'rsatadi. Sinang
16. Bitta jadvalni ataylab o'chiring: `DROP TABLE taksi.mashinalar;` — keyin skriptdan qayta yarating (qo'rqmang, o'rganish uchun! Avval `USE taksi;` deng, keyin faqat `CREATE TABLE mashinalar` va unga tegishli `INSERT` qismini ko'chirsangiz yetadi — `CREATE DATABASE` ni qayta bajarish shart emas)
17. `USE dokon;` holatida `SELECT * FROM kitoblar;` yozing — qanday xato chiqdi? Nega?
18. To'liq nom bilan murojaat qiling: `SELECT * FROM kutubxona.kitoblar;` — endi ishladimi? Xulosa: baza.jadval
19. Har bir tizimda qaysi jadval "bog'lovchi" vazifasini bajaradi? (masalan, `ijaralar` — kitob va a'zoni bog'laydi)
20. O'zingiz 5-tizim o'ylab toping (masalan: sportzal, mehmonxona) va jadvallarini qog'ozda chizing

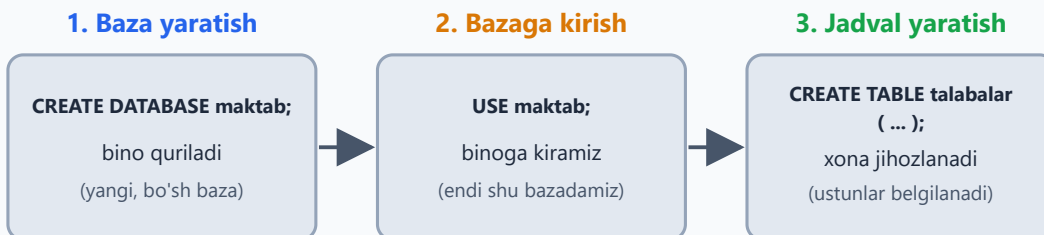
04 — CREATE — baza va jadval yaratish

← Oldingi: 03 — Amaliyot bazalarini tayyorlash (4 ta tizim) · 🏠 README ·
Keyingi: 05 — Ma'lumot turlari →

Bu bobda: SQL'dagi birinchi "qurilish" buyruqlarini o'rganamiz: `CREATE DATABASE` bilan baza yaratamiz, `USE` bilan uning ichiga kiramiz, `CREATE TABLE` bilan jadval quramiz; `PRIMARY KEY`, `AUTO_INCREMENT`, `NOT NULL`, `UNIQUE`, `DEFAULT` qoidalari nimaligini tushunib olamiz va `SHOW`, `DESCRIBE`, `DROP` yordamchi buyruqlari bilan tanishamiz.

Ma'lumotlar bazasini binoga o'xshating: avval **binoning o'zini qurasiz** (`CREATE DATABASE`), keyin **ichiga kirasiz** (`USE`), so'ng **xonalarni jihozlaysiz** (`CREATE TABLE`). Tartib aynan shunday — xonani binosiz qurib bo'lmaydi.

Baza va jadval yaratishning uch qadami



Tartibni buzib bo'lmaydi: jadval yaratishdan oldin `USE` bilan bazaga kiring!

`USE`'siz `CREATE TABLE` yozsangiz: `ERROR 1046 — No database selected`

Baza yaratish — CREATE DATABASE

```
CREATE DATABASE baza_nomi CHARACTER SET utf8mb4 COLLATE  
utf8mb4_unicode_ci;
```

- `CHARACTER SET utf8mb4` — o'zbek, rus, emoji — hammasini saqlaydi. **Har doim shuni yozing.** MySQL 8.0 da bu allaqachon standart, lekin ochiq yozish — yaxshi odat: kodingiz eski sozlangan serverda ham to'g'ri ishlaydi

- `COLLATE utf8mb4_unicode_ci` — harflarni solishtirish qoidasi. Oxiridagi `ci` (case-insensitive) — qidiruvda katta-kichik harf farqlanmaydi: `Aziz = aziz`
- Baza nomida bo'shliq bo'lmasin: `mening bazam` ❌ → `mening_bazam` ✅
- Kichik harf va pastki chiziq ishlating, nom raqam bilan boshlanmasin: `1baza` ❌ → `baza1` ✅

Agar shu nomli baza allaqachon mavjud bo'lsa, `CREATE DATABASE` xato beradi. Skriptlarda buni `IF NOT EXISTS` bilan yumshatish mumkin — "bo'lmasa yarat, bo'lsa indama":

```
CREATE DATABASE IF NOT EXISTS maktab CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
```

Bazaga kirish — USE

Bazani yaratish — bu hali uning ichiga kirish degani emas. Uyni qurib, hali kalitni qo'lga olmaganleksiz. Jadval yaratishdan **oldin** qaysi bazada ishlamoqchi ekaningizni aytib qo'yishingiz kerak:

```
SHOW DATABASES;      -- serverda qanday bazalar bor?
USE maktab;           -- endi "maktab" ichidamiz
SELECT DATABASE();    -- hozir qaysi bazadaman? (tekshirish)
```

💡 Boshlovchilarning klassik xatosi: `USE` 'siz to'g'ridan-to'g'ri `CREATE TABLE` yozish. Natija — `ERROR 1046: No database selected`. Bu xato chiqsa, bilingki — siz hali hech qaysi bazaga "kirmagansiz".

Jadval yaratish — CREATE TABLE

```
CREATE TABLE jadval_nomi (
    ustun_nomi TUR [qoidalar],
    ustun_nomi TUR [qoidalar],
    ...
);
```

Har bir ustun uch qismdan iborat: **nomi** (siz tanlaysiz), **turi** (qanday ma'lumot saqlanadi — 5-bobda batafsil) va ixtiyoriy **qoidalar** (cheklovlar). Ustunlar orasiga vergul qo'yiladi, lekin **oxirgi ustundan keyin vergul yo'q** — bu eng ko'p uchraydigan sintaksis xatosi!



Misol — bosqichma-bosqich tahlil

```
CREATE TABLE talabalar (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  ism VARCHAR(100) NOT NULL,  
  yosh INT,  
  email VARCHAR(150) UNIQUE,  
  kurs INT DEFAULT 1  
);
```

Har bir qatorni o'qiymiz:

Yozuv	Ma'nosi
INT	butun son saqlaydi
AUTO_INCREMENT	har yangi qatorga raqamni o'zi beradi: 1, 2, 3...
PRIMARY KEY	asosiy kalit — takrorlanmas, bo'sh bo'lmaydi

Yozuv	Ma'nosi
<code>VARCHAR(100)</code>	100 belgigacha matn
<code>NOT NULL</code>	bo'sh qoldirib bo'lmaydi
<code>UNIQUE</code>	takrorlanishi mumkin emas (2 ta bir xil email yo'q)
<code>DEFAULT 1</code>	qiymat berilmasa, avtomatik 1 yoziladi

E'tibor bering: `yosh` ustunida hech qanday qoida yo'q — demak u bo'sh (`NULL`) qolishi mumkin. `NOT NULL` — "majburiy maydon" degani: ismsiz talabani saqlab bo'lmaydi. `DEFAULT` esa "aytmasangiz ham o'zim bilaman" degani: yangi talaba odatda 1-kursda o'qiydi.

PRIMARY KEY + AUTO_INCREMENT — ajralmas juftlik

`PRIMARY KEY` — jadvaldagi har bir qatorning **pasport raqami**: ikki qatorda bir xil bo'lmaydi va bo'sh qolmaydi. Nega bu kerak? Jadvalda ikkita "Aziz Karimov" o'qisa, ularni faqat id orqali farqlaysiz. Keyinroq jadvallarni bog'lashda (12-bob, JOIN) ham aynan shu id'lar ishlatiladi.

`AUTO_INCREMENT` esa shu raqamlarni qo'lda bermaslik uchun: MySQL ichida hisoblagich saqlaydi va har yangi qatorga keyingi raqamni o'zi yozib qo'yadi. Shuning uchun deyarli har bir jadvalda `id INT AUTO_INCREMENT PRIMARY KEY` qatorini ko'rasiz — bu eng keng tarqalgan andoza, yodlab oling.

PRIMARY KEY + AUTO_INCREMENT qanday ishlaydi

Siz yozasiz — id'siz!

```
INSERT INTO talabalar (ism)
VALUES ('Aziz');
INSERT ... ('Malika');
INSERT ... ('Jasur');

id ustunini umuman yozmadik
```

Jadvalda — id tayyor!

id	ism
1	Aziz
2	Malika
3	Jasur

Hisoblagich: keyingi id = 4

PRIMARY KEY — pasport raqami kabi: takrorlanmaydi, bo'sh bo'lmaydi

AUTO_INCREMENT — hisoblagich: har yangi qatorga keyingi raqamni o'zi beradi

O'chirilgan raqam qayta berilmaydi: 3-qator o'chsa ham, keyingi yangi qator baribir 4

💡 Qator o'chirilsa, uning raqami **qayta ishlatilmaydi**: 3-qatorni o'chirsangiz, keyingi yangi qator baribir 4 bo'ladi. Bu xato emas — pasport raqami ham boshqa odamga qaytadan berilmaydi.

Yordamchi buyruqlar

```
SHOW TABLES;           -- jadvallar ro'yxati
DESCRIBE talabalar;       -- jadval tuzilishi (qisqasi: DESC
talabalar;)
SHOW CREATE TABLE talabalar; -- to'liq CREATE kodi
DROP TABLE talabalar;     -- jadvalni O'CHIRISH (ehtiyot
bo'ling!)
DROP TABLE IF EXISTS talabalar; -- bo'lsa o'chir, bo'lmasa xato berma
DROP DATABASE IF EXISTS sinov; -- butun bazani o'chirish (sinov
bazasini masalalarda yaratamiz)
```

DESCRIBE — eng yaqin do'stingiz: qaysi ustun majburiy-u (NULL ustunida NO), qaysi biri kalit (Key ustunida PRI) ekanini bir qarashda ko'rsatadi. DROP esa qaytarib bo'lmas buyruq: jadval bilan birga **ichidagi barcha ma'lumot ham yo'qoladi** va MySQL "rostdan o'chirasizmi?" deb so'rab o'tirmaydi.

Tez-tez uchraydigan xatolar

Xato xabari	Sababi
ERROR 1046: No database selected	USE baza_nomi; ni unutdingiz
ERROR 1064: ...syntax error...	ko'pincha vergul aybdor: yetishmaydi yoki oxirgi ustundan keyin ortiqcha turibdi
ERROR 1050: Table ... already exists	bunday jadval allaqachon bor — DROP qiling yoki IF NOT EXISTS yozing
ERROR 1060: Duplicate column name	ikkita ustunga bir xil nom berilgan
ERROR 1068: Multiple primary key defined	PRIMARY KEY jadvalda faqat bitta bo'ladi

Xato xabaridan qo'rqmang — uni **o'qing**. MySQL ko'pincha muammo aynan qaysi qatorda ekanini ham aytib beradi.

4-bob masalalari

Yangi sinov bazasi yarating va hammasini shu yerda qiling: CREATE DATABASE sinov; USE sinov;

- oquvchilar jadvali yarating: id (PK, auto), ism (matn 100, majburiy), sinf (butun son)
- DESCRIBE oquvchilar; bilan tekshiring — har bir ustunning turi va NULL qiymatiga e'tibor bering
- fanlar jadvali: id, nomi (majburiy, takrorlanmas)
- baholar jadvali: id, oquvchi_id, fan_id, baho (butun son), sana (DATE)
- xodimlar jadvali: id, ism, maosh (DECIMAL(10,2)), ishga_kirgan (DATE)
- Jadval yaratishda NOT NULL'siz telefon ustuni qo'shing — farqini DESCRIBE'da ko'ring (NULL ustunida YES)
- DEFAULT sinovi: mehmonlar jadvali yarating, davlat VARCHAR(50) DEFAULT 'O'zbekiston' — apostrofni atayin tushirdik (3-bobdagi kelishuv): 'O'zbekiston' deb yozsangiz, string ichidagi apostrof sintaksisni buzadi; yechimini keyinroq ko'ramiz
- UNIQUE sinovi: userlar jadvali yarating, login VARCHAR(50) UNIQUE

9. Ataylab xato: ikkita ustunga bir xil nom bering — xato xabarini o'qing
10. Ataylab xato: `PRIMARY KEY` ni ikki ustunga alohida-alohida yozing — nima dedi?
11. Jadval nomini bo'shliq bilan yarating: `CREATE TABLE mening jadvalim (...)` — xato chiqdimi? Backtick bilan sinang: ``mening jadvalim`` (lekin bunday qilmang!)
12. `DROP TABLE mehmonlar;` — o'chiring, `SHOW TABLES` bilan tekshiring
13. `DROP TABLE mehmonlar;` ni QAYTA bajaring — qanday xato? Endi `DROP TABLE IF EXISTS mehmonlar;` — farqi?
14. Sportzal uchun `azolar` jadvalini O'ZINGIZ loyihalang (kamida 5 ustun) va yarating
15. Mehmonxona uchun `xonalar` jadvali: raqami, qavat, narx, band (BOOLEAN)
16. `BOOLEAN` ustunli jadvalni DESCRIBE qiling — MySQL uni qanday ko'rsatdi? (`tinyint(1)` — bu normal)
17. Restoran menyusi jadvalini yarating: taom nomi, kategoriya, narx, mavjudligi
18. Avtosalon: `mashinalar` jadvali — model, rang, yil, narx, sotilganmi
19. `sinov` bazasidagi barcha jadvallarni bitta-bitta DROP qiling
20. `DROP DATABASE sinov;` qilib, bazani qaytadan yarating — endi bu jarayon sizga tanish!

05 — Ma'lumot turlari

← Oldingi: 04 — CREATE — baza va jadval yaratish · 🏠 README · Keyingi: 06 — INSERT — ma'lumot kiritish →

Bu bobda: MySQL'dagi asosiy ma'lumot turlarini — sonlar (INT, DECIMAL, FLOAT), matnlar (CHAR, VARCHAR, TEXT), sana-vaqt (DATE, DATETIME, TIMESTAMP) va maxsus turlarni (ENUM, BOOLEAN, JSON) o'rganamiz; har bir ustunga to'g'ri tur tanlashni va pul nima uchun har doim DECIMAL'da saqlanishini ko'rib chiqamiz.

To'g'ri tur — to'g'ri poydevor. Har bir ustunga "bu yerda NIMA saqlanadi?" deb savol bering.

Bu xuddi oshxonada mahsulotga idish tanlashga o'xshaydi: guruch — qopda, yog' — shishada, tuz — tuzdonda. Har bir mahsulotga o'z idishi bor. Bazada ham shunday: har bir ma'lumotga o'z turi bor. Noto'g'ri "idish" tanlasangiz — yo joy isrof bo'ladi, yo ma'lumotning o'zi buziladi.

MySQL ma'lumot turlari xaritasi

Sonlar

TINYINT, INT, BIGINT — butun
DECIMAL(M,D) — pul, aniq kasr
FLOAT / DOUBLE — taxminiy kasr
narx DECIMAL(12,2), yosh TINYINT

Matnlar

CHAR(N) — qat'iy uzunlik
VARCHAR(N) — eng ko'p ishlatiladi
TEXT — katta matn (maqola)
ism VARCHAR(100), kod CHAR(2)

Sana va vaqt

DATE — 2026-06-09
TIME — 14:30:00
DATETIME — sana + vaqt birga
TIMESTAMP — vaqt mintaqasini biladi

Maxsus turlar

ENUM — ro'yxatdan faqat bittasi
BOOLEAN — 0 yoki 1
JSON — strukturali ma'lumot
holat ENUM('yangi','tugadi')

Har bir ustunga savol bering: bu yerda NIMA saqlanadi?

Sonlar

Tur	Diapazon	Qachon	Misol
TINYINT	-128..127 (UNSIGNED: 0..255)	yosh, status	yosh TINYINT UNSIGNED
SMALLINT	±32 767	qavat, o'rinlar soni	qavat SMALLINT
INT	±2.1 milliard	oddiy butun sonlar	sahifa INT
BIGINT	±9.2 kvintillion	juda katta sonlar, ID'lar	id BIGINT
DECIMAL(M,D)	jami M raqam, D tasi kasrda	PUL va aniq kasrlar	narx DECIMAL(12,2)
FLOAT/DOUBLE	taxminiy kasrlar	ilmiy o'lchovlar	harorat FLOAT

UNSIGNED — "ishorasiz" degani: manfiy qiymat taqiqlanadi, evaziga musbat chegara ikki baravar oshadi. Yosh, ombordagi miqdor kabi hech qachon manfiy bo'lmaydigan butun sonlarga juda mos.

DECIMAL(12,2) o'qilishi: jami 12 raqam, shundan 2 tasi kasrdan keyin → maksimum 9 999 999 999.99.

OLTIN QOIDA: pul = DECIMAL. Hech qachon FLOAT emas!

```
-- Nega? Sinab ko'ring:
SELECT 0.1 + 0.2;           -- 0.3 (MySQL buni aniq DECIMAL
sifatida hisoblaydi)
SELECT 0.1e0 + 0.2e0;      -- 0.30000000000000004 (e0 qo'shilsa
son DOUBLE — taxminiy suzuvchi nuqtali songa aylanadi!)
```

FLOAT kompyuterda ikkilik (binar) kasr sifatida taxminan saqlanadi: 0.1 ni ikkilikda aniq yozib bo'lmaydi — xuddi 1/3 ni o'nlikda aniq yozib bo'lmagani kabi (0.3333...). Pulda "taxminan" degani — mijoz pulining yo'qolishi degani. Million tranzaksiyada shu mayda farqlar yig'ilib, hisobotdagi haqiqiy kamomadga aylanadi.

Pul uchun nega DECIMAL? 0.1 + 0.2 sinovi

DECIMAL — aniq ✓

SELECT 0.1 + 0.2;

= 0.3

raqamlar aynan saqlanadi

narx DECIMAL(12,2)

FLOAT — taxminiy ✗

SELECT 0.1e0 + 0.2e0;

= 0.30000000000000004

ikkilikda 0.1 aniq ifodalanmaydi

million amalda — yo'qolgan tiyinlar

OLTIN QOIDA: pul = DECIMAL. Hech qachon FLOAT emas!

FLOAT faqat taxminiy o'lchovlar uchun: harorat, sensor, ilmiy hisob

Matnlar

Tur	Hajm	Qachon
CHAR(N)	doim aniq N belgi	viloyat kodi CHAR(2), valyuta CHAR(3)
VARCHAR(N)	N gacha belgi	ism, telefon, email — eng ko'p ishlatiladi
TEXT	65 535 baytgacha (~64 KB)	maqola, izoh
LONGTEXT	4 GB gacha	juda katta matn (kamdan-kam kerak)

Farqi nimada? CHAR(10) har doim 10 belgilik joy egallaydi: 'abc' yozsangiz, qolgan 7 joyni probel bilan to'ldiradi (o'qiyotganda esa bu probellarni o'zi olib tashlaydi). VARCHAR(10) esa faqat kerakli joyni oladi: 'abc' uchun 3 belgi + uzunlikni eslab qolish uchun 1 bayt. Shuning uchun uzunligi o'zgarib turadigan matnlarga VARCHAR, doim bir xil uzunlikdagi kodlarga CHAR mos.

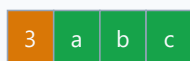
CHAR, VARCHAR va TEXT — joyni qanday egallaydi?

CHAR(10)



doim 10 joy band
(qolganini to'ldiradi)

VARCHAR(10)



uzunlik + 3 belgi

faqat kerakli joy
(joy tejaladi)

TEXT

Uzun maqola, izoh... (~64 KB gacha)

katta matnlar uchun
(hajmi baytda o'lchanadi)

Qoida: oddiy matn → VARCHAR(N) · qat'iy uzunlikdagi kod → CHAR(N)
maqola, katta izoh → TEXT

Hajmni real ehtiyojga qarab tanlang: telefon uchun VARCHAR(20) yetadi, VARCHAR(255) shart emas.

✦ TEXT hajmi belgida emas, **baytda** o'lchanadi: utf8mb4 kodlashda bitta belgi 4 baytgacha joy olishi mumkin, shuning uchun sig'adigan belgilar soni 65 535 dan kamroq bo'lishi mumkin.

Sana va vaqt

Tur	Format	Diapazon	Misol
DATE	YYYY-MM-DD	1000..9999-yillar	tug'ilgan kun
TIME	HH:MM:SS	—	dars vaqti
DATETIME	YYYY-MM-DD HH:MM:SS	1000..9999-yillar	buyurtma vaqti
TIMESTAMP	DATETIME kabi	1970..2038-yillar	yaratilgan vaqt

DATETIME siz yozgan vaqtni "qotirib" saqlaydi. TIMESTAMP esa vaqtni UTC'da saqlab, o'qiyotganda sessiyaning vaqt mintaqasiga moslab ko'rsatadi — server boshqa mamlakatga ko'chsa ham vaqt to'g'ri qoladi. Lekin chegarasi tor: faqat 1970–2038 yillar (mashhur "2038-yil muammosi"). Tug'ilgan kun kabi eski sanalarga DATE / DATETIME ishlatilg.

Yozuv qachon yaratilganini avtomatik saqlash — juda keng tarqalgan usul:

```
yaratilgan TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

Sana **har doim** '2026-06-09' ko'rinishda — YYYY-MM-DD tartibida, bitta tirnoq (apostrof) ichida yoziladi. MySQL '2026/06/09' kabi boshqa ajratgichlarni ham tushunadi, lekin tartib doim yil-oy-kun bo'lishi shart. Odat qiling: faqat YYYY-MM-DD.

Hozirgi sana-vaqtни so'rash uchun tayyor funksiyalar bor (10-bobda batafsil ko'ramiz):

```
SELECT NOW();      -- hozirgi sana va vaqt: 2026-06-09 14:30:00
SELECT CURDATE();  -- bugungi sana: 2026-06-09
SELECT CURTIME();  -- hozirgi vaqt: 14:30:00
```

Maxsus turlar

```
holat ENUM('yangi', 'jarayonda', 'tugadi')  -- faqat shu 3 qiymatdan biri
faol BOOLEAN                               -- 0 yoki 1 (TRUE/FALSE)
sozlamalar JSON                             -- JSON ma'lumot
```

- **ENUM** — ro'yxatdan tashqari qiymat kiritsangiz, MySQL 8 xato beradi. Ehtiyot bo'ling: keyinchalik yangi qiymat qo'shish uchun **ALTER TABLE** kerak bo'ladi, shuning uchun tez-tez o'zgaradigan ro'yxatlarga ENUM emas, alohida jadval yaxshiroq.
- **BOOLEAN** — aslida **TINYINT(1)** ning boshqa nomi: **TRUE** 1 bo'lib, **FALSE** 0 bo'lib saqlanadi. **SELECT**'da 1 yoki 0 ko'rinadi — bu normal.
- **JSON** — MySQL kiritilgan matn haqiqiy JSON ekanini tekshiradi: format noto'g'ri bo'lsa, xato olasiz.

5-bob masalalari

sinov bazasida ishlang.

1. Quyidagilarga tur tanlang (yozing): odam yoshi, kitob narxi, telefon raqami, tug'ilgan sana, maqola matni
2. Quyidagilarga tur tanlang: mahsulot soni omborda, mashina raqami, e-mail, ro'yxatdan o'tgan vaqt (sana+soat), reyting (4.85 kabi)
3. `SELECT 0.1 + 0.2;` va `SELECT 0.1e0 + 0.2e0;` — ikkalasini bajarib, farqni o'z ko'zingiz bilan ko'ring
4. `tovarlar` jadvali yarating: narx DECIMAL(10,2) bilan. `INSERT` qilib 19999.99 kiritib, `SELECT` bilan ko'ring
5. O'sha jadvalga 999999999.99 kiritib ko'ring (sig'adimi? DECIMAL(10,2) = max 8 ta butun raqam!) — xatoni o'qing
6. `VARCHAR(5)` ustunli jadval yarating, 'Salomlar' (8 belgi) kiritib ko'ring — nima bo'ldi?
7. `CHAR(10)` va `VARCHAR(10)` ustunli jadval yarating, ikkalasiga 'abc' kiritib, `SELECT LENGTH(ustun)` bilan uzunliklarni solishtiring. Kutilmagan natija: ikkalasi ham 3! Sababi — MySQL CHAR'ni o'qiyotganda oxiriga qo'shilgan probellarni avtomatik olib tashlaydi
8. `DATE` ustunli jadvalga '2026-13-45' kiritib ko'ring — MySQL nima dedi?
9. '09.06.2026' (kun-oy-yil tartibi) kiritib ko'ring — qabul qildimi? Xulosa: tartib faqat YYYY-MM-DD
10. ENUM sinovi: `holat ENUM('yangi', 'eski')` ustuniga 'ortacha' kiritib ko'ring
11. BOOLEAN ustun yarating, TRUE kiritib, `SELECT` qiling — nima ko'rinadi? (1)
12. TINYINT ustunga 300 kiritib ko'ring — xato yoki ogohlantirishni o'qing
13. TINYINT UNSIGNED ga -5 kiritib ko'ring
14. Bankomat tizimi uchun `kartalar` jadvalini loyihalang: karta raqami, egasi, balans, amal qilish muddati — har turni asoslang
15. Ob-havo jadvali: shahar, sana, harorat (manfiy ham bo'ladi!), namlik foizi — turlarni tanlang
16. `SELECT NOW();` natijasi qaysi turga mos: DATE, TIME yoki DATETIME?
17. `SELECT CURDATE();` va `SELECT CURTIME();` — natijalarini solishtiring
18. JSON ustunli jadval yarating: `malumot JSON . INSERT ... VALUES ('{"rang": "qizil"}')` kiritib
19. Telefon raqamni INT'da saqlasa nima muammo? (boshidagi 0 va + yo'qoladi, 998... INT'ga sig'maydi) — VARCHAR'da saqlab tekshiring
20. O'zingiz ishlatadigan biror ilovaning bitta jadvalini tasavvur qilib, 8 ustunga tur tanlang va yarating

06 — INSERT — ma'lumot kiritish

◀ Oldingi: 05 — Ma'lumot turlari · 🏠 README · Keyingi: 07 — SELECT — ma'lumot o'qish ▶

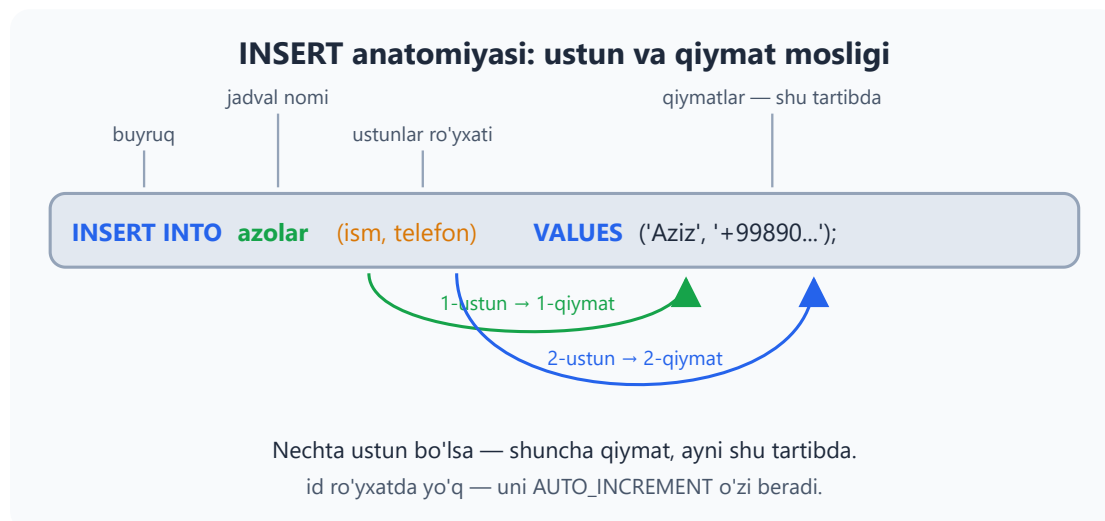
Bu bobda: jadvalga yangi qator qo'shishni o'rganamiz: `INSERT INTO` sintaksisining anatomiyasi, bitta va bir nechta qatorni birdan kiritish, `AUTO_INCREMENT` va `DEFAULT` qanday ishlashi, `NOW()` va `LAST_INSERT_ID()` yordamchilari hamda eng ko'p uchraydigan `INSERT` xatolarini o'qib tushunish.

4-bobda jadval yaratdik — bu xuddi ustunlari chizilgan bo'sh daftar tayyorlash edi. Endi unga birinchi yozuvlarni kiritamiz. Kutubxonaga yangi a'zo keldi, do'konga yangi mahsulot keltirildi, klinikaga yangi bemor yozildi — bularning hammasi SQL tilida bitta buyruq: `INSERT`.

Asosiy shakl

```
INSERT INTO jadval (ustun1, ustun2) VALUES (qiymat1, qiymat2);
```

O'qilishi: "*jadvalga, shu ustunlarga, shu qiymatlarni kirit*". Eng muhim qoida — **moslik**: birinchi ustun birinchi qiymatni oladi, ikkinchi ustun — ikkinchisini. Ustunlar nechta bo'lsa, qiymatlar ham shuncha bo'lishi shart.



Amalda ko'ramiz:

```
USE kutubxona;

-- Bitta qator:
INSERT INTO azolar (ism, telefon, qoshilgan_sana)
VALUES ('Umid Rasulov', '+998907778899', '2026-06-09');
```

E'tibor bering: `id` ustunini umuman yozmadik. Bu xato emas — aksincha, to'g'ri odat.

AUTO_INCREMENT — id'ni MySQL o'zi beradi

`azolar` jadvalini yaratganda `id INT AUTO_INCREMENT PRIMARY KEY` deb yozgan edik. Bu MySQL'ga "har yangi qatorga navbatdagi raqamni o'zing ber" degani. MySQL har bir bunday jadval uchun ichki hisoblagich yuritadi: oxirgi id 3 bo'lsa, keyingi qator 4 ni oladi, hisoblagich esa 5 ga o'tadi. Siz hech qachon "bo'sh id qaysi ekan?" deb o'ylamaysiz.

AUTO_INCREMENT: id'ni MySQL o'zi beradi

azolar jadvali (3-bobdagi seed: 6 qator)

id	ism
1	Aziz Karimov
2	Malika Tosheva
3	Jasur Aliyev
4	Nilufar Rahimova
5	Bobur Saidov
6	Zilola Ergasheva
7	Umid Rasulov

yangi qator — id 7 avtomatik

`SELECT LAST_INSERT_ID();` — hozirgina berilgan id'ni qaytaradi (bu yerda 7).

Hisoblagich faqat oldinga yuradi: qator o'chirilsa ham id qayta ishlatilmaydi.

Shuning uchun id'larda "teshiklar" (1, 2, 5, 9...) bo'lishi normal holat.

Ichki hisoblagich

keyingi id = 7

INSERT'dan keyin: 8

id = 7 ni beradi

INSERT INTO azolar (ism, telefon, ...)
VALUES ('Umid Rasulov', ...); ← id yozilmadi!

Hozirgina berilgan id kerak bo'lsa (masalan, buyurtma yaratib, keyin uning qatorlarini shu id bilan qo'shish uchun):

```
SELECT LAST_INSERT_ID(); -- shu ulanishda oxirgi berilgan id
```


✦ Hisoblagich faqat oldinga yuradi: qatorni o'chirsangiz ham, uning id'si qayta ishlatilmaydi. Shuning uchun id'larda "teshiklar" (1, 2, 5, 9...) bo'lishi mutlaqo normal — id tartib raqami emas, shunchaki har bir qatorning takrorlanmas yorlig'i.

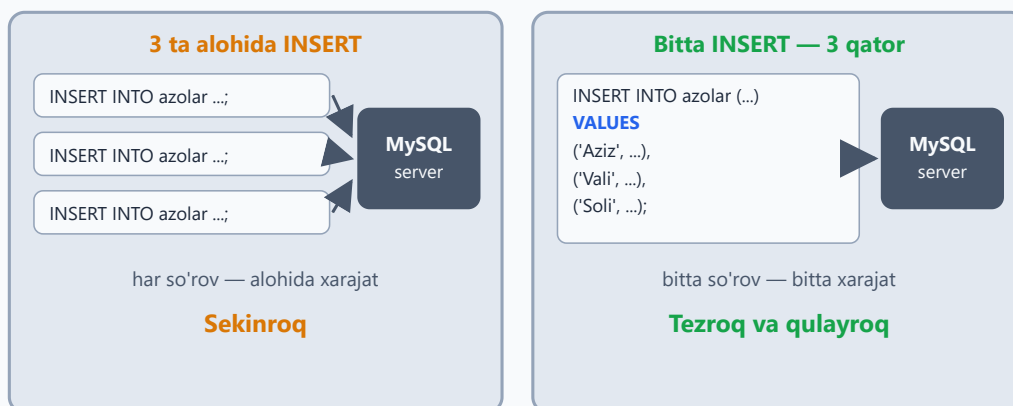
Bir nechta qatorni birdan

Har bir qator uchun alohida INSERT yozish shart emas — qiymat guruhlarini vergul bilan ajratib, hammasini bittada kiritish mumkin:

```
INSERT INTO azolar (ism, telefon, qoshilgan_sana) VALUES  
('Sitora Yusupova', '+998908889900', '2026-06-09'),  
('Akbar Toirov', NULL, '2026-06-09');
```

Bu shunchaki qulay emas — **tezroq** ham: server bitta so'rovni qabul qilib, hammasini bir yo'la yozadi. 1000 ta qatorni 1000 ta alohida INSERT bilan kiritish — do'konga 1000 marta alohida borib, har safar bittadan non olib kelish bilan barobar.

3 ta alohida INSERT yoki bitta INSERT?



Maslahat: bir nechta qator kiritganda bitta INSERT ishlating.
Qavs guruhlari orasida vergul, eng oxirida nuqta-vergul (;).

Sintaksisga e'tibor: har bir qator o'z qavsida, qavs guruhlari **orasida vergul**, eng oxirida — nuqta-vergul.

Qiymat yozish qoidalari

- Matn va sana — bitta tirnoq ichida: 'Aziz', '2026-06-09'

- Sonlar — tirnoqsiz: 25 , 4200000
- Bo'sh qiymat — NULL (tirnoqsiz! 'NULL' deb yozsangiz, u to'rt harfli oddiy matn bo'lib qoladi)
- Hozirgi sana-vaqt — NOW() , faqat bugungi sana — CURDATE() (bular ham tirnoqsiz)
- Ustunlar va qiymatlar **soni va tartibi mos** kelishi shart

✦ SQL standartida matn uchun **bitta tirnoq** (') ishlatiladi. MySQL qo'shtirnoqni (") ham qabul qiladi, lekin boshqa bazalarda (masalan, PostgreSQL'da) qo'shtirnoq butunlay boshqa ma'noni anglatadi — shuning uchun boshidanoq bitta tirnoqqa odatlaning.

Ko'rsatilmagan ustunga nima yoziladi?

Ustunni ro'yxatda tashlab ketsangiz, MySQL shu tartibda qaror qiladi:

Ustun ta'rifida nima bor	Natija
AUTO_INCREMENT	navbatdagi raqam beriladi
DEFAULT qiymat	o'sha standart qiymat yoziladi
NULL ga ruxsat (NOT NULL yo'q)	NULL qoladi
NOT NULL bor, DEFAULT yo'q	xato — qator kiritilmaydi

```
USE dokon;
```

```
INSERT INTO mahsulotlar (nomi, kategoriya_id, narx)
VALUES ('Klaviatura Logitech', 3, 250000);
-- soni ko'rsatilmadi      → DEFAULT 0 yozildi
-- yaratilgan_sana ham yo'q → NULL qoldi (ruxsat bor)
```

Ustunlarsiz shakl — qisqa, lekin xavfli

```
USE kutubxona;
```

```
INSERT INTO azolar VALUES (NULL, 'Test User', '+998900000000', '2026-06-09');
```

Ustunlar ro'yxatini yozmasangiz, **hamma** ustunga, jadvaldagi **ayni tartibda** qiymat berishingiz shart (`id` o'rnida `NULL` — `AUTO_INCREMENT` baribir o'zi raqam beradi). Nega bu xavfli? Ertaga jadvalga bitta yangi ustun qo'shilsa, bu kod darhol buziladi (`Column count doesn't match` xatosi); agar ustunlar qayta tartiblansa yoki bir xil turdagi ikki ustun o'rin almashtirilsa — undan ham yomoni — xato ham chiqmaydi, qiymatlar indamay noto'g'ri ustunlarga tushib qoladi. Real loyihalarda har doim ustunlar ro'yxatini yozing.

Ko'p uchraydigan xatolar

```
USE kutubxona;
```

```
-- NOT NULL ustunni tashlab ketish:
```

```
INSERT INTO azolar (telefon) VALUES ('+99890...');
```

```
-- Xato: Field 'ism' doesn't have a default value
```

```
-- UNIQUE'ni buzish:
```

```
INSERT INTO dokon.mijozlar (ism, telefon) VALUES ('Test', '+998911234567');
```

```
-- Xato: Duplicate entry '+998911234567' for key ... — bu telefon allaqachon bor
```

```
-- Ustun soni mos emas:
```

```
INSERT INTO azolar (ism, telefon) VALUES ('Aziz');
```

```
-- Xato: Column count doesn't match value count at row 1
```

Eng tez-tez uchraydigan xabarlar va ma'nolari:

Xato xabari	Sababi
Field '...' doesn't have a default value	NOT NULL ustun tashlab ketildi
Duplicate entry '...' for key ...	UNIQUE yoki PRIMARY KEY qiymati takrorlandi
Column count doesn't match value count	ustunlar va qiymatlar soni teng emas

Xato xabari	Sababi
Incorrect integer value / Incorrect decimal value	son kutilgan joyga matn kiritildi
Data too long for column '...'	matn VARCHAR(N) chegarasidan uzun

Xato xabarlarini **o'qishni** o'rganing — MySQL aniq qaysi ustunda, qaysi qatorda muammo borligini aytadi. Xato — dushman emas, yo'l ko'rsatkich.

6-bob masalalari

Mos bazada ishlang (har masala boshida ko'rsatilgan).

- (kutubxona) Yangi muallif qo'shing: o'zingiz bilgan yozuvchi
- (kutubxona) O'sha muallifga 2 ta kitob qo'shing (muallif_id ni to'g'ri ko'rsating — `SELECT * FROM mualliflar` bilan id'sini toping)
- (kutubxona) Telefoni yo'q (NULL) a'zo qo'shing
- (kutubxona) 3 ta a'zoni BITTA INSERT bilan qo'shing
- (dokon) Yangi kategoriya qo'shing: 'Kitoblar'
- (dokon) Shu kategoriyaga 3 ta mahsulot qo'shing
- (dokon) `soni` ko'rsatmasdan mahsulot qo'shing — DEFAULT 0 ishladimi? `SELECT` bilan tekshiring
- (dokon) Mavjud telefon raqami bilan mijoz qo'shib ko'ring — UNIQUE xatosini oling
- (klinika) O'zingizni bemor sifatida qo'shing
- (klinika) Yangi shifokor qo'shing va unga bugungi sanada bitta qabul yozing (`NOW()` dan foydalaning: `VALUES (... , NOW(), ...)`)
- (taksi) Yangi haydovchi va uning mashinasini qo'shing (2 ta INSERT)
- (taksi) Baho qo'yilmagan (NULL) safar qo'shing
- (kutubxona) Ataylab xato: `ism` siz a'zo kiriting — xatoni o'qing
- (kutubxona) Ataylab xato: 3 ustun ko'rsatib 2 qiymat bering
- (dokon) Ataylab xato: narxga `'qimmat'` deb matn kiriting — nima bo'ldi?
- (kutubxona) `INSERT INTO azolar VALUES (NULL, 'Test User', '+9989000000000', '2026-06-09');` — ustunlarsiz shakl. NULL o'rnida id

avtomatik berildi. Lekin nega bu usul xavfli? (jadvalga ustun qo'shilsa kod buziladi)

17. (klinika) Bitta INSERT bilan 4 ta bemor qo'shing
18. (taksi) Bugungi sana uchun 3 ta safar qo'shing, NOW() ishlating
19. (dokon) Buyurtma yarating: avval `buyurtmalar` ga, keyin uning id'si bilan `buyurtma_qatorlari` ga 2 ta mahsulot qo'shing. Oxirgi id'ni bilish: `SELECT LAST_INSERT_ID();`
20. (hammasi) Har bir bazaga kamida 2 tadan yangi qator qo'shib, jadvallaringizni "boyitib" oling — keyingi boblarda ko'proq data foydali

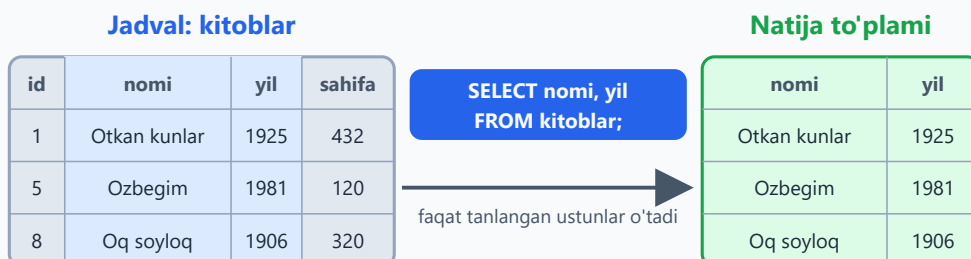
07 — SELECT — ma'lumot o'qish

◀ Oldingi: 06 — INSERT — ma'lumot kiritish · 🏠 README · Keyingi: 08 — WHERE — filtrlash ▶

Bu bobda: SQL'ning yuragi — SELECT bilan jadvaldan ma'lumot o'qishni o'rganamiz: kerakli ustunlarni tanlash, ustunlarga vaqtincha nom berish (AS alias), SELECT ichida hisob-kitob qilish va DISTINCT bilan takror qiymatlarni olib tashlash.

O'tgan bobda jadvallarga ma'lumot kiritdik. Endi eng muhim savol: uni qanday o'qib olamiz? Javob — `SELECT`. Bu SQL'dagi eng ko'p ishlatiladigan buyruq: real tizimlarda so'rovlarning aksariyati aynan o'qish (saytni ochsangiz — `SELECT`, qidirsangiz — `SELECT`, profilingizni ko'rsangiz — yana `SELECT`). Va xotirjam bo'ling: `SELECT` jadvaldagi ma'lumotni **hech qachon o'zgartirmaydi** — faqat o'qiydi.

SELECT qanday ishlaydi



SELECT jadvalni o'zgartirmaydi — faqat o'qiydi va natijani ko'rsatadi

tanlangan ustunlar

natija to'plami

Asosiy shakllar

```
USE kutubxona;
```

```
SELECT * FROM kitoblar;
```

```
SELECT nomi FROM kitoblar;
```

```
SELECT nomi, yil FROM kitoblar;
```

```
-- hamma ustun, hamma qator
```

```
-- bitta ustun
```

```
-- bir nechta ustun
```

```
SELECT nomi AS kitob, yil AS chiqqan_yili FROM kitoblar; -- alias
(vaqtincha nom)
```

O'qilishi oddiy: `SELECT` — "tanla", `FROM` — "qayerdan". Ya'ni: "kitoblar jadvalidan nomi va yil ustunlarini tanla".

Yana ikkita foydali fakt:

- `SELECT` qaytargan narsa **natija to'plami** (result set) deyiladi — bu vaqtinchalik "jadvalcha": ekranga chiqadi, lekin bazada saqlanmaydi.
- Ustunlar **siz yozgan tartibda** chiqadi. `SELECT yil, nomi FROM kitoblar;` desangiz, natijada avval yil, keyin nom turadi — jadvaldagi tartib muhim emas.

SELECT * yoki aniq ustunlar?

* — "hamma ustun" degani. O'rganishda va jadvalga tez ko'z tashlashda juda qulay. Lekin real dasturlarda kerakli ustunlarni sanab yozish yaxshi odat:

	SELECT *	Aniq ustunlar
Nima keladi	hamma ustun, kerak-nokerak	faqat so'raganlaringiz
Tezlik	ortiqcha ma'lumot uzatiladi	yengilroq, tezroq
O'qish	keng va chalkash natija	ixcham, maqsadi aniq
Qachon ishlatamiz	o'rganish, tez tekshirish	real dastur kodi

SELECT * yoki aniq ustunlar?

SELECT * FROM mahsulotlar;

- Hamma ustun keladi — kerak-nokerak
- Keng natija, o'qish noqulay
- Ortiqcha ma'lumot uzatiladi
- Qaysi ustun kerakligi noaniq

O'rganishda — mayli

SELECT nomi, narx FROM mahsulotlar;

- Faqat kerakli ustunlar keladi
- Natija ixcham va tushunarli
- Tezroq ishlaydi
- Kodning maqsadi ko'rinib turadi

Real loyihada — shu yo'l

AS — natijadagi ustunga vaqtincha nom (alias)

SELECT narx AS mahsulot_narxi FROM mahsulotlar;

Natijada ustun sarlavhasi: mahsulot_narxi — jadvaldagi asl nom o'zgarmaydi

AS — ustunga vaqtincha nom (alias)

Alias — natija to'plamidagi ustun sarlavhasiga beriladigan vaqtincha nom. Jadvaldagi asl ustun nomi joyida qoladi — alias faqat shu bitta so'rovning natijasida amal qiladi.

```
SELECT nomi AS kitob_nomi, sahifa AS betlar_soni FROM kitoblar;

-- AS so'zisiz ham ishlaydi, lekin AS bilan o'qish osonroq:
SELECT nomi kitob_nomi FROM kitoblar;

-- Bo'shliqli nom kerak bo'lsa — teskari tirnoq (`) ichiga oling:
SELECT nomi AS `kitob nomi` FROM kitoblar;
```

💡 Alias ayniqsa hisob-kitobli ustunlarda asqotadi — aks holda ustun sarlavhasi `narx * soni` kabi "xunuk" ifoda bo'lib chiqadi.

Hisob-kitob SELECT ichida

SELECT ichida arifmetik amallar ishlatish mumkin: `+`, `-`, `*`, `/` va `%` (qoldiq).


```
SELECT nomi, narx, narx * soni AS jami_qiymat
FROM dokon.mahsulotlar;

SELECT nomi, sahifa, sahifa / 30 AS oqish_kunlari  -- kuniga 30 bet
o'qisak
FROM kitoblar;
```

Bilib qo'ying:

- 🚩 Hisob faqat **natija to'plamida** bajariladi — jadvaldagi `narx` ham, `soni` ham o'z joyida, o'zgarmagan holda qoladi.
- 💡 `/` doim kasr qaytaradi: `432 / 30` → `14.4000`. Faqat butun qismi kerak bo'lsa, `sahifa DIV 30` deb yozing.
- ⚠️ Agar hisobda qatnashayotgan ustunda `NULL` bo'lsa, natija ham `NULL` bo'ladi. Masalan, `taksi.safarlar` da `baho * 2` — baho qo'yilmagan safarlar uchun `NULL` chiqadi.

FROM'siz SELECT — kalkulyator rejimi

MySQL'da jadvalsiz ham SELECT yozsa bo'ladi — kalkulyator kabi:

```
SELECT 2 + 3;           -- 5
SELECT 5 * 8 AS javob;  -- ustun nomi: javob
SELECT NOW();           -- hozirgi sana-vaqt
```

Bu rejim funksiyalarni tez sinab ko'rish uchun juda qulay — keyingi boblarda ko'p ishlatamiz.

DISTINCT — takrorlarsiz

`DISTINCT` natijadagi takror qiymatlarni olib tashlaydi — har qiymat faqat bir marta chiqadi:

```
SELECT DISTINCT janr FROM kitoblar;           -- har janr bir marta
SELECT DISTINCT shahar FROM dokon.mijozlar;
```

Bir nechta ustun bilan yozsangiz, `DISTINCT` ustunlar **kombinatsiyasiga** qaraydi:

```
SELECT DISTINCT boshlanish, tugash FROM taksi.safarlar;
-- boshlanish + tugash JUFTLIGI takrorlanmaydi:
-- 'Markaz → Chilonzor' yo'nalishida ikkita safar bor, natijada bir
marta chiqadi
```

✦ DISTINCT uchun `NULL` ham oddiy qiymat — takror `NULL`lar bitta bo'lib chiqadi.

7-bob masalalari

Har masala qaysi bazada ishlashi qavsda ko'rsatilgan.

1. (kutubxona) Barcha kitoblarni chiqaring
2. (kutubxona) Faqat kitob nomlarini chiqaring
3. (kutubxona) Kitob nomi va yilini chiqaring
4. (kutubxona) Ustunlarga o'zbekcha alias bering: `nomi AS kitob_nomi, sahifa AS betlar_soni`
5. (kutubxona) Barcha janrlarni takrorsiz chiqaring (`DISTINCT`)
6. (kutubxona) Mualliflarning davlatlarini takrorsiz chiqaring
7. (dokon) Mahsulot nomi va narxini chiqaring
8. (dokon) Har mahsulot uchun `narx * soni` ni `ombor_qiymati` aliasi bilan hisoblang
9. (dokon) Mijozlar yashaydigan shaharlarni takrorsiz chiqaring
10. (dokon) Narxni ming so'mda ko'rsating: `narx / 1000 AS ming_somda`
11. (klinika) Shifokorlar ismi va mutaxassisligini chiqaring
12. (klinika) Har shifokor uchun 10 ta qabuldan tushadigan summani hisoblang: `qabul_narxi * 10`
13. (klinika) Bemorlarning takrorsiz jinslarini chiqaring (natija nechta qator bo'ladi?)
14. (taksi) Safarlar: boshlanish, tugash, narx ustunlarini chiqaring
15. (taksi) Har safar uchun 1 km narxini hisoblang: `narx / masofa_km AS km_narxi`
16. (taksi) Haydovchilarga 20% komissiya: haydovchiga qoladigan ulushni hisoblang — `narx * 0.8 AS haydovchi_ulushi`
17. (taksi) Safar boshlanish nuqtalarini takrorsiz chiqaring

18. `SELECT 5;` `SELECT 'matn';` `SELECT 5, 'matn', NOW();` — FROM'siz SELECT ham ishlaydi, sinang
19. (dokon) `SELECT *` va kerakli 2 ustunli SELECT'ni bajarib, natija kengligini solishtiring — qaysi birini o'qish oson?
20. (hammasi) Har bazadan bittadan o'zingiz savol o'ylab, SELECT yozing

08 — WHERE — filtrlash

◀ Oldingi: 07 — SELECT — ma'lumot o'qish · 🏠 README · Keyingi: 09 — ORDER BY, LIMIT, DISTINCT ▶

Bu bobda: WHERE yordamida jadvaldan faqat kerakli qatorlarni ajratib olishni o'rganamiz: taqqoslash operatorlari (= , != , > , <), AND/OR/NOT mantiqiy bog'lovchilari va qavslar, BETWEEN va IN, LIKE bilan naqsh bo'yicha qidirish hamda NULL bilan to'g'ri ishlash (IS NULL).

WHERE — SQL'ning eng ko'p ishlatiladigan qismi. "Hammasi emas, faqat KERAKLILARI" degani. Buni g'alvirga o'xshating: jadvaldagi HAR BIR qator shartdan o'tkaziladi — shart rost bo'lsa, qator natijaga tushadi; yolg'on bo'lsa, elakda qolib ketadi. Muhim jihati: WHERE jadvaldagi ma'lumotni o'chirmaydi, faqat KO'RSATILADIGAN qatorlarni tanlaydi.

```
SELECT ustunlar FROM jadval WHERE shart;
```

WHERE — qatorlarni filtrlash

```
SELECT * FROM kitoblar WHERE yil > 1950;
```

kitoblar — 10 qator

Otkan kunlar — 1925
Mehrobdan chayon — 1928
Sariq devni minib — 1968
Sariq devning olimi — 1973
Ozbegin — 1981
Urush va tinchlik — 1869
Hayot chorrahallari — 1975
... yana 3 qator (mos emas)

WHERE
yil > 1950

Natija — 4 qator

Sariq devni minib — 1968
Sariq devning olimi — 1973
Ozbegin — 1981
Hayot chorrahallari — 1975

Yashil qatorlar shartga mos — faqat shular natijaga o'tadi. Jadvalning o'zi o'zgarmaydi!

Taqqoslash operatorlari

Operator	Ma'nosi	Misol
=	teng	janr = 'roman'
!= yoki <>	teng emas	janr != 'roman'
>	katta	yil > 1950
<	kichik	sahifa < 300
>=	katta yoki teng	yil >= 1950
<=	kichik yoki teng	yil <= 1928

```

SELECT * FROM kitoblar WHERE yil > 1950;      -- katta
SELECT * FROM kitoblar WHERE yil <= 1928;      -- kichik yoki teng
SELECT * FROM kitoblar WHERE janr = 'roman';    -- teng
SELECT * FROM kitoblar WHERE janr != 'roman';    -- teng emas (<> ham shu)

```

✦ Matn qiymatlari doim bitta tirnoq ichida yoziladi: 'roman' . Sonlar tirnoqsiz: 1950 . Tirnoqni unutsangiz (janr = roman), MySQL roman degan USTUN qidiradi va "Unknown column" xatosini beradi.

AND, OR, NOT — shartlarni birlashtirish

Bitta shart kamlik qilsa, ularni mantiqiy bog'lovchilar bilan ulaymiz:

```

-- IKKALA shart ham bajarilsin:
SELECT * FROM kitoblar WHERE janr = 'roman' AND yil > 1900;

-- KAMIDA BITTASI bajarilsin:
SELECT * FROM kitoblar WHERE janr = 'ertak' OR janr = 'shariyat';

-- Shartning TESKARISI:
SELECT * FROM kitoblar WHERE NOT janr = 'roman';    -- roman bo'lmaganlar

```

AND, OR, NOT — mantiq jadvali

AND			OR			NOT	
A	B	A AND B	A	B	A OR B	A	NOT A
✓	✓	✓	✓	✓	✓	✓	✗
✓	✗	✗	✓	✗	✓	✗	✓
✗	✓	✗	✗	✓	✓		
✗	✗	✗	✗	✗	✗		

ikkalasi ham rost bo'lsa kamida bittasi rost bo'lsa

✓ = shart rost (TRUE) · ✗ = shart yolg'on (FALSE)

Bajarilish tartibi: avval NOT, keyin AND, oxirida OR

Aralash shartlarda qavs ishlatish: (janr='roman' OR janr='ertak') AND sahifa < 400

Qavslar — aralash shartlarda majburiy. SQL avval NOT ni, keyin AND ni, oxirida OR ni bajaradi — xuddi matematikada ko'paytirish qo'shishdan oldin bajarilgani kabi:

```
-- Maqsad: 400 betdan kam roman YOKI sarguzashtlar
SELECT * FROM kitoblar
WHERE (janr = 'roman' OR janr = 'sarguzasht') AND sahifa < 400;
```

-- ✗ Qavssiz yozsak, ma'no o'zgaradi:
-- WHERE janr = 'roman' OR janr = 'sarguzasht' AND sahifa < 400
-- SQL buni shunday tushunadi:
-- WHERE janr = 'roman' OR (janr = 'sarguzasht' AND sahifa < 400)
-- Natija: HAMMA romanlar (hatto 1225 betlik "Urush va tinchlik" ham!)
-- + 400 betdan kam sarguzashtlar

💡 Oddiy qoida: AND va OR bitta so'rovda aralashgan zahoti qavs qo'ying. Hatto shart bo'lmasa ham — niyatingiz kodni o'qigan odamga (va bir oydan keyingi o'zingizga) darrov tushunarli bo'ladi.

BETWEEN va IN — qisqa yozish usullari

```
SELECT * FROM kitoblar WHERE yil BETWEEN 1900 AND 1950; -- ikkala
chegara ham kiradi
SELECT * FROM kitoblar WHERE janr IN ('roman', 'ertak'); --
ro'yxatdan biri
```

```
SELECT * FROM kitoblar WHERE janr NOT IN ('roman', 'ertak'); --  
ro'yxatda yo'qlar
```

Ikkalasi ham uzun shartning qisqa, o'qishga qulay shakli:

Qisqa yozuv	Uzun ekvivalenti
yil BETWEEN 1900 AND 1950	yil >= 1900 AND yil <= 1950
janr IN ('roman', 'ertak')	janr = 'roman' OR janr = 'ertak'

✳️ BETWEEN ikkala chegarani ham O'Z ICHIGA oladi: 1900-yilgi kitob ham, 1950-yilgisi ham natijaga kiradi.

💡 BETWEEN sanalar bilan ham juda qulay: WHERE olingan_sana BETWEEN '2026-05-01' AND '2026-05-31' — may oyidagi ijaralar.

LIKE — naqsh bo'yicha qidirish

Aniq qiymatni emas, "shunga O'XSHASH" matnni qidirish kerak bo'lsa — LIKE. Ikkita maxsus belgi bor:

- % — istalgan miqdordagi istalgan belgi (0 ta ham bo'lishi mumkin)
- _ — aynan BITTA belgi

```
SELECT * FROM azolar WHERE ism LIKE 'A%'; -- A bilan boshlanadi  
SELECT * FROM azolar WHERE ism LIKE '%ova'; -- ova bilan tugaydi  
SELECT * FROM azolar WHERE ism LIKE '%kar%'; -- ichida kar bor  
SELECT * FROM kitoblar WHERE nomi LIKE '_ariq%'; -- 1-belgi istalgan,  
keyin 'ariq' ("Sariq...")
```

LIKE naqsh belgilari



istalgan sondagi istalgan belgi

(0 ta bo'lishi ham mumkin)



aynan BITTA belgi

(ko'p ham, kam ham emas)

'A%'

A bilan boshlanadi

Aziz ✓ · Malika ✗

'%ova'

ova bilan tugaydi

Rahimova ✓ · Tosheva ✗

'%kar%'

ichida 'kar' bor

Hulkar ✓ · Karimov ✓ · Bobur ✗

'_ariq%'

1-belgi istalgan, keyin 'ariq'

Sariq ✓ · ariq ✗

Bizning sozlamada (utf8mb4_unicode_ci) LIKE katta-kichik harfni farqlamaydi:

'a%' ham, 'A%' ham Aziz'ni topadi

✦ Bizning sozlamada (utf8mb4_unicode_ci) LIKE katta-kichik harfni farqlamaydi: 'a%' ham, 'A%' ham 'Aziz Karimov'ni topadi.

💡 '%kar%' kabi ikki tomoni % bilan qidirish katta jadvallarda sekin ishlaydi — sababini 21-bobda (indekslar) ko'ramiz. Hozircha bema'lol ishlataverasiz.

NULL — alohida dunyo!

Eng muhim qoida: NULL bilan = ishlamaydi!

```
SELECT * FROM azolar WHERE telefon = NULL; -- ✗ HECH NARSA qaytarmaydi!
SELECT * FROM azolar WHERE telefon IS NULL; -- ✓ telefoni yo'qlar
SELECT * FROM azolar WHERE telefon IS NOT NULL; -- ✓ telefoni borlar
```

Nega? NULL — "noma'lum" degani. "Noma'lum noma'lumga tengmi?" — buni bilib bo'lmaydi, javob ham noma'lum :) WHERE esa faqat aniq ROST bo'lgan qatorlarni o'tkazadi — shuning uchun = NULL bo'sh natija qaytaradi (xato ham bermaydi, shunisi xavfli!). NULL uchun maxsus tekshiruv bor: IS NULL va IS NOT NULL. Esda tuting: != NULL ham xuddi = NULL kabi ishlamaydi.

Amaliy foydasi: ijaralar da qaytarilgan_sana IS NULL = kitob hali qaytarilmagan!


```
-- Hozir kimning qo'lida kitob bor:  
SELECT * FROM ijaralar WHERE qaytarilgan_sana IS NULL;
```

Tez-tez uchraydigan xatolar

1. Alias'ni WHERE'da ishlatish. MySQL avval WHERE'ni tekshiradi, SELECT'dagi alias esa undan KEYIN "tug'iladi" — shuning uchun WHERE alias'ni tanimaydi:

```
-- ❌ Xato: Unknown column 'jami'  
SELECT nomi, narx * soni AS jami  
FROM dokon.mahsulotlar  
WHERE jami > 50000000;  
  
-- ✅ To'g'ri: ifodani WHERE'da qaytadan yozamiz  
SELECT nomi, narx * soni AS jami  
FROM dokon.mahsulotlar  
WHERE narx * soni > 50000000;
```

2. = NULL yozish — yuqorida ko'rdik: `IS NULL` ishlatish.

3. Tirnoqni unutish — `WHERE shahar = Toshkent` ❌ (MySQL "Toshkent" degan ustunni qidiradi), `WHERE shahar = 'Toshkent'` ✅.

8-bob masalalari

Har masalaga alohida SELECT yozing. Avval natijani xayolan taxmin qiling, keyin bajarib tekshiring — shunda WHERE miyaga yaxshi o'rtnashadi.

1. (kutubxona) 1950-yildan keyin chiqqan kitoblar
2. (kutubxona) Janri 'roman' bo'lgan kitoblar
3. (kutubxona) 300 betdan kam kitoblar
4. (kutubxona) Roman janridagi VA 400 betdan ko'p kitoblar
5. (kutubxona) Ertak YOKI she'riyat janridagi kitoblar — ikki usulda: OR va IN (eslatma: bazada janr 'shariyat' deb yozilgan)
6. (kutubxona) 1900–1970 oralig'ida chiqqan kitoblar (BETWEEN)
7. (kutubxona) Nomi 'S' bilan boshlanadigan kitoblar

8. (kutubxona) Ismi 'ova' bilan tugaydigan a'zolar
9. (kutubxona) Telefoni YO'Q a'zolar (IS NULL)
10. (kutubxona) Hali qaytarilmagan ijaralar
11. (kutubxona) Nusxa soni 3 dan kam VA 1950-yilgacha chiqqan kitoblar
12. (dokon) Narxi 5 mln dan qimmat mahsulotlar
13. (dokon) Omborda tugagan (soni = 0) mahsulotlar
14. (dokon) Narxi 1–5 mln oralig'idagi mahsulotlar
15. (dokon) Toshkentlik mijozlar
16. (dokon) Toshkentlik BO'LMAGAN mijozlar (2 usul: != va NOT IN)
17. (dokon) 'yetkazildi' yoki 'yolda' holatidagi buyurtmalar
18. (dokon) Nomida 'Samsung' so'zi bor mahsulotlar
19. (klinika) 10 yildan ortiq tajribali shifokorlar; 2000-yildan keyin tug'ilgan bemorlar; tashxisida 'tish' so'zi bor qabullar — 3 ta alohida so'rov
20. (taksi) Bahosi 4 dan past safarlar; baho qo'yilmagan safarlar; 15 km dan uzun VA 40000 dan qimmat safarlar; Chilonzordan boshlangan YOKI Chilonzorda tugagan safarlar — 4 ta alohida so'rov

09 — ORDER BY, LIMIT, DISTINCT

◀ Oldingi: 08 — WHERE — filtrlash · 🏠 README · Keyingi: 10 — Built-in funksiyalar (matn, son, sana) ➡

Bu bobda: natijani `ORDER BY` bilan saralashni (bir va bir necha ustun bo'yicha, ASC/DESC, NULL qayerga tushishini), `LIMIT` va `OFFSET` bilan natijani kesishni ("eng katta N ta" qolipi va sahifalash), `DISTINCT` bilan takror qiymatlarni olib tashlashni o'rganamiz — va oxirida ularning hammasini bitta query ichida to'g'ri tartibda yozishni mashq qilamiz.

ORDER BY — saralash

Muhim haqiqat: `SELECT` qatorlarni **kafolatlangan tartibda qaytarmaydi**. Bugun bir tartibda chiqqani, ertaga ham shunday chiqadi degani emas. Tartib kerakmi — uni doim `ORDER BY` bilan ochiq-oydin aytamiz. Bu xuddi kutubxonachiga "kitoblarni yili bo'yicha terib ber" deb aniq topshiriq berishga o'xshaydi.

```
SELECT * FROM kitoblar ORDER BY yil;           -- o'sish tartibida
(ASC — default)
SELECT * FROM kitoblar ORDER BY yil DESC;      -- kamayish tartibida
SELECT * FROM kitoblar ORDER BY janr, yil DESC; -- avval janr A-Z, har
janr ichida yil yangi→eski
```

Har xil turdagi ustunlar qanday saralanadi:

Tur	ASC (default)	DESC
Son (INT, DECIMAL)	kichik → katta	katta → kichik
Matn (VARCHAR)	alifbo: A → Z	Z → A
Sana (DATE, DATETIME)	eski → yangi	yangi → eski

✦ `ORDER BY janr, yil DESC` da `DESC` **faqat yil ga** taalluqli! Ikkalasini ham teskari qilmoqchi bo'lsangiz, har biriga alohida yozasiz: `ORDER BY janr DESC, yil DESC`.

💡 NULL ham saralanadi: MySQL'da ASC tartibda NULL'lar **eng boshida**, DESC'da **eng oxirida** chiqadi. Sinab ko'ring: `SELECT * FROM taksi.safarlar ORDER BY baho;` — baho qo'yilmagan safar birinchi chiqadi.

Alias va hisob-kitob natijasi bo'yicha ham saralash mumkin:

```
SELECT nomi, narx * soni AS ombor_qiymati
FROM dokon.mahsulotlar
ORDER BY ombor_qiymati DESC;      -- eng "qimmat zaxira" tepada
```

⚠️ Sonlarni VARCHAR ustunda saqlasangiz, ular **matn sifatida** saralanadi: '10' '9' dan oldin keladi (chunki alifboda '1' '9' dan kichik). Sonlar uchun doim son turlarini ishlatib (5-bobni eslang).

LIMIT — faqat N ta qator

LIMIT natijaning faqat boshidagi N ta qatorini qoldiradi. OFFSET esa "boshidan nechtasini tashlab ket" deydi:

```
SELECT * FROM kitoblar ORDER BY sahifa DESC LIMIT 3;      -- eng qalin 3
ta kitob
SELECT * FROM kitoblar ORDER BY yil LIMIT 5 OFFSET 5;      -- 6-10-
qatorlar (2-"sahifa")
SELECT * FROM kitoblar ORDER BY yil LIMIT 5, 5;            -- xuddi shu:
LIMIT offset, soni
```

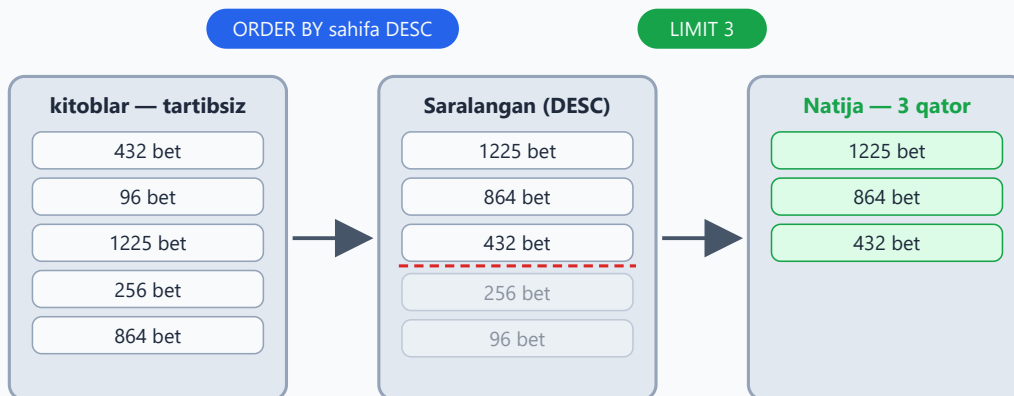
✳️ LIMIT 5, 5 qisqa yozuvida **birinchi son — OFFSET**, ikkinchisi — qatorlar soni. Chalkashtirib yubormaslik uchun LIMIT 5 OFFSET 5 shakli tushunarliroq — biz shuni ishlatamiz.

⚠️ LIMIT 'ni ORDER BY 'siz yozish — lotereya o'ynash: MySQL qaysi 3 qatorni berishi kafolatlanmagan. "Eng katta", "eng oxirgi", "birinchi" degan so'z bor joyda ORDER BY va LIMIT doim juft yuradi.

TOP-N pattern (yodlab oling): "eng katta/kichik N ta" = ORDER BY ... DESC/ASC LIMIT N.

```
-- Eng qimmat mahsulot:
SELECT * FROM dokon.mahsulotlar ORDER BY narx DESC LIMIT 1;
```

Top-N konveyeri: avval saralash, keyin kesish



LIMIT'ni ORDER BY'siz yozsangiz, qaysi 3 qator kelishi kafolatlanmaydi!

Qolip: ORDER BY ustun DESC LIMIT N — "eng katta N ta"

LIMIT + OFFSET — saytlardagi **sahifalash** (pagination) asosi. Har sahifada 5 ta mahsulot ko'rsatsak:

Sahifa	Query oxiri
1-sahifa	LIMIT 5 OFFSET 0
2-sahifa	LIMIT 5 OFFSET 5
3-sahifa	LIMIT 5 OFFSET 10

Formula oddiy: $OFFSET = (sahifa - 1) \times sahifa_hajmi$.

💡 Qiziq hiyla: ORDER BY RAND() LIMIT 1 — jadvaldan **tasodifiy** bitta qator. Kichik jadvallarda viktorina savoli tanlash kabi ishlarga qulay (katta jadvallarda sekin ishlaydi).

DISTINCT — takrorlarni olib tashlash

7-bobda DISTINCT bilan qisqacha tanishgan edik, endi chuqurroq ko'ramiz. DISTINCT natijadagi **takror qatorlarni** olib tashlaydi — "qanday qiymatlar bor o'zi?" degan savolga javob beradi:

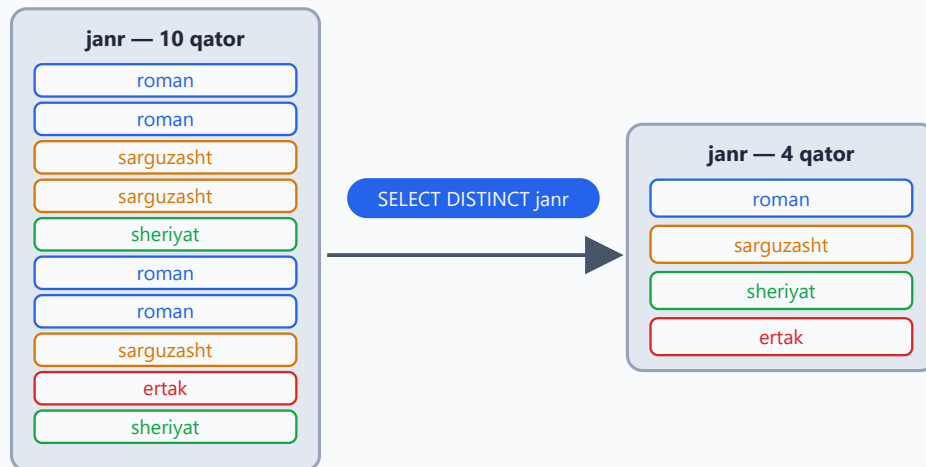
```
SELECT janr FROM kitoblar;  
sarguzasht, ...
```

```
-- 10 qator: roman, roman,
```



```
SELECT DISTINCT janr FROM kitoblar; -- 4 qator: roman, sarguzasht, sheriyat, ertak
```

DISTINCT — takror qiymatlarni olib tashlaydi



10 qator → 4 qator: har bir qiymatdan faqat bittasi qoladi

DISTINCT SELECT'dagi barcha ustunlarga birgalikda taalluqli — faqat birinchisiga emas

Bir nechta ustun yozsangiz, **kombinatsiya** (juftlik) takrorsiz bo'ladi:

```
SELECT DISTINCT boshlanish, tugash FROM taksi.safarlar;
-- 10 safar bor, lekin 9 qator chiqadi: "Markaz → Chilonzor" yo'nalishi
2 marta uchragan
```

✦ **DISTINCT** SELECT 'dagi **hamma ustunlarga birgalikda** taalluqli, faqat birinchisiga emas. **SELECT DISTINCT janr, yil** deganda "janri takrorlanmasin" emas, "janr+yil juftligi takrorlanmasin" deyilyapti.

DISTINCT 'ni **ORDER BY** va **LIMIT** bilan birga ishlatish mumkin:

```
SELECT DISTINCT janr FROM kitoblar ORDER BY janr; -- ✓ takrorsiz
va alifbo tartibida
-- SELECT DISTINCT janr FROM kitoblar ORDER BY yil; -- ✗ MySQL 8
xato beradi:
-- DISTINCT bilan faqat SELECT'da BOR ustunlar bo'yicha saralash mumkin
```

Nega xato? Bir janrga bir nechta yil to'g'ri keladi ("roman" 1869 hammi, 1928 hammi?) — MySQL qaysi yil bo'yicha saralashni bilmaydi.

💡 "Nechta **xil** janr bor?" deb sanash uchun `COUNT(DISTINCT janr)` ishlatiladi — bu bilan 11-bobda tanishamiz.

Hammasi birga — yozish tartibi

Bu bobgacha o'rgangan qismlarimiz queryda **qat'iy tartibda** turadi:

```
SELECT DISTINCT ustunlar      -- 1) nimani olamiz (DISTINCT — ixtiyoriy)
FROM jadval                  -- 2) qayerdan
WHERE shart                  -- 3) qaysi qatorlar (filtrlash)
ORDER BY ustun DESC          -- 4) qanday tartibda
LIMIT 5 OFFSET 0;           -- 5) nechtasini (doim eng oxirida)
```

Tartibni buzib `LIMIT 3 ORDER BY yil` yozsangiz — sintaksis xatosi. Hammasi birga ishlagan real misol:

```
-- Roman janridagi eng yangi 2 ta kitob:
SELECT nomi, yil FROM kitoblar
WHERE janr = 'roman'
ORDER BY yil DESC
LIMIT 2;
-- Natija: Mehrobdan chayon (1928), Otkan kunlar (1925)
```

9-bob masalalari

1. (kutubxona) Kitoblarni yil bo'yicha eski→yangi tartibda chiqaring
2. (kutubxona) Kitoblarni qalinligi bo'yicha qalin→yupqa tartibda chiqaring
3. (kutubxona) Eng eski 3 ta kitob
4. (kutubxona) Eng qalin kitob (1 ta)
5. (kutubxona) Eng yupqa kitob (maslahat: 4-masalaning teskarisi)
6. (kutubxona) A'zolari qo'shilgan sanasi bo'yicha yangi→eski tartibda chiqaring
7. (kutubxona) Eng oxirgi qo'shilgan 2 ta a'zo
8. (kutubxona) Avval janr bo'yicha alifbo, har janr ichida yil bo'yicha yangi→eski

9. (kutubxona) Mualliflarni tug'ilgan yili bo'yicha saralang va eng keksa 2 tasini chiqaring
10. (dokon) Eng qimmat 3 ta mahsulot
11. (dokon) Eng arzon mahsulot
12. (dokon) Omborda eng ko'p qolgan 3 ta mahsulot
13. (dokon) Mahsulotlar 2-sahifasi: 5 tadan bo'lganda 6–10-mahsulotlar (LIMIT + OFFSET; formulani eslang)
14. (dokon) Eng oxirgi 3 ta buyurtma (sana bo'yicha)
15. (klinika) Eng tajribali shifokor
16. (klinika) Qabul narxi bo'yicha arzon→qimmat shifokorlar; klinikada qanday mutaxassisliklar bor — takrorsiz ro'yxat (DISTINCT) — 2 ta query
17. (klinika) Eng katta yoshli bemor (tug'ilgan_sana bo'yicha o'ylang: eng katta yosh = eng KICHIK sana!)
18. (taksi) Eng uzun 3 ta safar
19. (taksi) Eng qimmat safar; eng yuqori reytingli haydovchi; safarlar qaysi manzillarda tugagan — takrorsiz ro'yxat (DISTINCT) — 3 ta query
20. (taksi) Safarlarni sana bo'yicha yangi→eski, bir xil sanada narx bo'yicha qimmat→arzon tartibda chiqaring (maslahat: ikkinchi kalit faqat birinchisi teng bo'lganda ishga tushadi)

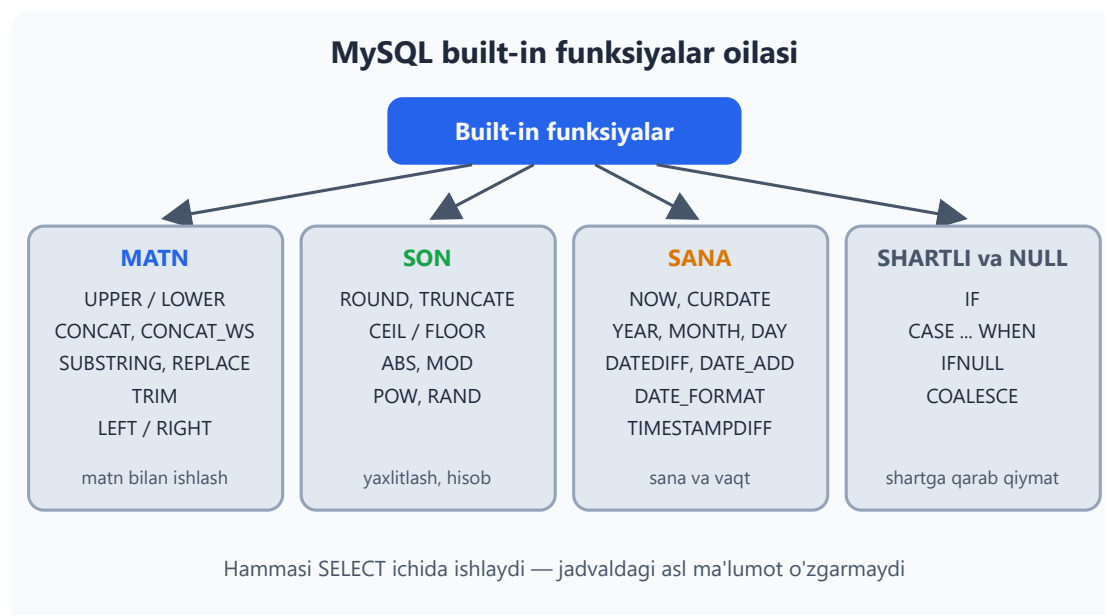
10 — Built-in funksiyalar (matn, son, sana)

← Oldingi: 09 — ORDER BY, LIMIT, DISTINCT · 🏠 README · Keyingi: 11 — Aggregate funksiyalar va GROUP BY →

Bu bobda: MySQL'ning tayyor (built-in) funksiyalari bilan tanishamiz: matnni o'zgartirish (UPPER, CONCAT, SUBSTRING, REPLACE), sonlarni yaxlitlash (ROUND, CEIL, FLOOR), sana hisob-kitoblari (NOW, DATEDIFF, DATE_FORMAT, TIMESTAMPDIFF), shartga qarab qiymat tanlash (IF, CASE) va NULL bilan ishlash (IFNULL, COALESCE)ni o'rganamiz.

MySQL'da yuzlab tayyor funksiya bor. Ularni kalkulyatordagi tugmalar deb tasavvur qiling: kvadrat ildizni qog'ozda hisoblab o'tirmaysiz — tayyor tugmani bosasiz. SQL funksiyasi ham xuddi shunday ishlaydi: unga qiymat berasiz, u amalni bajarib, natijani qaytaradi. Hammasini yodlash shart emas — bu bobda eng ko'p ishlatiladiganlarini ko'ramiz, qolganini kerak bo'lganda hujjatdan qarab olasiz.

📌 Eng muhim qoida: bu funksiyalar jadvaldagi asl ma'lumotni **o'zgartirmaydi**. Ular faqat SELECT natijasi qanday ko'rinishini belgilaydi: `SELECT UPPER(nomi) ...` deb yozsangiz ham, jadvalda `nomi` avvalgidek o'z holicha qolaveradi. (Ma'lumotni haqiqatan o'zgartirish — UPDATE, uni 17-bobda ko'ramiz.)



Matn funksiyalari

```
SELECT UPPER('salom');           -- SALOM (katta harfga)
SELECT LOWER('SALOM');           -- salom (kichik harfga)
SELECT LENGTH('salom');           -- 5 (BAYTlarda o'lchaydi)
SELECT CHAR_LENGTH('salom');      -- 5 (BELGIlarda o'lchaydi)
SELECT CONCAT('Aziz', ' ', 'Karimov'); -- Aziz Karimov (matnlarni
    ulaydi)
SELECT SUBSTRING('Salom dunyo', 1, 5); -- Salom (1-belgidan boshlab
    5 ta)
SELECT REPLACE('+998901112233', '+998', ''); -- 901112233
    (almashtiradi)
SELECT TRIM(' salom ');           -- salom (chetdagi
    bo'shliqlar ketadi)
SELECT LEFT('Salom', 3);          -- Sal (boshidan 3 ta)
SELECT RIGHT('Salom', 3);         -- lom (oxiridan 3 ta)
```

LENGTH va CHAR_LENGTH — qachon farq qiladi? Lotin harflarida ikkalasi bir xil son beradi, chunki har lotin harfi 1 bayt joy oladi. Lekin kirill va boshqa "keng" belgilar bir necha bayt egallaydi:

```
SELECT LENGTH('ТовшекТ');        -- 14 (har kirill harfi 2 bayt)
SELECT CHAR_LENGTH('ТовшекТ');    -- 7  (harflar soni)
```

Xulosa oddiy: "necha ta harf?" degan savolga doim CHAR_LENGTH to'g'ri javob beradi.

SUBSTRING 1 dan sanaydi. Ko'p dasturlash tillarida sanash 0 dan boshlanadi, SQL'da esa 1 dan: birinchi belgi — 1-pozitsiya. Uchinchi argumentni yozmasangiz, oxirigacha olib beradi:

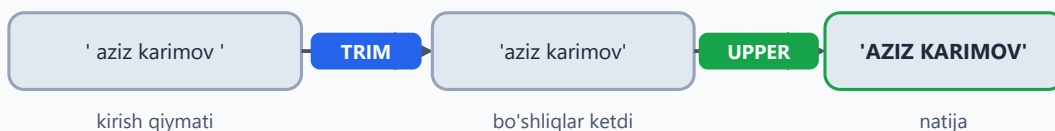
```
SELECT SUBSTRING('Salom dunyo', 7); -- dunyo (7-belgidan oxirigacha)
```

CONCAT'ning NULL tuzog'i. Argumentlardan bittasi NULL bo'lsa, butun natija NULL bo'lib qoladi. Bunday holatda CONCAT_WS (WS — "with separator", ya'ni ajratuvchi bilan) qutqaradi: birinchi argument ajratuvchi bo'ladi, NULL'larni esa shunchaki tashlab ketadi:

```
SELECT CONCAT('Aziz', NULL, 'Karimov');      -- NULL!
SELECT CONCAT_WS(' ', 'Aziz', NULL, 'Karimov'); -- Aziz Karimov
```

Funksiyalarni ichma-ich yozish ham mumkin — MySQL ularni ichkaridan tashqariga qarab hisoblaydi:

Matn konveyeri: ichkaridan tashqariga



SELECT UPPER(TRIM(' aziz karimov ')); → 'AZIZ KARIMOV'

Avval eng ichkaridagi TRIM ishlaydi, uning natijasi UPPER'ga uzatiladi

Son funksiyalari

```
SELECT ROUND(4.567, 2);      -- 4.57 (2 xonagacha yaxlitlaydi)
SELECT ROUND(4.567);         -- 5 (butun songa yaxlitlaydi)
SELECT TRUNCATE(4.567, 2);   -- 4.56 (yaxlitlamaydi, shunchaki kesadi)
SELECT CEIL(4.1);            -- 5 (doim yuqoriga)
SELECT FLOOR(4.9);           -- 4 (doim pastga)
SELECT ABS(-15);             -- 15 (modul — ishorani tashlaydi)
SELECT MOD(10, 3);           -- 1 (bo'lishdan qolgan qoldiq; 10 % 3 ham shu)
SELECT POW(2, 10);           -- 1024 (daraja: 2 ning 10-darajasi)
SELECT RAND();               -- 0 va 1 orasida tasodifiy son (har safar boshqa)
```

ROUND va TRUNCATE farqiga e'tibor bering: ROUND matematik qoida bo'yicha yaxlitlaydi (4.567 → 4.57), TRUNCATE esa ortiqcha xonalarni shunchaki uzib tashlaydi (4.567 → 4.56). Pul hisob-kitoblarida bu kichik farq katta ahamiyatga ega bo'lishi mumkin!

Sana funksiyalari

Sanalar bilan ishlash — real loyihalardagi eng ko'p uchraydigan vazifalardan biri: "necha kun o'tdi?", "muddat qachon tugaydi?", "mijoz necha yoshda?". MySQL bunga boy asboblari to'plamini beradi:

```
SELECT NOW(); -- hozirgi sana+vaqt
(DATETIME)
SELECT CURDATE(); -- bugungi sana (faqat DATE)
SELECT YEAR('2026-06-09'); -- 2026
SELECT MONTH('2026-06-09'); -- 6
SELECT DAY('2026-06-09'); -- 9
SELECT HOUR('2026-06-09 18:45:00'); -- 18 (MINUTE va SECOND ham bor)
SELECT DAYNAME('2026-06-09'); -- Tuesday (seshanba)
SELECT DATEDIFF('2026-06-09', '2026-05-01'); -- 39 (ikki sana orasidagi kunlar)
SELECT DATE_ADD('2026-06-09', INTERVAL 15 DAY); -- 2026-06-24 (15 kun keyin)
SELECT DATE_SUB(NOW(), INTERVAL 1 MONTH); -- 1 oy oldin
SELECT DATE_FORMAT(NOW(), '%d.%m.%Y'); -- masalan:
09.06.2026
SELECT TIMESTAMPDIFF(YEAR, '1990-08-25', CURDATE()); -- yosh hisoblash!
```

✦ DAYNAME (va MONTHNAME) natijani inglizcha qaytaradi. Tilni `lc_time_names` sozlamasi bilan o'zgartirsa bo'ladi (masalan, `SET lc_time_names = 'ru_RU';`), lekin o'zbekcha lokal MySQL'da yo'q — kerak bo'lsa CASE bilan o'zingiz tarjima qilasiz.

Sana funksiyalari ishda

DATEDIFF — ikki sana orasida necha kun?



DATE_ADD — sanaga muddat qo'shish



DATE_FORMAT — ko'rinishni bezash



DATE_FORMAT bazadagi sanani o'zgartirmaydi — faqat chiqish ko'rinishini bezaydi

DATE_FORMAT kodlari. Eng ko'p ishlatiladiganlari:

Kod	Ma'nosi	Misol
%d	kun (2 xonali)	09
%m	oy (2 xonali)	06
%Y	yil (4 xonali)	2026
%H	soat (24 soatlik)	18
%i	minut	05
%s	sekund	30
%W	hafta kuni nomi	Tuesday
%M	oy nomi	June

⚠️ Klassik tuzoq: minut — %i, %m emas (%m — oy!). %M esa umuman boshqa narsa — oy nomi. Katta-kichik harf farqiga diqqat qiling.

Yosh hisoblashda TIMESTAMPDIFF ishonchli. "Yillarni bir-biridan ayirib qo'ya qolaman" desangiz, xato chiqishi mumkin:

```
-- Bugun 2026-06-10 deb olaylik. 1990-08-25 da tug'ilgan odam:
SELECT TIMESTAMPDIFF(YEAR, '1990-08-25', '2026-06-10'); -- 35
(to'g'ri: avgust hali kelmadi)
SELECT YEAR('2026-06-10') - YEAR('1990-08-25');           -- 36 (xato: 1
yosh oshirib yubordi)
```

TIMESTAMPDIFF faqat **to'liq** o'tgan yillarni sanaydi — xuddi hayotdagidek: odam tug'ilgan kunigacha hali 35 yoshda, 36 emas.

IF va CASE — shartli qiymat

Ba'zan natija ustunini shartga qarab hosil qilish kerak: omborda mahsulot "bor"mi yoki "tugagan"mi? IF funksiyasi uch qism oladi — IF(shart, rost bo'lsa shu, yolg'on bo'lsa bu):

```
SELECT nomi, soni, IF(soni > 0, 'bor', 'tugagan') AS holati
FROM dokon.mahsulotlar;
```

Shartlar ikkitadan ko'p bo'lsa, CASE ishlatamiz. U shartlarni yuqoridan pastga tekshiradi va **birinchi** mos kelganida to'xtaydi — shuning uchun eng kichik chegara birinchi turibdi. ELSE esa "hech biri mos kelmasa" bo'limi:

```
SELECT nomi, narx,
CASE
    WHEN narx < 1000000 THEN 'arzon'
    WHEN narx < 10000000 THEN 'ortacha'
    ELSE 'qimmat'
END AS toifa
FROM dokon.mahsulotlar;
```

💡 IF() — MySQL'ga xos funksiya, CASE esa SQL standarti: PostgreSQL, SQLite va boshqa bazalarda ham aynan shunday ishlaydi. Shart ikkitadan ko'p bo'lsa yoki kod boshqa bazaga ko'chishi mumkin bo'lsa — CASE'ni tanlang.

NULL bilan ishlash funksiyalari

8-bobdan eslang: NULL — "qiymat yo'q" degani va u bilan oddiy taqqoslash ishlamaydi. Natijada NULL o'rniga tushunarli matn ko'rsatish uchun ikkita funksiya bor:

```
SELECT ism, IFNULL(telefon, 'telefon yoq') FROM kutubxona.azolar;
SELECT COALESCE(NULL, NULL, 'birinchi NULL bo'lmagan', 'x'); -- 3-
qiymat qaytadi
```

- **IFNULL(a, b)** — ikki argument: **a** NULL bo'lmasa **a** ni, NULL bo'lsa **b** ni qaytaradi.
- **COALESCE(...)** — istalgancha argument: chapdan boshlab **birinchi NULL bo'lmagan** qiymatni qaytaradi. "Ish telefoni bo'lsa — shuni, bo'lmasa uy telefonini, u ham bo'lmasa 'aloqa yoq' deb ko'rsat" kabi mantiq uchun juda qulay.

💡 Funksiyalarni WHERE ichida ham bimalol ishlatish mumkin:

```
-- 2026-yilda olingan ijaralar:  
SELECT * FROM kutubxona.ijaralar WHERE YEAR(olingan_sana) = 2026;
```

Lekin bilib qo'ying: katta jadvalda ustunga funksiya qo'llab filtrlash qidiruvni sekinlashtirishi mumkin — bunga 21-bobda (indekslar) qaytamiz.

10-bob masalalari

1. (kutubxona) Kitob nomlarini KATTA harflarda chiqaring
2. (kutubxona) Har kitob nomining uzunligini chiqaring (CHAR_LENGTH)
3. (kutubxona) Muallif ismi va davlatini bitta ustunda: 'Abdulla Qodiriy (Ozbekiston)' — CONCAT
4. (kutubxona) A'zolar telefonidan '+998' ni olib tashlab ko'rsating (REPLACE)
5. (kutubxona) Har kitob necha yoshda: 2026 - yil yoki YEAR(CURDATE()) - yil
6. (kutubxona) Ijara necha kun davom etgan: DATEDIFF(qaytarilgan_sana, olingan_sana) — qaytarilganlar uchun
7. (kutubxona) Qaytarilmagan kitoblar bugungacha necha kundan beri o'qilmoqda: DATEDIFF(CURDATE(), olingan_sana)
8. (kutubxona) Har ijara uchun "muddat": 15 kundan oshsa 'kechikkan', aks holda 'normal' (IF + DATEDIFF)
9. (dokon) Narxlarni yaxlitlab ming so'mda: ROUND(narx/1000) AS ming_som
10. (dokon) Mahsulot holati CASE bilan: soni=0 → 'tugagan', soni<10 → 'oz qoldi', aks holda 'yetarli'
11. (dokon) Mahsulot nomining birinchi 10 belgisi + '...' (LEFT va CONCAT)
12. (dokon) Buyurtma sanasini '09.06.2026 18:00' formatida (DATE_FORMAT: '%d.%m.%Y %H:%i')
13. (dokon) Har buyurtma qaysi hafta kuni qilinganini chiqaring (DAYNAME)
14. (klinika) Har bemorning yoshini hisoblang (TIMESTAMPDIFF)
15. (klinika) Bemor ismini 'LOLA MIRZAYEVA' ko'rinishida (UPPER)
16. (klinika) Telefoni NULL bo'lsa 'korsatilmagan' deb chiqaring (IFNULL)
17. (klinika) Yosh toifasi CASE bilan: <18 'bola', 18-60 'katta', >60 'keksa'
18. (taksi) Narxni 500 ga yaxlitlang: ROUND(narx/500)*500 — sinab ko'ring va nega aynan shunday yozilishini tushunib oling

19. (taksi) Har safarning km narxini 2 xona aniqlikda: `ROUND(narx/masofa_km, 2)`
20. (taksi) Safar sanasidan faqat soatni chiqaring (`HOUR` funksiyasi); 18:00 dan keyingilarga 'kechki', oldingilariga 'kunduzgi' deb belgi qo'yning

11 — Aggregate funksiyalar va GROUP BY

← Oldingi: 10 — Built-in funksiyalar (matn, son, sana) · 🏠 README · Keyingi: 12 — JOIN — jadvallarni bog'lash →

Bu bobda: ko'p qatordan bitta xulosa chiqaradigan aggregate funksiyalarni (`COUNT` , `SUM` , `AVG` , `MIN` , `MAX`) va "har bir ... bo'yicha" savollariga javob beradigan `GROUP BY` 'ni o'rganamiz. Yo'l-yo'lakay `COUNT(*)` va `COUNT(ustun)` orasidagi `NULL` farqini, tayyor guruhlarini filtrlaydigan `HAVING` 'ni va query'ning bajarilish tartibini ko'rib chiqamiz.

Aggregate — ko'p qatordan BITTA natija

Shu paytgacha yozgan query'larimiz qatorlarni **ro'yxat** qilib qaytarardi. Endi boshqa turdagi savollarga o'tamiz: "jami nechta?", "o'rtacha qancha?", "eng kattasi qaysi?". Bunday savollarga ro'yxat emas, **bitta qiymat** javob bo'ladi.

Aggregate funksiya — ko'p qatorni "presslab" bitta qiymat chiqaradigan mashina. Sinf jurnalini eslang: jurnalda 30 ta baho turibdi, lekin "sinfning o'rtacha bahosi" — bitta son. Beshta asosiy aggregate bor:

Funksiya	Nima qiladi	Qanday savolga javob
<code>COUNT(*)</code>	qatorlarni sanaydi	"jami nechta?"
<code>SUM(ustun)</code>	yig'indini hisoblaydi	"hammasi qo'shilsa qancha?"
<code>AVG(ustun)</code>	o'rtachani topadi	"o'rtacha qancha?"
<code>MIN(ustun)</code>	eng kichigini topadi	"eng kichigi/arzoni/eskisi?"
<code>MAX(ustun)</code>	eng kattasini topadi	"eng kattasi/qimmat/yangisi?"

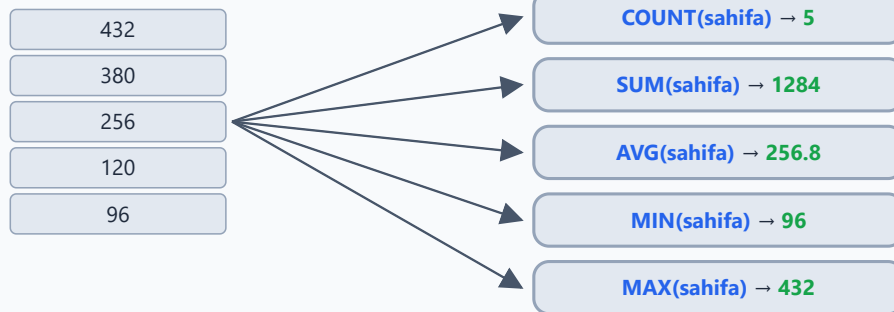
```
USE dokon;

SELECT COUNT(*) FROM mahsulotlar;      -- nechta mahsulot bor:
12
SELECT SUM(soni) FROM mahsulotlar;     -- omborda jami nechta
```

```
dona: 361
SELECT AVG(narx) FROM mahsulotlar;           -- o'rtacha narx: ~5 235
833
SELECT MIN(narx), MAX(narx) FROM mahsulotlar; -- eng arzon (50 000) va
eng qimmat (16 000 000)
```

Aggregate: ko'p qator kiradi — bitta qiymat chiqadi

kitoblar.sahifa (5 qator)



COUNT(ustun), SUM, AVG, MIN, MAX — hammasi NULL qiymatlarni tashlab ketadi.

Faqat COUNT(*) hech narsaga qaramay HAMMA qatorni sanaydi.

Bir nechta aggregate'ni bitta query'da yozish mumkin — alias bilan natija chiroyli o'qiladi:

```
SELECT COUNT(*) AS mahsulotlar_soni,
       MIN(narx) AS eng_arzon,
       MAX(narx) AS eng_qimmat
FROM mahsulotlar;
```

⚠ Eng ko'p uchraydigan boshlovchi xatosi — aggregate yoniga oddiy ustun qo'shib yuborish:

```
SELECT nomi, MAX(narx) FROM mahsulotlar; -- ❌ Error 1140
```

"Eng qimmat mahsulotning nomi chiqadi" deb o'ylash tabiiy, lekin MySQL uchun bu ikkita qarama-qarshi topshiriq: MAX 12 qatordan **bitta** son yasaydi, nomi esa **12 ta**. Qaysi nomni yozsin? MySQL 8 bunga to'g'ridan-to'g'ri xato beradi. "Eng qimmat qaysi?" degan savolga to'g'ri yo'l — 9-bobdagi tanish qolip:

```
SELECT nomi, narx FROM mahsulotlar ORDER BY narx DESC LIMIT 1;
```

COUNT(*) vs COUNT(ustun) — NULL tuzog'i

Muhim nozik joy:

```
SELECT COUNT(*) FROM kutubxona.azolar;      -- 6 — HAMMA qator
SELECT COUNT(telefon) FROM kutubxona.azolar; -- 5 — kamroq! NULL'lar
SANALMAYDI
```

COUNT(*) qatorlarning o'zini sanaydi, COUNT(ustun) esa shu ustundagi **NULL bo'lmagan** qiymatlarni. Nilufarning telefoni yozilmagan (NULL) — shuning uchun u ikkinchi sanoqqa kirmadi. (6-bobdagi mashqlarda o'zingiz qator qo'shgan bo'lsangiz, sonlaringiz kattaroq chiqadi — farqning o'zi baribir ko'rinadi.)

COUNT(*) va COUNT(telefon): NULL farqi

ism	telefon
Aziz Karimov	+998901112233
Malika Tosheva	+998902223344
Jasur Aliyev	+998903334455
Nilufar Rahimova	NULL
Bobur Saidov	+998905556677
Zilola Ergasheva	+998906667788

Diagram illustrating the difference between COUNT(*) and COUNT(telefon):

- COUNT(*) = 6: hamma qatorni sanaydi (counts all rows, including the one with NULL).
- COUNT(telefon) = 5: NULL sanalmadi! (counts only rows where the 'telefon' column is not NULL).

COUNT(ustun) — faqat NULL bo'lmagan qiymatlarni sanaydi.
SUM va AVG ham NULL'ni e'tiborsiz qoldiradi.

Xuddi shu qoida SUM va AVG 'ga ham tegishli: ular NULL'larni shunchaki **e'tiborsiz qoldiradi**. Masalan, AVG(baho) — baho qo'yilgan safarlarninggina o'rtachasi; NULL'lar yig'indiga ham, bo'luvchiga ham kirmaydi. Bu ko'pincha aynan biz xohlagan narsa, lekin bilib turish kerak.

💡 Yana bir foydali shakl — COUNT(DISTINCT ustun) : nechta **har xil** qiymat borligini sanaydi:

```
SELECT COUNT(DISTINCT janr) FROM kutubxona.kitoblar; -- 4 xil janr
```

GROUP BY — "har bir ... bo'yicha"

Aggregate butun jadvalga **bitta** natija beradi. Savol "har bir ... bo'yicha" bo'lsa-chi? Mana shu yerda `GROUP BY` sahnaga chiqadi. Savolda "**har bir**" so'zini eshitsangiz — bu `GROUP BY`:

- "**Har bir** janrda nechta kitob?" → `GROUP BY janr`
- "**Har bir** shahardan nechta mijoz?" → `GROUP BY shahar`
- "**Har bir** haydovchi qancha topdi?" → `GROUP BY haydovchi_id`

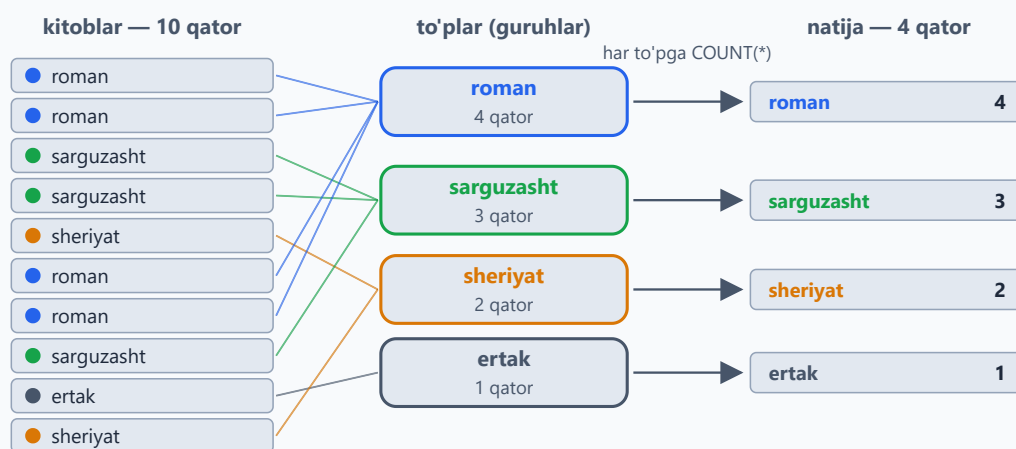
```
USE kutubxona;

SELECT janr, COUNT(*) AS soni
FROM kitoblar
GROUP BY janr;

-- janr      | soni
-- roman     | 4
-- sarguzasht | 3
-- sheriyat  | 2
-- ertak     | 1
```

Qanday ishlaydi: MySQL qatorlarni `janr` qiymati bo'yicha "to'plarga" ajratadi (roman to'pi, sarguzasht to'pi...), keyin **har to'pga alohida** `COUNT/SUM/AVG` qo'llaydi. Nechta to'p bo'lsa — natijada shuncha qator.

GROUP BY: qatorlar to'plarga, har to'pdan bitta qator



```
SELECT janr, COUNT(*) AS soni FROM kitoblar GROUP BY janr;
```

Nechta to'p bo'lsa — natijada shuncha qator.

✦ MySQL 8'da `GROUP BY` natijani **saralamaydi** (eski versiyalarda saralanardi, 8.0 dan bu olib tashlangan) — guruhlar istalgan tartibda chiqishi mumkin. Tartib kerak bo'lsa, oxiriga `ORDER BY` qo'shing.

Har guruhga bir nechta aggregate'ni birdan qo'llash mumkin:

```
SELECT janr,  
       COUNT(*) AS soni,  
       ROUND(AVG(sahifa)) AS ortacha_sahifa  
FROM kitoblar  
GROUP BY janr;
```

```
-- janr      | soni | ortacha_sahifa  
-- roman     | 4    | 725  
-- sarguzasht | 3    | 288  
-- sheriyat  | 2    | 160  
-- ertak     | 1    | 96
```

Bir necha ustun bo'yicha guruhlash ham bo'ladi — bunda har bir **kombinatsiya** alohida to'p:

```
-- Har mijozning har holatdagi buyurtmalari soni:  
SELECT mijoz_id, holat, COUNT(*) AS soni  
FROM dokon.buyurtmalar  
GROUP BY mijoz_id, holat;
```

```
-- mijoz_id | holat      | soni  
-- 1        | yetkazildi | 2  
-- 1        | yangi      | 1  
-- 2        | yetkazildi | 1  
-- ...
```

Oltin qoida (ONLY_FULL_GROUP_BY)

SELECT'da faqat ikki narsa turishi mumkin: GROUP BY'dagi ustun yoki aggregate funksiya. Boshqa ustun yozsangiz:

```
SELECT janr, nomi, COUNT(*) FROM kitoblar GROUP BY janr;  
-- ❌ Error 1055: ...'kutubxona.kitoblar.nomi' is not in GROUP BY  
   clause...
```

Nega? `roman` to'pida 4 ta kitob bor — `nomi` ustuniga MySQL qaysi birini yozsin? To'g'ri javob yo'q. MySQL 8'da `ONLY_FULL_GROUP_BY` rejimi default yoqilgan va bunday query darhol xato bilan to'xtaydi. Eski versiyalarda bu rejim o'chiq edi: MySQL to'pdan **duch kelgan** bitta nomni qaytarardi va xato sezilmasdan noto'g'ri hisobotlar tarqalaverardi. Shuning uchun bu xato — dushman emas, qo'riqchi.

HAVING — guruhlarni filtrlash

"2 tadan ko'p kitobi bor janrlar" — bu yerda alohida kitoblarni emas, tayyor **to'plarning o'zini** filtrlash kerak. Buning uchun `HAVING` bor:

```
SELECT janr, COUNT(*) AS soni
FROM kitoblar
GROUP BY janr
HAVING soni > 2;
```

```
-- roman      | 4
-- sarguzasht | 3
```

WHERE vs HAVING:

	WHERE	HAVING
Nimani filtrlaydi	alohida qatorlarni	tayyor guruhlarni
Qachon ishlaydi	guruhlashdan oldin	guruhlashdan keyin
Aggregate (COUNT, SUM...)	✗ ishlatib bo'lmaydi	✓ bimalol

Ikkalasi bitta query'da bimalol birga keladi — har biri o'z ishini qiladi:

```
-- 1950-yildan keyingi kitoblar ichida, 1 tadan ko'p kitobli janrlar:
SELECT janr, COUNT(*) AS soni
FROM kitoblar
WHERE yil > 1950      -- avval qatorlar filtrlanadi
GROUP BY janr        -- qolganlari to'plarga ajratiladi
HAVING soni > 1;      -- keyin to'plar filtrlanadi

-- sarguzasht | 2
-- shariyat   | 2
```

WHERE — guruhlashdan OLDIN, HAVING — KEYIN



10 qator → 8 qator → 4 to'p → 3 to'p

WHERE ichida COUNT/SUM ishlamaydi — u paytda to'plar hali yo'q!

Guruh sharti doim HAVING'da: HAVING COUNT(*) >= 2

💡 HAVING soni > 2 dagi soni — SELECT'dagi alias. Alias'ni HAVING'da ishlatish — MySQL'ning qulayligi (standart SQL'da HAVING COUNT(*) > 2 deb yoziladi; MySQL'da ikkalasi ham ishlaydi). WHERE 'da esa alias ISHLAMAYDI — sababi quyida.

Guruhlarni saralash — TOP-N to'plarga ham ishlaydi

GROUP BY natijasi ham oddiy natija — unga 9-bobdagi ORDER BY + LIMIT qolipini qo'llash mumkin:

```
-- Eng ko'p kitobli janr:  
SELECT janr, COUNT(*) AS soni  
FROM kitoblar  
GROUP BY janr  
ORDER BY soni DESC  
LIMIT 1; -- roman | 4
```

"Eng sersafar haydovchi", "eng sertushum kun", "eng ko'p buyurtma qilgan mijoz" — hammasi shu qolip: avval guruhlab sanaymiz, keyin saralab tepadagisini olamiz.

Query bajarilish tartibi (devorga yopishtiring!)

```
FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT
```

Siz query'ni SELECT'dan boshlab yozasiz, lekin MySQL uni FROM'dan boshlab bajaradi. Shu tartibdan ikkita amaliy xulosa kelib chiqadi:

- `WHERE COUNT(*) > 2` ❌ — WHERE ishlayotgan paytda hali to'plar YO'Q, demak aggregate ham yo'q. Guruh sharti — faqat HAVING'da.
- `WHERE soni > 2` ❌ — alias SELECT'da tug'iladi, SELECT esa WHERE'dan ancha keyin keladi. (HAVING'da alias ishlashi — MySQL'ning maxsus qulayligi. ORDER BY esa tartibda SELECT'dan keyin turadi, shuning uchun unda alias tabiiy ishlaydi.)

11-bob masalalari

1. (kutubxona) Jami nechta kitob bor?
2. (kutubxona) Barcha kitob nusxalarining yig'indisi (`SUM(nusxa_soni)`)
3. (kutubxona) O'rtacha sahifa soni (`ROUND` bilan yaxlitlang)
4. (kutubxona) Eng qadimgi va eng yangi kitob yili (`MIN` , `MAX` — bitta query'da)
5. (kutubxona) Telefoni bor a'zolar soni (`COUNT(telefon)` — NULL'lar sanalmasligini tekshiring!)
6. (kutubxona) Har janrda nechta kitob?
7. (kutubxona) Har janrning o'rtacha sahifa soni
8. (kutubxona) Har muallif (`muallif_id`) nechta kitob yozgan?
9. (kutubxona) 2 tadan ko'p kitob yozgan mualliflar (`HAVING`)
10. (kutubxona) Har a'zo nechta marta kitob olgan? (`ijara_lar` jadvali, `GROUP BY azo_id`)
11. (dokon) Har kategoriyada nechta mahsulot bor va o'rtacha narxi qancha?
12. (dokon) Har kategoriyaning ombor qiymati: `SUM(narx * soni)`
13. (dokon) Har shahardan nechta mijoz?
14. (dokon) Har mijoz nechta buyurtma qilgan? (`GROUP BY mijoz_id`)
15. (dokon) Har buyurtmaning umumiy summasi: `buyurtma_qatorlari` da `GROUP BY buyurtma_id` , `SUM(narx * soni)`
16. (dokon) Holatlar bo'yicha buyurtmalar soni (`GROUP BY holat`)
17. (klinika) Har shifokor nechta qabul qilgan va jami qancha to'lov yig'gan? (`COUNT` + `SUM(to'lov)`)
18. (klinika) Har bemor klinikaga jami qancha to'lagan? Faqat 300 000 dan ko'p to'laganlarni qoldiring (`HAVING`)

19. (taksi) Har haydovchi: safarlar soni, jami daromad, o'rtacha baho (`AVG(baho)` — `NULL`'lar avtomatik tashlanadi!)
20. (taksi) Har kun (`DATE(sana)`) bo'yicha jami tushum; eng sertushum kunni toping (`ORDER BY` + `LIMIT` qo'shing)

12 — JOIN — jadvallarni bog'lash

← Oldingi: 11 — Aggregate funksiyalar va GROUP BY · 🏠 README · Keyingi: 13 — UNION — natijalarni birlashtirish →

Bu bobda: ma'lumotni bir necha jadvalga bo'lib saqlaganimizdan keyin ularni qaytadan "yig'ishni" — JOIN'ni o'rganamiz: `ON` sharti qatorlarni qanday ulashini, `INNER` va `LEFT JOIN` farqini, "hech narsa qilmaganlarni topish" (anti-join) qolipini, `LEFT JOIN`'ni buzib qo'yadigan `WHERE` tuzog'ini, 3-4 jadvalli zanjirlarni va `JOIN` + `GROUP BY` bilan real hisobotlar tuzishni ko'rib chiqamiz.

Muammo

Kitoblar da `muallif_id = 1` turibdi. "1" kim? Buni bilish uchun `mualliflar` ga qarash kerak. 3-bobda ma'lumotni ataylab shunday bo'lib saqlagan edik: muallif haqidagi hamma narsa bitta joyda turadi, kitob esa unga faqat raqam orqali "ishora qiladi". Endi teskari amalni o'rganamiz — bo'lingan ma'lumotni qaytadan yig'ish. `JOIN` — ikki jadvalni **yonma-yon qo'yish**:

```
USE kutubxona;

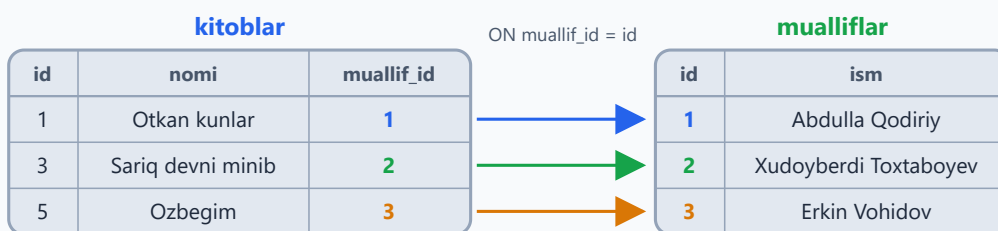
SELECT kitoblar.nomi, mualliflar.ism
FROM kitoblar
JOIN mualliflar ON kitoblar.muallif_id = mualliflar.id;
```

O'qilishi: "kitoblar jadvalini ol, unga mualliflar'ni ULA, ulash sharti: kitobdagi `muallif_id` = muallifdagi `id`".

nomi	ism
Otkan kunlar	Abdulla Qodiriy
Mehrobdan chayon	Abdulla Qodiriy
Sariq devni minib	Xudoyberdi Toxtaboyev
... (jami 10 qator)	...

MySQL buni qanday qiladi? `kitoblar` ning har bir qatorini oladi, `mualliflar` dan `ON` shartiga mos qatorni topadi va ikkalasini bitta uzun qatorga "yelimlaydi". Mos qator topilmasa, oddiy JOIN'da bu qator natijaga umuman tushmaydi — birozdan keyin bu juda muhim bo'ladi.

JOIN ON: ikki jadval qatorlari qanday ulanadi



Natija (JOIN'dan keyin):

nomi	ism
Otkan kunlar	Abdulla Qodiriy
Sariq devni minib	Xudoyberdi Toxtaboyev
Ozbegim	Erkin Vohidov

Har bir kitob qatoriga ON sharti mos kelgan muallif qatori yonma-yon yelimlanadi

Alias bilan qisqaroq (standart uslub):

```
SELECT k.nomi, m.ism, k.yil
FROM kitoblar k
JOIN mualliflar m ON k.muallif_id = m.id;
```

✦ Ikkala jadvalda bir xil nomli ustun bo'lsa (bizda ikkalasida ham `id` bor), uni "yalang'och" yozib bo'lmaydi: `SELECT id, nomi ...` → `Error: Column 'id' in field list is ambiguous` — "qaysi id'ni nazarda tutdingiz?". Shuning uchun JOIN'da ustunlarni doim alias bilan aniq yozish odat tusiga kiradi: `k.id`, `m.id`.

INNER JOIN vs LEFT JOIN — asosiy farq

Tasavvur: ikki ro'yxatni solishtirayapsiz.

INNER JOIN (yoki shunchaki **JOIN** — bu ikkisi bir xil narsa) — faqat **ikkala tomonda ham borlari**:

```
-- Faqat kitob olgan a'zolar ko'rinadi:  
SELECT a.ism, i.olingan_sana  
FROM azolar a  
INNER JOIN ijaralar i ON i.azo_id = a.id;
```

✦ JOIN qatorlar sonini o'zgartirishi mumkin: Aziz 2 ta kitob olgan, shuning uchun natijada Aziz 2 qatorda chiqadi. Bu xato emas — har bir ijara alohida voqea.

LEFT JOIN — chap jadval to'liq, o'ngdan mosi bo'lmasa NULL:

```
-- HAMMA a'zo ko'rinadi, kitob olmaganlarda sana NULL:  
SELECT a.ism, i.olingan_sana  
FROM azolar a  
LEFT JOIN ijaralar i ON i.azo_id = a.id;
```

💡 Bizning ma'lumotlarda 6 a'zoning hammasi kamida bitta kitob olgan, shuning uchun hozircha ikkala query bir xil natija beradi. Farqni o'z ko'zingiz bilan ko'rish uchun kitob olmagan yangi a'zo qo'shing:

```
INSERT INTO azolar (ism, telefon, qoshilgan_sana)  
VALUES ('Sevara Nizomova', NULL, '2026-06-08');
```

Endi ikkala queryni qayta ishlating: INNER JOIN'da Sevara umuman yo'q, LEFT JOIN'da esa bor — `olingan_sana` o'rnida `NULL`. Bu qatorni o'chirmang: bob davomida va masalalarda asqotadi.

INNER JOIN vs LEFT JOIN — farq qayerda?

INNER JOIN

ism	olingan_sana
Aziz Karimov	2026-05-01
Malika Tosheva	2026-05-10
Jasur Aliyev	2026-04-01
... (davomi bor)	...

Kitob olmagan a'zo ro'yxatdan tushib qoladi

LEFT JOIN

ism	olingan_sana
Aziz Karimov	2026-05-01
Malika Tosheva	2026-05-10
Jasur Aliyev	2026-04-01
Sevara Nizomova	NULL
... (davomi bor)	...

HAMMA a'zo bor, mosi yo'q bo'lsa — NULL

Rasmda qatorlarning bir qismi ko'rsatilgan: ko'p kitob olgan a'zo (mas. Aziz) har ijarasi uchun alohida qatorda chiqadi

INNER JOIN: faqat ikkala tomonda ham mosi bor qatorlar

LEFT JOIN: chap (FROM'dagi) jadval to'liq saqlanadi

✦ **RIGHT JOIN** ham bor — LEFT'ning aksi: o'ng jadval to'liq saqlanadi. Amalda juda kam ishlatiladi, chunki jadvallarning o'rnini almashtirib LEFT JOIN yozsangiz, xuddi shu natija chiqadi-yu, query "asosiy jadval — FROM'da turadi" qoidasi bilan osonroq o'qiladi.

Oltin pattern: "hech narsa qilmaganlarni topish"

```
-- Hech qachon kitob olmagan a'zolar:  
SELECT a.*  
FROM azolar a  
LEFT JOIN ijaralar i ON i.azo_id = a.id  
WHERE i.id IS NULL;
```

Mantiq: LEFT JOIN hammaga joy beradi; ijarasi yo'qlarda ijara ustunlari NULL; biz NULL'larni ushlaymiz. Bu pattern intervyularda doim so'raladi!

Anti-join: LEFT JOIN + WHERE ... IS NULL

1) LEFT JOIN natijasi

a.ism	i.id
Aziz Karimov	1
Malika Tosheva	3
... (davomi bor)	...
Sevara Nizomova	NULL

(har ijara — alohida qator: Aziz/Malika 2 martadan)

2) WHERE i.id IS NULL

3) Kitob olmaganlar

a.ism
Sevara Nizomova
(faqat 1 qator qoldi)

LEFT JOIN hammaga joy beradi — ijarasi yo'qlarda ijara ustunlari NULL

WHERE i.id IS NULL faqat o'sha "bo'sh" qatorlarni ushlab qoladi

i.id — PRIMARY KEY: jadvalda hech qachon NULL emas, NULL faqat JOIN'dan keladi

🔗 Nega aynan `i.id`? Chunki `id` — ijaralar ning PRIMARY KEY'i, jadvalning o'zida u hech qachon NULL bo'lmaydi. Demak, natijada `i.id` NULL bo'lsa, bu faqat bitta narsani anglatadi: LEFT JOIN bu a'zoga mos ijara topa olmagan. (`i.azo_id` ham bo'laverardi — muhimi, NOT NULL ustun tanlash.)

💡 Sevara'ni qo'shgan bo'lsangiz, bu query uni topib beradi. Qo'shmagan bo'lsangiz, natija bo'sh chiqadi — bu xato emas: "hech kim topilmadi" ham javob.

Tuzoq: LEFT JOIN + WHERE = yashirin INNER JOIN

LEFT JOIN yozdingizmi — o'ng jadval ustuniga WHERE'da shart qo'yishdan ehtiyot bo'ling:

```
-- Maqsad: HAMMA a'zo + faqat iyun ijaralari. Lekin...
SELECT a.ism, i.olingan_sana
FROM azolar a
LEFT JOIN ijaralar i ON i.azo_id = a.id
WHERE i.olingan_sana >= '2026-06-01';    -- ❌ kitob olmaganlar
yo'qolib qoldi!
```

Nega? 11-bobdagi bajarilish tartibini eslang: WHERE JOIN'dan **keyin** ishlaydi. Kitob olmaganlarda `i.olingan_sana` NULL, NULL bilan taqqoslash esa hech qachon ROST bo'lmaydi (8-bob) — shu qatorlar filtrdan o'tolmaydi va LEFT JOIN amalda INNER JOIN'ga aylanib qoladi. Yechim — shartni `ON` ichiga ko'chirish:

```
SELECT a.ism, i.olingan_sana
FROM azolar a
LEFT JOIN ijaralar i ON i.azo_id = a.id
                     AND i.olingan_sana >= '2026-06-01'; -- ✅ hamma
a'zo joyida
```

Qoida: **o'ng** jadvalga oid shart — **ON** ichiga, **chap** jadvalga oid shart — **WHERE** ga.
(Yagona istisno — yuqoridagi anti-join: u yerda **IS NULL** ataylab **WHERE** da turadi.)

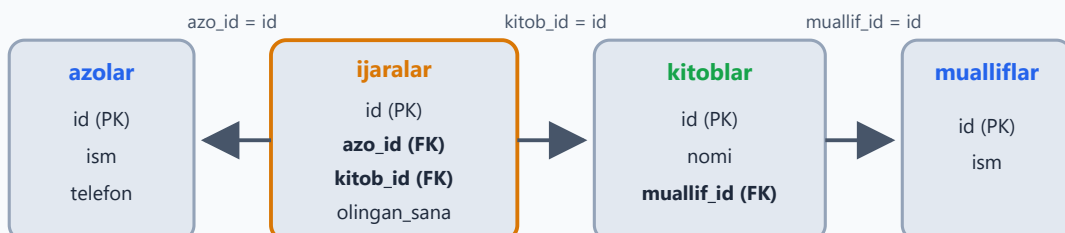
3 va undan ko'p jadval

"Kim qaysi kitobni qachon olgan?" — bu savolga bitta jadval javob bera olmaydi: ismlar **azolar** da, nomlar **kitoblar** da, sanalar **ijaralar** da. E'tibor bering: **ijaralar** — "ko'prik" jadval, ikkala FK ham unda turibdi. Shuning uchun FROM'ni undan boshlash qulay:

```
-- Kim qaysi kitobni qachon olgan (3 jadval):
SELECT a.ism AS azo, k.nomi AS kitob, i.olingan_sana
FROM ijaralar i
JOIN azolar a ON a.id = i.azo_id
JOIN kitoblar k ON k.id = i.kitob_id;

-- 4 jadval — muallifgacha:
SELECT a.ism AS azo, k.nomi AS kitob, m.ism AS muallif
FROM ijaralar i
JOIN azolar a ON a.id = i.azo_id
JOIN kitoblar k ON k.id = i.kitob_id
JOIN mualliflar m ON m.id = k.muallif_id;
```

3-4 jadval zanjiri: ijaralar — ko'prik jadval



FROM ijaralar'dan boshlaymiz — ikkala FK shu yerda turibdi
Keyin har bir FK orqali kerakli jadvalni JOIN bilan ulayveramiz

Qoida oddiy: har yangi jadval o'zining `JOIN ... ON ...` qatori bilan keladi va zanjirga bitta FK orqali ulanadi. 10 ta jadval bo'lsa ham mantiq o'zgarmaydi.

JOIN + GROUP BY = real hisobotlar

11-bobdagi GROUP BY'ga JOIN'ni qo'shsak, ish suhbatlaridagi klassik savol chiqadi: "har bir muallif nechta kitob yozgan — id emas, **ismi** bilan":

```
-- Har muallifning kitoblari soni (ismi bilan!):  
SELECT m.ism, COUNT(k.id) AS kitoblar_soni  
FROM mualliflar m  
LEFT JOIN kitoblar k ON k.muallif_id = m.id  
GROUP BY m.id, m.ism;
```

Nega LEFT? Kitobi yo'q muallif ham ro'yxatda 0 bilan ko'rinsin. Hozirgi ma'lumotlarda hamma muallifning kitobi bor — 0 ni ko'rish uchun bitta kitobsiz muallif qo'shing (buni ham o'chirmang, masalalarda kerak bo'ladi):

```
INSERT INTO mualliflar (ism, davlat, tugilgan_yil)  
VALUES ('Oybek', 'Ozbekiston', 1905);
```

ism	kitoblar_soni
Abdulla Qodiriy	2
Xudoyberdi Toxtaboyev	2
...	...
Oybek	0

`COUNT(k.id)` — NULL'larni sanamaydi, shuning uchun Oybek'da 0 chiqadi. `COUNT(*)` bo'lsa 1 chiqardi — tuzoq! Chunki LEFT JOIN Oybek uchun ham bitta qator yaratadi (kitob ustunlari NULL bo'lsa ham), `COUNT(*)` esa qatorning o'zini sanayveradi.

✦ Nega `GROUP BY m.id, m.ism` — ikkalasi ham? MySQL 8 da `ONLY_FULL_GROUP_BY` rejimi default yoqilgan: SELECT'dagi har bir "oddiy" ustun GROUP BY'da bo'lishi (yoki undagi ustunga funksional bog'liq bo'lishi) shart.

`m.id` PRIMARY KEY bo'lgani uchun MySQL 8 `GROUP BY m.id` ning o'ziga ham ruxsat beradi, lekin ikkalasini yozish — boshqa SQL tizimlarida ham ishlaydigan eng xavfsiz uslub. Faqat `GROUP BY m.ism` deb yozish esa xavfli: ikki muallifning ismi bir xil bo'lsa, ular bitta qatorga qo'shilib ketadi — id bo'yicha guruhlash bundan saqlaydi.

12-bob masalalari

💡 5-, 7- va 8-masalalar uchun yuqorida qo'shgan Sevara va Oybek qatorlari asqotadi.

1. (kutubxona) Kitob nomi + muallif ismi
2. (kutubxona) Kitob nomi + muallif ismi + muallif davlati, faqat o'zbek mualliflar
3. (kutubxona) Kim qaysi kitobni olgan (a'zo ismi + kitob nomi)
4. (kutubxona) Hozir qaytarilmagan kitoblar: kitob nomi + kimda + qachondan beri
5. (kutubxona) Hech qachon kitob olmagan a'zolar (anti-join!)
6. (kutubxona) Hech kim olmagan kitoblar
7. (kutubxona) Har muallif nechta kitob yozgan (ismi bilan, kitobsizlar 0 bilan)
8. (kutubxona) Har a'zo nechta kitob olgan (ismi bilan, olmaganlar 0 bilan)
9. (kutubxona) Har kitob necha marta ijaraga berilgan, eng mashhuridan boshlab
10. (kutubxona) 4 jadvalli query: a'zo + kitob + muallif + olingan sana
11. (dokon) Mahsulot nomi + kategoriya nomi
12. (dokon) Buyurtmalar: mijoz ismi + sana + holat
13. (dokon) Buyurtma tarkibi: buyurtma id + mahsulot nomi + soni + narx (buyurtma_qatorlari JOIN mahsulotlar)
14. (dokon) Hech narsa buyurtma qilmagan mijozlar
15. (dokon) Hech qachon sotilmagan mahsulotlar (buyurtma_qatorlari'da yo'qlar)
16. (dokon) Har mijoz jami qancha pulga xarid qilgan (3 jadval + GROUP BY):
mijozlar → buyurtmalar → buyurtma_qatorlari, SUM(narx*soni); 'bekor'
buyurtmalarni chiqarib tashlang!
17. (klinika) Qabullar: bemor ismi + shifokor ismi + sana + tashxis
18. (klinika) Har shifokor (ismi bilan) nechta bemor ko'rgan va qancha yig'gan;
birorta qabul qilmaganlar ham 0 bilan ko'rinsin

19. (taksi) Har haydovchi + mashinasi modeli + raqami; mashinasiz haydovchi bormi — LEFT JOIN bilan tekshiring
20. (taksi) Har haydovchi (ismi bilan): safarlar soni, jami daromad, o'rtacha baho — daromad bo'yicha kamayish tartibida

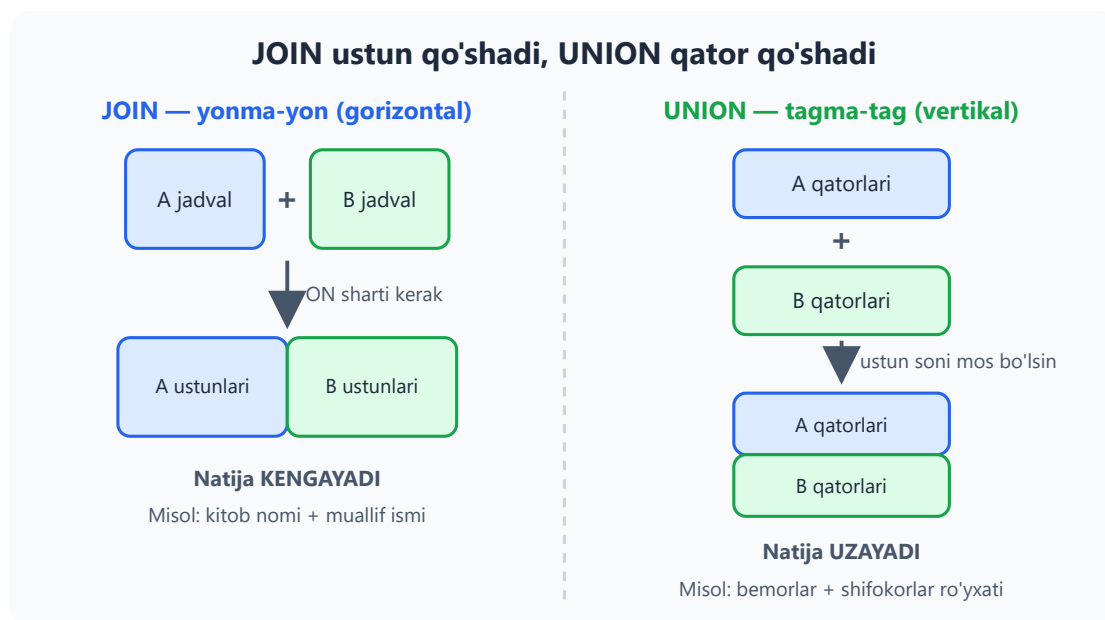
13 — UNION — natijalarni birlashtirish

◀ Oldingi: 12 — JOIN — jadvallarni bog'lash · 🏠 README · Keyingi: 14 — Subquery — query ichida query ➡

Bu bobda: bir nechta SELECT natijasini bitta ro'yxatga "tagma-tag" ulaydigan UNION va UNION ALL'ni o'rganamiz: ularning farqini (takrorlar taqdiri!), qoidalarini, ustun yetishmaganda NULL bilan to'ldirishni, UNION'da ORDER BY va LIMIT ishlatish sirlarini hamda JOIN bilan adashtirmaslik yo'llarini ko'rib chiqamiz.

JOIN'dan nimasi farq qiladi?

JOIN ustunlarni "yonma-yon" qo'shsa, UNION qatorlarni "tagma-tag" qo'shadi. Tasavvur qiling: ikkita sinf jurnalidan bitta umumiy ro'yxat tuzyapsiz — ikkinchi ro'yxatni birinchisining **tagiga** ko'chirib yozasiz. Jadval kengaymaydi, **uzayadi**.



```
-- Klinikadagi hamma odamlar ro'yxati (bemor ham, shifokor ham):  
SELECT ism, 'bemor' AS turi FROM klinika.bemorlar  
UNION ALL  
SELECT ism, 'shifokor' AS turi FROM klinika.shifokorlar;
```

Natijada 11 qator chiqadi (6 bemor + 5 shifokor):

ism	turi
Karim Abdullayev	bemor
Lola Mirzayeva	bemor
... (jami 6 bemor)	bemor
Dr. Anvar Sodiqov	shifokor
... (jami 5 shifokor)	shifokor

💡 `'bemor'` va `'shifokor'` — qo'lda yozilgan belgi ustuni. Bu juda foydali odat: bo'lmasa, natijadagi qator qaysi jadvaldan kelganini bilolmay qolasiz.

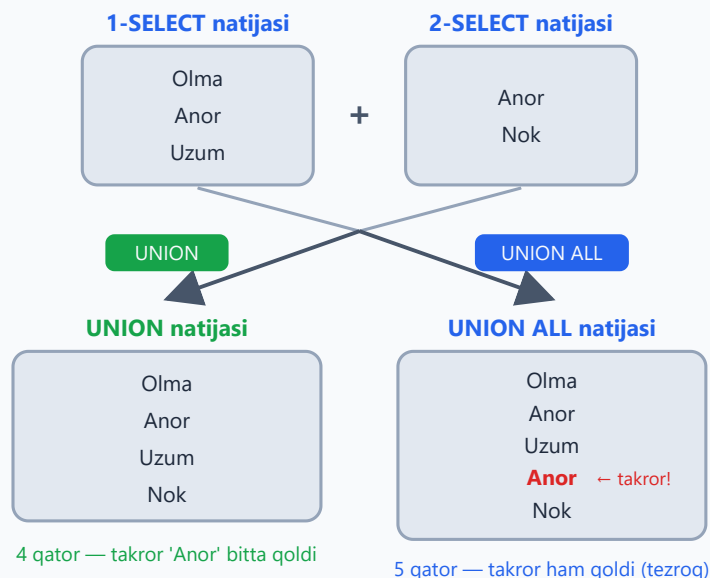
Qoidalar

- Ustunlar **soni** har bir SELECT'da bir xil bo'lishi **shart** — bo'lmasa MySQL xato beradi
- Ustunlar **turi** mos kelgani ma'qul: turlicha bo'lsa MySQL o'zi moslashtirishga urinadi, lekin son ustiga matn ulasangiz g'alati natija chiqishi mumkin
- Natijadagi ustun **nomlari birinchi SELECT'dan** olinadi — keyingi SELECT'larda `AS` yozish shart emas
- SELECT'lar mustaqil: turli jadvaldan, hatto turli bazadan bo'lishi mumkin (`baza.jadval` sintaksisi), bitta jadvalning o'zidan ham bo'laveradi

UNION vs UNION ALL — takrorlar taqdiri

- `UNION` — takror qatorlarni **olib tashlaydi** (xuddi DISTINCT kabi; buning uchun MySQL hamma qatorni o'zaro solishtirib chiqadi — **sekinroq**)
- `UNION ALL` — hammasini **qoldiradi** (hech narsa tekshirmaydi — **tezroq**)

UNION va UNION ALL — farq takrorlarda



Buni o'z ko'zingiz bilan ko'ring:

```
USE kutubxona;

SELECT janr FROM kitoblar
UNION
SELECT janr FROM kitoblar;
-- 4 qator: roman, sarguzasht, sheriyat, ertak

SELECT janr FROM kitoblar
UNION ALL
SELECT janr FROM kitoblar;
-- 20 qator: 10 kitobning janri ikki marta
```

✦ Diqqat: `UNION` takrorlarni faqat ikki `SELECT` **orasida** emas, **butun natija** bo'ylab o'chiradi — yuqoridagi misolda bitta `SELECT` ichidagi takror janrlar ham ketdi.

Qaysi birini tanlash? Oddiy qoida: takror bo'lishi mumkin emasligini bilsangiz (masalan, bemorlar va shifokorlar ro'yxati kesishmaydi) — doim `UNION ALL` ishlatib, bekorga tekshiruvga vaqt sarflamang. Takror chiqishi mumkin va u sizga kerak bo'lmasa — `UNION`.

Ustun yetishmasa — NULL bilan to'ldiring

Ustunlar soni teng bo'lishi shartligini aytdik. Lekin bir jadvalda bor ustun ikkinchisida bo'lmasa-chi? O'rnini `NULL` (yoki biror standart qiymat) bilan to'ldirib qo'ying:

```
-- Bemorlarda telefon bor, shifokorlarda bunday ustun yo'q:
SELECT ism, telefon FROM klinika.bemorlar
UNION ALL
SELECT ism, NULL FROM klinika.shifokorlar;
```

ORDER BY va LIMIT — UNION bilan

`ORDER BY` ni **eng oxiriga** yozsangiz, u **butun birlashgan natijaga** qo'llanadi:

```
-- Arzon va qimmat mahsulotlar bitta ro'yxatda, belgisi bilan, narx
tartibida:
SELECT nomi, narx, 'arzon' AS toifa FROM dokon.mahsulotlar WHERE narx <
200000
UNION ALL
SELECT nomi, narx, 'qimmat' FROM dokon.mahsulotlar WHERE narx >
100000000
ORDER BY narx;
```

Agar har bir `SELECT`'ga **alohida** `ORDER BY ... LIMIT` kerak bo'lsa, uni **qavsga** oling:

```
-- Eng qimmat 3 ta + eng arzon 3 ta mahsulot bitta natijada:
(SELECT nomi, narx FROM dokon.mahsulotlar ORDER BY narx DESC LIMIT 3)
UNION ALL
(SELECT nomi, narx FROM dokon.mahsulotlar ORDER BY narx ASC LIMIT 3);
```

✦ Qavs ichidagi `ORDER BY` faqat `LIMIT` bilan birga ma'noga ega — u "qaysi 3 tani olishni" hal qiladi, xolos. Yakuniy natijaning tartibi kafolatlanmaydi; tartib kerak bo'lsa, eng oxiriga yana bitta `ORDER BY` qo'shing.

JOIN va UNION — adashtirmang

	JOIN	UNION
Yo'nalishi	gorizontal — ustun qo'shadi	vertikal — qator qo'shadi
Sharti	<code>ON</code> bog'lash sharti kerak	shart yo'q, faqat ustun soni mos bo'lsin
Qachon kerak	bir qatorga qo'shimcha ma'lumot ulash (kitob + muallif ismi)	ikki o'xshash ro'yxatni bitta qilish (bemorlar + shifokorlar)

💡 Bitta jadvalning o'zidan ikki xil shart bilan tanlayotgan bo'lsangiz, ko'pincha UNION shart ham emas — oddiy `WHERE ... OR ...` yetadi (buni 20-masalada sinaysiz).

13-bob masalalari

- (klinika) Bemorlar va shifokorlar ismlarini bitta ro'yxatda chiqaring
- (klinika) O'sha ro'yxatga 'turi' ustuni qo'shing ('bemor'/'shifokor')
- (klinika) Hamma telefon raqamlar: bemorlardan ham, shifokorlardan ham... shifokorlarda telefon yo'q-ku! O'rniga NULL chiqaring: `SELECT ism, telefon FROM bemorlar UNION ALL SELECT ism, NULL FROM shifokorlar`
- (kutubxona) 1900-yilgacha va 1970-yildan keyingi kitoblar bitta ro'yxatda, 'davr' belgisi bilan ('eski'/'yangi')
- (dokon) Tugagan mahsulotlar ('tugagan' belgisi) + 5 tadan kam qolganlar ('oz') — UNION ALL
- (taksi) 5 baho olgan safarlar ('alo') + 3 va undan past baholilar ('yomon')
- (kutubxona) UNION va UNION ALL farqini sinash: `SELECT janr FROM kitoblar UNION SELECT janr FROM kitoblar;` va ALL versiyasi — qator sonlarini solishtiring
- (kutubxona+dokon) Ikki BAZAdan: kutubxona a'zolari va do'kon mijozlari ismlari bitta ro'yxatda (baza.jadval sintaksisi)
- Ataylab xato: 2 ustunli SELECT'ni 3 ustunli bilan UNION qiling — xatoni o'qing
- (dokon) Eng qimmat 3 ta va eng arzon 3 ta mahsulot bitta natijada (har SELECT'ni qavsga oling: `(SELECT ... LIMIT 3) UNION ALL (SELECT ... LIMIT 3)`)
- (klinika) May oyidagi va iyun oyidagi qabullar, 'oy' belgisi bilan

12. (taksi) Eng uzun safar + eng qisqa safar (qavsli LIMIT usuli)
13. (kutubxona) Roman janri kitoblari nomi + mualliflar ismi — bitta "qidiruv natijasi" ro'yxatida ('kitob'/'muallif' belgisi bilan)
14. (dokon) Toshkentlik mijozlar + 10 mln dan qimmat mahsulot sotib olganlar (hozircha takror bo'lsa bo'ladi — UNION bilan takrorni oling)
15. ORDER BY UNION'da: 10-masala natijasini narx bo'yicha saralang (ORDER BY eng oxirida, butun natijaga qo'llanadi)
16. (hammasi) 4 bazadan 'odamlar' ro'yxati: a'zolar, mijozlar, bemorlar, haydovchilar — turi belgisi bilan, ism bo'yicha alifbo tartibida
17. (dokon) Har oy buyurtmalari sonini alohida SELECT'larda hisoblab (WHERE MONTH(sana)=5 , =6), UNION ALL bilan birlashting — keyin 11-bobdagi GROUP BY usuli bilan solishtiring: qaysi yaxshi?
18. (taksi) 'Chilonzor'dan boshlangan va 'Chilonzor'da tugagan safarlar — UNION (takrorsiz) va UNION ALL (takror bilan) — farq bormi?
19. (kutubxona) Telefoni bor a'zolar + telefoni yo'q a'zolar UNION ALL — jami a'zolar soniga tengmi? Tekshiring
20. O'ylang: qaysi holatda UNION o'rniga oddiy WHERE ... OR ... yetarli? (bitta jadvaldan bo'lsa!) 18-masalani OR bilan qayta yozing

14 — Subquery — query ichida query

◀ Oldingi: 13 — UNION — natijalarni birlashtirish · 🏠 README · Keyingi: 15 — CTE — WITH bilan ishlash ▶

Bu bobda: query ichiga yana bitta query joylashni — subquery'ni o'rganamiz: bitta qiymat qaytaradigan (skalyar), ro'yxat qaytaradigan (IN bilan) va tashqi qatorga "qarab turadigan" (correlated, EXISTS) turlarini ko'ramiz, mashhur NOT IN + NULL tuzog'idan qanday qochishni va FROM ichidagi subquery (hosila jadval) bilan ikki bosqichli hisob-kitob qilishni mashq qilamiz.

G'oya — savol ichidagi savol

"O'rtachadan qimmat mahsulotlar qaysilar?" — bu savolga bitta oddiy WHERE bilan javob berib bo'lmaydi, chunki savol aslida **ikki bosqichli**: avval o'rtacha narxning o'zini bilish kerak, keyin esa har bir mahsulotni shu son bilan solishtirish kerak.

Qo'lda yechsak bo'lardi: avval `SELECT AVG(narx) ...` bajarib natijani qog'ozga yozib olamiz, keyin uni qo'lda WHERE ga ko'chiramiz. Ishlaydi, lekin nochor usul — ma'lumot o'zgarishi bilan qog'ozdagi son eskiradi. SQL buni bitta queryda hal qiladi: **subquery** — query ichidagi query. Ichki query avval bajariladi, natijasi tashqisiga uzatiladi:

```
SELECT nomi, narx
FROM dokon.mahsulotlar
WHERE narx > (SELECT AVG(narx) FROM dokon.mahsulotlar);
-- Natija: 5 ta mahsulot (iPhone 15, MacBook Air M2, Acer Aspire 5, ...)
```

MySQL avval qavs ichidagi `SELECT AVG(narx)` ni bajaradi — bizning bazada bu ≈ 5235833 . Keyin tashqi query xuddi `WHERE narx > 5235833.33` deb yozilganday ishlayveradi. Qoida sodda: **ichkaridan tashqariga**.

Subquery qanday bajariladi: ichkaridan tashqariga

1

Ichki query **AVVAL** bajariladi

`SELECT AVG(narx) FROM mahsulotlar`

→ `≈ 5 235 833`



2

Natija tashqi query'ga qiymat bo'lib qo'yiladi

`WHERE narx > (SELECT AVG(narx) ...)`
→ `WHERE narx > 5 235 833`



3

Tashqi query odatdagidek ishlaydi

Natija: o'rtachadan qimmat 5 ta mahsulot

✦ Subquery **doim qavs** () **ichida** yoziladi. Qavsni unutsangiz — sintaksis xatosi.

💡 O'qish va yozishni osonlashtiradigan hiyla: avval ichki query'ni **alohida bajarib ko'ring** — nima qaytarayotganini o'z ko'zingiz bilan ko'rasiz, keyin tashqi query'ga "joylaysiz". Murakkab querylar shunday ichkaridan quriladi.

Skalyar subquery — bitta qiymat

Bitta qator, bitta ustun — ya'ni **bitta qiymat** qaytaradigan subquery skalyar deyiladi. Uni oddiy son turgan istalgan joyda (= , > , < yonida) ishlatish mumkin:

```
-- Eng qalin kitob:  
SELECT nomi, sahifa FROM kitoblar  
WHERE sahifa = (SELECT MAX(sahifa) FROM kitoblar);  
-- Natija: Urush va tinchlik (1225 sahifa)
```

9-bobda buni `ORDER BY sahifa DESC LIMIT 1` bilan yechgan edik. Ikkalasi ham to'g'ri, lekin nozik farq bor: agar ikki kitob bir xil 1225 sahifa bo'lsa, `LIMIT 1` faqat **bittasini** ko'rsatadi, subquery esa **hammasini** chiqaradi. "Eng katta qiymatga ega HAMMA qatorlar" kerak bo'lganda subquery — ishonchli yo'l.

⚠ Skalyar kutilgan joyda subquery **bir nechta qator** qaytarsa, MySQL xato beradi: `ERROR 1242: Subquery returns more than 1 row`. Masalan, `WHERE sahifa`

= (SELECT sahifa FROM kitoblar) — 10 ta qiymatning qaysi biri bilan solishtirsin? Bunday holda = o'rniga IN ishlatish yoki ichki query'ni MAX / AVG kabi aggregate bilan bitta qiymatga keltirish.

💡 Subquery hech narsa qaytarmasa — bu xato emas: natija NULL deb olinadi va tashqi WHERE hech narsani o'tkazmaydi (8-bobdagi NULL qoidalarini eslang).

Skalyar subquery SELECT ro'yxatida ham tura oladi — har qator yoniga umumiy ko'rsatkichni qo'yish uchun qulay:

```
-- Har mahsulot narxi va umumiy o'rtacha yonma-yon:
SELECT nomi, narx,
       (SELECT AVG(narx) FROM dokon.mahsulotlar) AS ortacha
FROM dokon.mahsulotlar;
```

Ro'yxat qaytaradigan subquery — IN bilan

Subquery bir ustunli **ro'yxat** qaytarsa, uni 8-bobdan tanish IN bilan ishlatamiz:

```
-- O'zbek mualliflarning kitoblari:
SELECT nomi, yil FROM kitoblar
WHERE muallif_id IN (SELECT id FROM mualliflar WHERE davlat =
                     'O'zbekiston');
-- Ichki query (1, 2, 3) ro'yxatini beradi → 6 ta kitob chiqadi
```

Farqi shundaki, IN (1, 2, 3) ro'yxatini endi qo'lda yozmaymiz — baza o'zi topib beradi. Mualliflar jadvaliga yangi o'zbek muallif qo'shilsa, query o'zgarmasdan to'g'ri ishlayveradi.

✦ IN ga beriladigan subquery **bitta ustun** tanlashi kerak. SELECT id, ism FROM ... yozsangiz: ERROR 1241: Operand should contain 1 column(s).

Teskari savol uchun NOT IN:

```
-- Hech kim olmagan kitoblar:
SELECT nomi FROM kitoblar
WHERE id NOT IN (SELECT kitob_id FROM ijaralar);
-- Natija: 4 ta kitob (Ozbeqim, Anna Karenina, ...)
```

Lekin NOT IN da yashirin tuzoq bor — quyida alohida bo'limda ko'ramiz.

EXISTS va correlated subquery

EXISTS boshqacha savol beradi: "shunday qator **bormi?**" — qiymatning o'zi emas, bor-yo'qligi muhim:

```
-- Kamida bir marta kitob olgan a'zolar:  
SELECT * FROM azolar a  
WHERE EXISTS (SELECT 1 FROM ijaralar i WHERE i.azo_id = a.id);
```

Ichki querydagi `SELECT 1` — odat bo'lib qolgan yozuv: nima tanlangani ahamiyatsiz, MySQL faqat qator topildimi-yo'qmi deb qaraydi.

E'tibor bering: ichki query `a.id` ni — **tashqi** querydagi qatorni ishlatyapti. Bunday subquery **correlated (bog'langan)** deyiladi: u bir marta emas, tashqi querydagi **har bir qator uchun** qayta tekshiriladi — Aziz uchun bir marta, Malika uchun bir marta va hokazo. Bizning bazada hamma a'zo kamida bir marta kitob olgan, shuning uchun 6 qator chiqadi. Teskari savol uchun — `NOT EXISTS`.

Correlated subquery skalyar joyda ham juda chiroyli ishlaydi — har qatorni "o'z guruhi" bilan solishtirish mumkin:

```
-- O'z janrining o'rtachasidan qalin kitoblar:  
SELECT nomi, janr, sahifa  
FROM kitoblar k  
WHERE sahifa > (SELECT AVG(sahifa) FROM kitoblar WHERE janr = k.janr);  
-- Natija: 4 ta kitob — har janrning o'z "qalinlari"
```

Bu yerda har kitob umumiy o'rtacha bilan emas, **janrdoshlari** o'rtachasi bilan solishtiriladi: roman — romanlar bilan, ertak — ertaklar bilan.

⚠ Correlated subquery har qator uchun bajarilgani sababli juda katta jadvallarda sekinlashishi mumkin. MySQL 8 optimizatori ko'p hollarda buni o'zi samaraliroq rejaga aylantiradi, lekin shubha tug'ilsa — 23-bobdagi `EXPLAIN` bilan tekshiramiz.

Subquery'ning uch turi

Skalyar

bitta qiymat qaytaradi

```
narx > (SELECT  
AVG(narx) ...)
```

=, >, < bilan ishlatiladi

Ko'p qator qaytsa —
ERROR 1242!

Ro'yxat (IN)

bir ustunli ro'yxat

```
id IN (SELECT  
kitob_id ...)
```

IN / NOT IN bilan
xuddi (1, 2, 3) ro'yxati
kabi ishlaydi

Correlated

tashqi qatorga bog'langan

```
EXISTS (SELECT 1 ...  
i.azo_id = a.id)
```

tashqi querydagi HAR BIR
qator uchun qayta
tekshiriladi

Subquery doim qavs () ichida yoziladi — avval ichkisini alohida sinab ko'ring

NOT IN va NULL tuzog'i

Ogohlantirish — bu xato real loyihalarda juda ko'p uchraydi. Subquery natijasida **birorta NULL** bo'lsa, **NOT IN** **hech narsa** qaytarmaydi!

Nega? **NOT IN** aslida ketma-ket tengsizliklar zanjiri:

```
id NOT IN (2, 5, NULL)  
-- ma'nosi: id <> 2 AND id <> 5 AND id <> NULL
```

8-bobdan eslang: `id <> NULL` — na rost, na yolg'on, javobi NOMA'LUM. **AND** zanjirida bitta NOMA'LUM bo'lsa, butun ifoda hech qachon aniq ROST bo'lolmaydi — **WHERE** esa faqat aniq ROST qatorlarni o'tkazadi. Natija: 0 qator, va eng xavflisi — **xato xabari ham chiqmaydi**, query "jimgina" bo'sh natija beradi.

NOT IN + NULL tuzog'i

`id NOT IN (2, 5, NULL) = id <> 2 AND id <> 5 AND id <> NULL`

Tuzoq

`id <> NULL` → javobi NOMA'LUM
AND zanjiri ROST bo'lolmaydi
WHERE faqat ROSTni o'tkazadi

Natija: 0 qator!

Yechim

- 1) NOT EXISTS ishlatish:
`WHERE NOT EXISTS (...)`
- 2) NULL'larni chiqarib tashlang:
`... WHERE ustun IS NOT NULL`

Bittagina NULL butun NOT IN natijasini bo'shatadi — xato xabari ham chiqmaydi!

NOT EXISTS bu tuzoqqa hech qachon tushmaydi

Bizning `ijaralar.kitob_id` ustuni `NOT NULL` deb e'lon qilingan, shuning uchun yuqoridagi misol bexavotir ishladi. Lekin bunga tayanib o'tirmang — ikkita xavfsiz yo'l bor:

```
-- 1-yo'l: NOT EXISTS (eng ishonchlisi — NULL'ga umuman parvo qilmaydi):
SELECT nomi FROM kitoblar k
WHERE NOT EXISTS (SELECT 1 FROM ijaralar i WHERE i.kitob_id = k.id);

-- 2-yo'l: NULL'larni ichki query'ning o'zida chiqarib tashlash:
SELECT nomi FROM kitoblar
WHERE id NOT IN (SELECT kitob_id FROM ijaralar WHERE kitob_id IS NOT NULL);
```

✦ Qoida qilib oling: `NOT IN` + subquery yozayotganda doim "ichkarida NULL bo'lishi mumkinmi?" deb so'rang. Ikkilansangiz — `NOT EXISTS` yozing, u bu tuzoqqa hech qachon tushmaydi.

FROM ichida subquery — hosila jadval

Subquery `WHERE` dan tashqari `FROM` da ham tura oladi — u holda natijasi **vaqtinchalik jadvaldek** ishlatiladi (rasmii nomi: hosila jadval, derived table):

```
-- Har buyurtma summasi ichidan eng kattasini topish:
SELECT MAX(jami) AS eng_katta
FROM (
    SELECT buyurtma_id, SUM(narx * soni) AS jami
    FROM dokon.buyurtma_qatorlari
```

```
GROUP BY buyurtma_id
) AS summalar;
-- Natija: 16 000 000
```

O'qilishi — ikki bosqich: 1) ichki query har buyurtmaning jami summasini hisoblaydi, kichik "jadvalcha" hosil bo'ladi; 2) tashqi query shu jadvalchadan `MAX` oladi. To'g'ridan-to'g'ri `MAX(SUM(...))` deb yozib bo'lmaydi — MySQL aggregate ichida aggregate'ga ruxsat bermaydi (`ERROR 1111: Invalid use of group function`). `FROM` subquery aynan shu muammoni yechadi.

✦ `FROM` dagi subquery'ga **alias majburiy** (`AS summalar`). Unutsangiz: `ERROR 1248: Every derived table must have its own alias` — bu yerdagi eng ko'p uchraydigan xato.

💡 `FROM` subquery o'sgan sari o'qish qiyinlashadi. Keyingi bobda xuddi shu ishni ancha ozoda yozadigan vosita — CTE (`WITH`) bilan tanishamiz.

Subquery yoki JOIN?

Ko'p masalani ikkala yo'l bilan ham yechish mumkin — masalan, "o'zbek mualliflarning kitoblari"ni `IN` subquery ham, 12-bobdagi `JOIN` ham topadi. Qaysi birini tanlash kerak?

Holat	Qulayroq vosita
Ikkinchi jadval faqat filtr uchun kerak, ustunlari natijaga chiqmaydi	subquery (<code>IN</code> / <code>EXISTS</code>)
Ikkala jadval ustunlari ham natijada kerak	<code>JOIN</code>
"Hech narsa qilmaganlar"ni topish	<code>NOT EXISTS</code> yoki <code>LEFT JOIN + IS NULL</code> — ikkalasi ham to'g'ri
Aggregate natijasini yana aggregatlash	<code>FROM</code> subquery (yoki 15-bobdagi CTE)

💡 Tezlik haqida ortiqcha tashvishlanmang: MySQL 8 optimizatori `IN` subquery'larni ichkarida baribir `JOIN`'ga o'xshash rejaga (semi-join) aylantiradi — ko'p hollarda tezlik bir xil. Asosiy mezon: **qaysi yozuv sizga tushunarli bo'lsa, o'shani yozing.**

14-bob masalalari

1. (kutubxona) O'rtacha sahifadan qalin kitoblar
2. (kutubxona) Eng eski kitob (MIN subquery bilan — LIMIT'siz usul!)
3. (kutubxona) Eng ko'p nusxali kitob
4. (kutubxona) O'zbek mualliflarning kitoblari (IN + subquery; keyin JOIN bilan ham yozing — solishtiring)
5. (kutubxona) 1900-yilgacha tug'ilgan mualliflarning kitoblari
6. (kutubxona) Kamida bir marta ijaraga berilgan kitoblar (IN yoki EXISTS)
7. (kutubxona) Hech qachon ijaraga berilmagan kitoblar (NOT EXISTS)
8. (kutubxona) Hech qachon kitob olmagan a'zolar — subquery usulida (12-bobdagi LEFT JOIN bilan solishtiring: ikkalasi ham to'g'ri! Bo'sh natija chiqsa xafa bo'lmang — bizning bazada hamma a'zo kitob olgan)
9. (kutubxona) 'Sariq devni minib' kitobini olgan a'zolar ismi (2 qavat subquery: kitob nomi → kitob_id → azo_id → ism)
10. (dokon) O'rtacha narxdan arzon mahsulotlar
11. (dokon) Eng qimmat mahsulot (subquery usuli)
12. (dokon) Buyurtma qilingan mahsulotlar (IN)
13. (dokon) Hech qachon buyurtma qilinmagan mahsulotlar (NOT IN — ehtiyot: mahsulot_id NULL emasligini tekshiring!)
14. (dokon) Toshkentlik mijozlarning buyurtmalari (mijoz_id IN subquery)
15. (dokon) O'rtacha buyurtma summasidan katta buyurtmalar (FROM subquery + HAVING yoki 2 bosqich)
16. (klinika) Eng qimmat qabul narxli shifokor
17. (klinika) O'rtacha tajribadan tajribali shifokorlar
18. (klinika) Kardiologga tushgan bemorlar ismi (2 qavat: mutaxassislik → shifokor_id → bemor_id → ism)
19. (taksi) O'rtacha masofadan uzun safarlar; reytingi eng baland haydovchining safarlari — 2 ta query
20. (taksi) Har haydovchining daromadi o'rtacha haydovchi daromadidan ko'p bo'lganlari (FROM subquery: avval GROUP BY bilan daromadlar jadvalini yasang, undan AVG oling)

15 — CTE — WITH bilan ishlash

◀ Oldingi: 14 — Subquery — query ichida query · 🏠 README · Keyingi: 16 — Window funksiyalar ▶

Bu bobda: murakkab query'ni `WITH` yordamida nomli, yuqoridan pastga o'qiladigan bosqichlarga bo'lishni, bir nechta CTE'ni zanjir qilib ulashni va recursive CTE bilan jadvalsiz sonlar ketma-ketligi, kalendar va daraxt strukturalar hosil qilishni o'rganamiz.

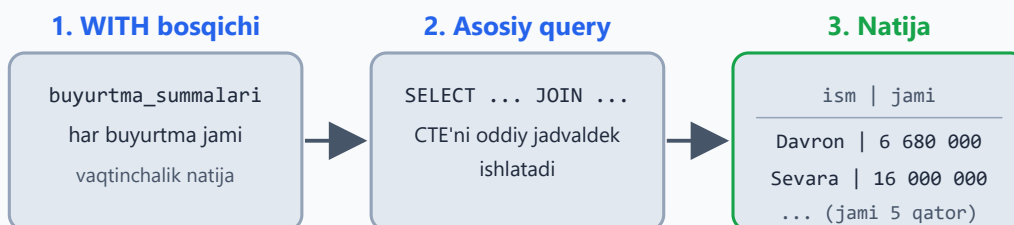
G'oya: query bo'lagiga nom qo'yamiz

14-bobda subquery'ni ko'rdik — query ichida query. Ishlaydi, lekin subquery ichida yana subquery paydo bo'lsa, o'qish azobga aylanadi. CTE (Common Table Expression) — subquery'ning **chiroyli** versiyasi: murakkab query'ni nomli bo'laklarga ajratadi. Xuddi oshxona retseptidek: "avval qiymani tayyorlang, keyin xamirni yoying, oxirida ikkalasini qo'shing" — har bosqichning o'z nomi bor, tartibi aniq:

```
WITH buyurtma_summalari AS (  
    SELECT buyurtma_id, SUM(narx * soni) AS jami  
    FROM dokon.buyurtma_qatorlari  
    GROUP BY buyurtma_id  
)  
SELECT b.id, m.ism, bs.jami  
FROM buyurtma_summalari bs  
JOIN dokon.buyurtmalar b ON b.id = bs.buyurtma_id  
JOIN dokon.mijozlar m ON m.id = b.mijoz_id  
WHERE bs.jami > 3000000;
```

O'qilishi: "AVVAL buyurtma_summalari degan vaqtinchalik jadval yasa, KEYIN undan foydalanib asosiy query'ni bajar". Asosiy SELECT uchun `buyurtma_summalari` xuddi oddiy jadvaldek: FROM'da yozasiz, JOIN qilasiz, WHERE'da filtrlaysiz.

CTE: avval nomli bosqich, keyin asosiy query



O'qilishi: "AVVAL nomli bosqichni yasa, KEYIN asosiy query'ni bajar"

CTE faqat shu bitta query davomida yashaydi — query tugashi bilan yo'qoladi

✦ CTE **faqat shu bitta query davomida** yashaydi: query tugadi — `buyurtma_summalari` ham yo'qoladi. Bazaga hech narsa yozilmaydi, hech nima o'zgarmaydi.

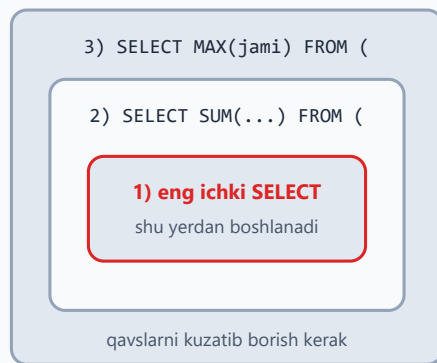
✦ CTE MySQL'ga **8.0 versiyada** kelgan. Eski MySQL 5.7 da `WITH` ishlamaydi — shu sababli internetdagi ba'zi eski darslarda hamma narsa subquery bilan yozilganini ko'rasiz.

Nega bu subquery'dan qulay?

Xuddi shu masalani FROM ichidagi subquery bilan ham yozsa bo'lardi (14-bobda shunday qildik). Farq — **o'qish tartibida**. Ichma-ich subquery'ni tushunish uchun eng ichkarisidan boshlab tashqariga qarab o'qiysiz — teskari tartib. CTE esa retseptdek yuqoridan pastga o'qiladi: 1-bosqich, 2-bosqich, natija.

Nega CTE o'qilishi oson?

Ichma-ich subquery



ichkaridan tashqariga — teskari tartib

CTE bilan



yuqoridan pastga — retseptdek o'qiladi

Qachon CTE, qachon subquery? Ma'no bir xil — MySQL ko'pincha ikkalasini bir xil rejada bajaradi. Oddiy qoida: query 5 qatordan oshsa yoki bitta oraliq natija 2 joyda kerak bo'lsa — CTE o'qilishi ancha oson. Qisqa `WHERE narx > (SELECT AVG(narx) ...)` uchun esa subquery bemalol yetadi.

Bir nechta CTE — vergul bilan

`WITH` so'zi **bir marta** yoziladi, keyingi CTE'lar vergul bilan qo'shiladi. Eng zo'r joyi: har keyingi CTE o'zidan **oldingi** CTE'lardan foydalana oladi — bosqichlar zanjiri hosil bo'ladi:

```
WITH
oylik AS (
    SELECT DATE_FORMAT(b.sana, '%Y-%m') AS oy, SUM(q.narx * q.soni) AS
tushum
    FROM dokon.buyurtmalar b
    JOIN dokon.buyurtma_qatorlari q ON q.buyurtma_id = b.id
    GROUP BY oy
),
ortacha AS (
    SELECT AVG(tushum) AS avg_tushum FROM oylik -- oldingi CTE'dan
foydalanyapti!
)
SELECT o.oy, o.tushum
FROM oylik o
CROSS JOIN ortacha a
WHERE o.tushum > a.avg_tushum;
```

Bizning ma'lumotlarda natija bitta qator bo'ladi:

oy	tushum
2026-06	27730000.00

Chunki may tushumi 23 380 000, iyun tushumi 27 730 000, o'rtacha esa taxminan 25,5 mln — faqat iyun undan oshgan.

✦ Halol izoh: soddalik uchun bu misolda `holat = 'bekor'` bo'lgan 4-buyurtmani (11 500 000, may oyi) ham "tushum"ga qo'shib yubordik — diqqatimiz CTE zanjirining o'zida. Real hisobotda esa, 12-bobning 16-masalasida o'rgatilganidek, bekor buyurtmalarni `WHERE b.holat <> 'bekor'` bilan chiqarib tashlash to'g'ri bo'ladi.

✦ `ortacha` bitta qator qaytaradi, shuning uchun uni `CROSS JOIN` bilan ulaymiz — har qatorga o'sha yagona o'rtacha qiymat "yopishtiriladi" (ON sharti kerak emas). Eski kitoblarda buni `FROM oylik o, ortacha a` deb vergul bilan yozishadi — ishlaydi, lekin JOIN turini aniq yozgan tushunarliroq.

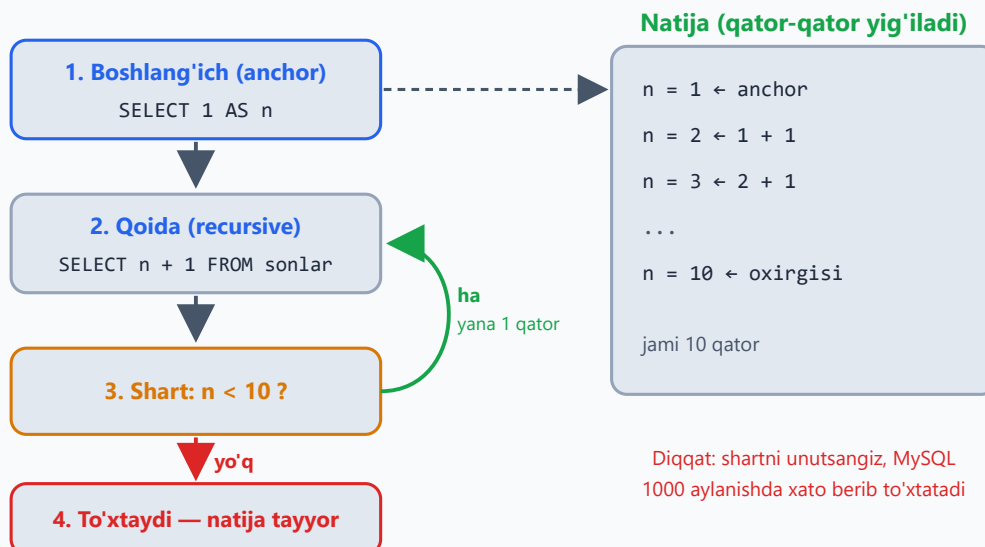
Recursive CTE — o'z-o'zini chaqiruvchi (qiziq mavzu!)

Oddiy CTE tayyor jadvallardan o'qiydi. Recursive CTE esa qatorlarni **o'zi yaratadi** — hech qanday jadvalsiz! U doim uch qismdan iborat: boshlang'ich qator (anchor), `UNION ALL`, va o'z-o'zini chaqiruvchi qoida:

```
-- 1 dan 10 gacha sonlar (jadvalsiz!):
WITH RECURSIVE sonlar AS (
    SELECT 1 AS n                                -- 1) boshlanish (anchor)
    UNION ALL
    SELECT n + 1 FROM sonlar WHERE n < 10        -- 2) qoida + to'xtash sharti
)
SELECT * FROM sonlar;
```

Qanday ishlaydi: avval `n = 1` qatori yaratiladi. Keyin qoida aylana boshlaydi: "oxirgi qatorga 1 qo'sh" — 2, 3, 4... Har aylanishda `WHERE n < 10` sharti tekshiriladi: shart bajarilmagan zahoti aylanish to'xtaydi. Natija — 1 dan 10 gacha 10 ta qator.

Recursive CTE qanday aylanadi (1 dan 10 gacha)



⚠ **To'xtash shartini unutmang!** `WHERE n < 10` bo'lmasa, qoida cheksiz aylanardi. Yaxshiyamki MySQL himoyalangan: standart sozlamada 1000 aylanishdan keyin Recursive query aborted xatosi bilan to'xtatadi (bu chegara `cte_max_recursion_depth` o'zgaruvchisida turadi). Demak, 11-masaladagi "1 dan 100 gacha" bimalol ishlaydi, "1 dan 5000 gacha" esa chegarani oshirishni talab qiladi.

Eng foydali amaliy misol — **kalendar yasash**:

```
-- 2026-yil iyun kalendari:
WITH RECURSIVE kunlar AS (
  SELECT DATE('2026-06-01') AS kun
  UNION ALL
  SELECT kun + INTERVAL 1 DAY FROM kunlar WHERE kun < '2026-06-30'
)
SELECT kun, DAYNAME(kun) FROM kunlar;
```

✳ `DAYNAME` standart sozlamada kun nomini inglizcha qaytaradi (Monday, Tuesday ...) — bu normal holat.

Nega kalendar shunchalik kerak? `safarlar` jadvalida safar bo'lmagan kun umuman **yo'q** — oddiy `GROUP BY` bunday kunni hech qachon ko'rsata olmaydi. Recursive kalendar + `LEFT JOIN` esa "oyning HAR kuni, jumladan tushumsiz kunlar 0 bilan" hisobotini beradi — real loyihalardagi eng ko'p uchraydigan patternlardan biri (16-masala — albatta ishlang!).

Recursive CTE daraxt strukturalar uchun ham asosiy qurol: kategoriya ichida kategoriya, boshliq–xodim–xodimning xodimi... G'oya o'sha-o'sha: anchor — daraxtning ildizi (masalan, bosh direktor), qoida — "har topilgan xodimning qo'l ostidagilarini qo'sh". Ishda bunga duch kelsangiz, qidiruv so'zingiz tayyor: "recursive CTE hierarchy".

15-bob masalalari

1. 14-bobdagi 15-masalani CTE bilan qayta yozing — qaysi biri o'qilishi oson?
2. (dokon) CTE: har buyurtma summasi → undan eng katta 3 tasini mijoz ismi bilan chiqaring
3. (dokon) CTE: har mijozning jami xaridi → o'rtacha xariddan ko'p xarid qilganlar
4. (kutubxona) CTE: har kitobning ijara soni → "mashhur" (2+ marta olingan) kitoblar muallifi bilan
5. (kutubxona) CTE: har muallifning kitoblari soni → eng sermahsul muallif(lar) — eng katta son bilan teng kelganlarning HAMMASINI chiqaring (bizning bazada bir nechta muallif teng)
6. (kutubxona) 2 ta CTE: o'zbek mualliflar + ularning kitoblari statistikasi
7. (klinika) CTE: har shifokorning daromadi → klinika jami daromadidagi ULUSHI foizda (2-CTE'da jami)
8. (klinika) CTE: har bemorning qatnovlari soni → 1 martadan ko'p kelganlar 'doimiy', qolganlar 'yangi'
9. (taksi) CTE: har haydovchi daromadi → eng yuqori va eng past daromadli haydovchilar farqi
10. (taksi) CTE: kunlik tushum → o'rtacha kunlik tushumdan past kunlar
11. RECURSIVE: 1 dan 100 gacha sonlar
12. RECURSIVE: 5 ning kararlari: 5, 10, 15... 50
13. RECURSIVE: 2 ning darajalari: 2, 4, 8, 16... 1024
14. RECURSIVE: 2026-yil dekabr oyining hamma kunlari + hafta kunlari nomi
15. RECURSIVE: bugundan 10 kun oldingacha sanalar ro'yxati (kun - INTERVAL 1 DAY)
16. RECURSIVE kalendar + LEFT JOIN: iyun oyining HAR KUNI uchun taksi tushumi (safarsiz kunlar 0 bilan!) — bu juda kuchli real pattern
17. (dokon) CTE zanjiri (3 ta): buyurtma summaları → mijoz jami xaridlari → shahar bo'yicha jami

18. (kutubxona) CTE: hozir o'qilayotgan (qaytarilmagan) kitoblar → ularning janr statistikasi
19. (taksi) CTE: har haydovchining o'rtacha bahosi → reytingi (jadvaldagi `reyting` ustuni) bilan solishtiring — farq bormi?
20. O'zingiz: istalgan bazada 2+ CTE'li "hisobot" yozing va har CTE nimani hisoblashini izohda (`-- izoh`) yozing

16 — Window funksiyalar

◀ Oldingi: 15 — CTE — WITH bilan ishlash · 🏠 README · Keyingi: 17 — UPDATE
va DELETE — chuqur ▶

Bu bobda: window (oyna) funksiyalarini o'rganamiz: OVER va PARTITION BY bilan qatorlarni yo'qotmasdan guruh hisob-kitobini qilish, ROW_NUMBER/RANK/DENSE_RANK bilan raqamlash, "har guruhda eng kattasi" klassik masalasi, LAG/LEAD bilan oldingi-keyingi qatorga qarash, yig'ilib boruvchi summa (running total) va LAST_VALUE'dagi mashhur tuzoq.

11-bobda GROUP BY bilan tanishganimizda bir narsaga ko'nikkan edik: guruhlash — bu qatorlarni "yig'ishtirib", har guruhdan bitta qator qoldirish. Lekin amaliyotda tez-tez boshqacha savol tug'iladi: "menga HAMMA qatorlar kerak, lekin har qatorning YONIDA guruh statistikasi ham tursin". Mana shu yerda **window funksiyalar** (oyna funksiyalari) sahnaga chiqadi — zamonaviy SQL'ning eng kuchli qurollaridan biri.

✦ Window funksiyalar MySQL'da **8.0 versiyadan** beri bor. Eski 5.7 da bu sintaksis xato beradi — yana bir sabab yangi versiyada ishlashga.

G'oya: GROUP BY qatorlarni yutadi, WINDOW yutmaydi

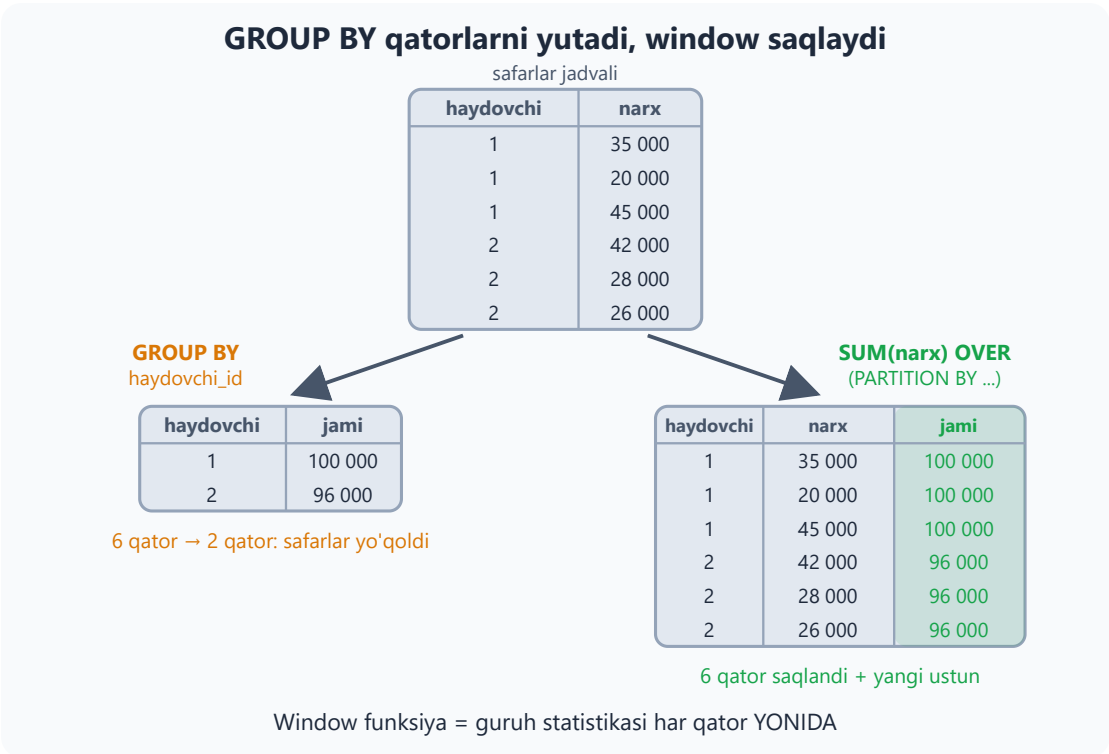
Taksi bazasini eslang: 10 ta safar, 5 ta haydovchi. `GROUP BY haydovchi_id` qilsak, 10 qator 5 qatorga "yiqilib" tushadi — har haydovchidan bittadan, safarlarning o'zi (sana, narx, mijoz) esa natijadan yo'qoladi. Lekin "har safar YONIDA shu haydovchining jami daromadi ham ko'rinsin" desak-chi? Window funksiya aynan shuni qiladi:

```
SELECT
    haydovchi_id, narx, sana,
    SUM(narx) OVER (PARTITION BY haydovchi_id) AS haydovchi_jami
FROM taksi.safarlar;
```

`OVER (PARTITION BY haydovchi_id)` so'zma-so'z: "haydovchi bo'yicha guruhlab hisobla, lekin qatorlarni saqlab qol". Qatorlar soni o'zgarmaydi — shunchaki yangi

ustun qo'shiladi (natijaning bir qismi):

haydovchi_id	narx	haydovchi_jami
1	35000	100000
1	20000	100000
1	45000	100000
2	42000	96000
2	28000	96000



Taqqoslab qo'yaylik:

	GROUP BY	Window (OVER)
Qatorlar soni	kamayadi (har guruhdan 1 ta)	o'zgarmaydi
Asl ustunlar	faqat guruhlanganlari qoladi	hammasi joyida turadi
Natija	guruh hisoboti	har qator + guruh statistikasi

"Window" (oyna) nomi ham shundan: funksiya har bir qator uchun jadvalga go'yo oynadan qaraydi — o'sha qatorga tegishli qatorlar to'plamini (oynasini) ko'radi va hisobni shu to'plam ustida bajaradi. `OVER` qavsi ichidagi ikki kalit so'zni eslab qoling:

- `PARTITION BY` — jadval qanday oynalarga bo'linishini aytadi (yozmasangiz — butun jadval bitta oyna);
- `ORDER BY` — oyna ichidagi tartibni belgilaydi (raqamlash, `LAG/LEAD` va `running total` uchun kerak bo'ladi).

OVER (PARTITION BY haydovchi_id) — oynalar

	haydovchi	narx	
1-oyna	1	35 000	SUM(narx) = 100 000
	1	20 000	
	1	45 000	
2-oyna	2	42 000	SUM(narx) = 96 000
	2	28 000	
	2	26 000	
3-oyna	3	25 000	SUM(narx) = 57 000
	3	32 000	

Har qator faqat O'Z oynasidagi qatorlarni ko'radi —
natija ustunida har qatorga o'z oynasining summasi yoziladi

✦ Bo'sh `OVER ()` ham mutlaqo qonuniy — u holda oyna butun jadval bo'ladi.
"Har mahsulot yonida UMUMIY o'rtacha narx tursin" kabi masalalarda juda qulay:

```
SELECT nomi, narx,  
       AVG(narx) OVER () AS umumiy_ortacha,  
       narx - AVG(narx) OVER () AS farq  
FROM dokon.mahsulotlar;
```

Raqamlash: ROW_NUMBER, RANK, DENSE_RANK

Window funksiyalarning ikkinchi katta oilasi — raqamlovchilar. Eng oddiysi `ROW_NUMBER()` — qatorlarni ko'rsatilgan tartibda 1 dan boshlab raqamlab chiqadi:

```
SELECT nomi, narx,  
       ROW_NUMBER() OVER (ORDER BY narx DESC) AS orin
```

```
FROM dokon.mahsulotlar;
-- Eng qimmat = 1, keyingisi = 2...
```

E'tibor bering: bu yerdagi `ORDER BY` oynaning **ichida** — u raqamlash tartibini belgilaydi, natija qatorlarining ekrandagi tartibini emas (buning uchun so'rov oxiridagi odatdagi `ORDER BY` xizmat qiladi).

Endi qiziq savol: ikkita qiymat TENG bo'lsa-chi? Aynan shu yerda uchala funksiya farq qiladi. Aytaylik, narxlar 100 000, 90 000, 90 000, 80 000:

narx	ROW_NUMBER	RANK	DENSE_RANK
100000	1	1	1
90000	2	2	2
90000	3	2	2
80000	4	4	3

- `ROW_NUMBER` — 1,2,3,4: har doim ketma-ket. Tenglarga ham har xil raqam beradi (qaysi biri 2, qaysi biri 3 bo'lishi — tasodif, chunki tartib noaniq);
- `RANK` — 1,2,2,4: tenglarga teng o'rin, keyin SAKRAYDI. Xuddi sportdagidek: ikki kishi 2-o'rinni bo'lishsa, keyingisi 4-o'rin oladi;
- `DENSE_RANK` — 1,2,2,3: tenglarga teng o'rin, lekin sakramaydi ("zich" rank).

Teng qiymatlarda: ROW_NUMBER, RANK, DENSE_RANK

narx	ROW_NUMBER	RANK	DENSE_RANK
100 000	1	1	1
90 000	2	2	2
90 000	3	2	2
80 000	4	4	3

teng
narx

`ROW_NUMBER`: tenglarga ham har xil raqam beradi (2, 3)

`RANK`: tenglarga teng o'rin, keyin sakraydi (2, 2, keyin 4)

`DENSE_RANK`: teng o'rin, sakramaydi (2, 2, keyin 3)

Tenglik bo'lmasa — uchalasi ham bir xil natija beradi

💡 Yana bir raqamlovchi — `NTILE(n)`: qatorlarni n ta teng guruhga bo'lib, har qatorga guruh raqamini (1 dan n gacha) yozadi. Masalan, `NTILE(4) OVER (ORDER`

BY narx) mahsulotlarni narx bo'yicha 4 ta "chorak"ka ajratadi — masalalarda sinab ko'rasiz.

Klassik masala: har guruhda eng kattasi

"Har kategoriyada eng qimmat mahsulot" — intervyularda ham, real ishda ham eng ko'p uchraydigan savollardan. Bu window'ning yulduzli oni:

```
WITH raqamlangan AS (  
    SELECT *,  
           ROW_NUMBER() OVER (PARTITION BY kategoriya_id ORDER BY narx  
DESC) AS rn  
    FROM dokon.mahsulotlar  
)  
SELECT * FROM raqamlangan WHERE rn = 1;
```

Mantiq ikki qadam: har kategoriya ICHIDA mahsulotlarni narx bo'yicha (qimmatdan arzonga) raqamlaymiz, keyin har guruhning 1-raqamlisini olamiz. Bu `rn = 1` patternini yodlab oling — juda ko'p ishlatasiz.

✳️ "Nega CTE kerak, to'g'ridan-to'g'ri `WHERE rn = 1` deb yozsak bo'lmaydimi?" — Yo'q. Window funksiyalar `WHERE` bosqichidan KEYIN hisoblanadi, shuning uchun `WHERE` ichida ularga murojaat qilib bo'lmaydi (MySQL "Unknown column 'rn'" deb xato beradi). Avval CTE (yoki subquery) ichida hisoblab olamiz, keyin tashqarida filtrlaymiz.

💡 Eng qimmat mahsulot ikkita bo'lsa-yu, IKKALASI ham kerak bo'lsa — `ROW_NUMBER` o'rniga `RANK` ishlateng: tenglarning hammasi `rn = 1` bo'lib chiqadi.

LAG va LEAD — oldingi/keyingi qatorga qarash

Ba'zan har qator o'z "qo'shnisi" bilan solishtirilishi kerak: bugungi tushum kechagidan qancha farq qiladi? `LAG` — oldingi qatordagi, `LEAD` — keyingi qatordagi qiymatni olib keladi:

```
-- Har kun tushumi + kechagi bilan farqi:  
WITH kunlik AS (  
    SELECT DATE(sana) AS kun, SUM(narx) AS tushum  
    FROM taksi.safarlar  
    GROUP BY kun
```

```

)
SELECT kun, tushum,
       LAG(tushum) OVER (ORDER BY kun) AS kechagi,
       tushum - LAG(tushum) OVER (ORDER BY kun) AS farq
FROM kunlik;

```

kun	tushum	kechagi	farq
2026-06-01	77000	NULL	NULL
2026-06-02	20000	77000	-57000
2026-06-03	25000	20000	5000
2026-06-04	50000	25000	25000

LAG — orqaga, LEAD — oldinga qarash



Joriy qator — 06-03 (ko'k ramka): LAG kechagi 20 000 ni,
LEAD ertangi 50 000 ni olib keladi. Chetda qo'shni bo'lmasa — NULL.

✦ Birinchi qatorning "kechasi" yo'q — shuning uchun LAG u yerda NULL qaytaradi (oxirgi qatorda LEAD ham xuddi shunday). To'liq shakli `LAG(ustun, qadam, standart)`: `LAG(tushum, 1, 0)` — "1 qadam orqaga qara, topolmasang 0 de". `LAG(tushum, 7)` desangiz — bir hafta oldingi kun bilan solishtirasiz.

💡 Bitta oynani ikki marta yozish zerikarli bo'lsa, unga nom bering — `WINDOW` bandi (MySQL 8 da bor). Yuqoridagi misolning SELECT qismi shunday qisqaradi (boshidagi `WITH kunlik AS (...)` qismi o'z joyida qoladi — usiz `kunlik` jadvali topilmaydi): `SELECT kun, tushum, LAG(tushum) OVER w AS kechagi, tushum - LAG(tushum) OVER w AS farq FROM kunlik WINDOW w AS (ORDER BY kun);`

Yig'ilib boruvchi summa (running total)

Agregat oynasiga `ORDER BY` qo'shsangiz, hisob "yig'ilib boruvchi" rejimga o'tadi — har qatorda boshidan shu qatorgacha bo'lgan jami chiqadi:

```
SELECT id, sana, narx,  
       SUM(narx) OVER (ORDER BY sana, id) AS yigilgan  
FROM taksi.safarlar;  
-- Har qatorda: shu paytgacha jami qancha tushum bo'lgan
```

sana	narx	yigilgan
2026-06-01 08:30	35000	35000
2026-06-01 09:15	42000	77000
2026-06-02 14:00	20000	97000
2026-06-03 18:45	25000	122000

Bu — bank ilovangizdagi "balans tarixi"ning aynan o'zi: har amaliyot yonida shu paytgacha yig'ilgan summa. `PARTITION BY` bilan birlashtirsangiz, har guruh o'z ichida noldan boshlab yig'iladi: `SUM(narx) OVER (PARTITION BY haydovchi_id ORDER BY sana)` — har haydovchi daromadi qanday o'sib borgani.

💡 `ORDER BY` da `sana` dan keyin `id` ham turibdi — ikki safar aynan bir vaqtda bo'lsa ham tartib bir ma'noli bo'lishi uchun. Tartib noaniq qolsa, teng qiymatli qatorlarda yig'ilgan summa "g'alati" ko'rinishi mumkin.

FIRST_VALUE, LAST_VALUE va bitta tuzoq

Oynadagi birinchi va oxirgi qiymatni olish uchun `FIRST_VALUE` va `LAST_VALUE` funksiyalari bor. Lekin `LAST_VALUE` da deyarli har bir boshlovchi yiqiladigan tuzoq yashiringan:

```
SELECT mijoz_id, sana,  
       FIRST_VALUE(sana) OVER (PARTITION BY mijoz_id ORDER BY sana) AS  
birinchi,  
-- Kutilmagan natija: bu "oxirgi" emas, JORIY qatorning o'zini
```

```
qaytaradi!  
    LAST_VALUE(sana) OVER (PARTITION BY mijoz_id ORDER BY sana) AS  
    oxirgi_emas  
FROM dokon.buyurtmalar;
```

Sababi: `OVER` ichida `ORDER BY` bo'lsa, oyna sukut bo'yicha "boshidan JORIY qatorgacha" deb qisqartiriladi (bu chegara **frame** deyiladi). `FIRST_VALUE` ga bu farq qilmaydi — boshi baribir boshida. `LAST_VALUE` uchun esa "oynaning oxiri" — joriy qatorning o'zi bo'lib qoladi. (Aniqroq aytsak: `ORDER BY` ustunida joriy qatorga TENG qiymatli qatorlar bo'lsa, frame ularni ham qamrab oladi va `LAST_VALUE` o'sha tengdoshlarning oxirgisini qaytaradi; bizning misolda sanalar takrorlanmagani uchun bu — joriy qatorning o'zi.) Davosi — frame'ni oxirigacha ochib qo'yish:

```
LAST_VALUE(sana) OVER (  
    PARTITION BY mijoz_id ORDER BY sana  
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING  
) AS oxirgi
```

💡 Amalda ko'pchilik soddaroq yo'lni tanlaydi: `MIN(sana) OVER (PARTITION BY mijoz_id)` va `MAX(sana) OVER (PARTITION BY mijoz_id)` — frame'siz, tuzoqsiz, o'sha natija. 20-masalada ikkala usulni ham sinaysiz.

16-bob masalalari

1. (dokon) Mahsulotlarni narx bo'yicha raqamlang (`ROW_NUMBER`)
2. (dokon) `RANK` va `DENSE_RANK` bilan ham qiling — bir xil narxli mahsulot qo'shib, farqni ko'ring
3. (dokon) Har kategoriya ichida mahsulotlarni narx bo'yicha raqamlang (`PARTITION BY`)
4. (dokon) Har kategoriyaning eng qimmat mahsuloti (`rn = 1` patterni)
5. (dokon) Har kategoriyaning eng ARZON mahsuloti
6. (dokon) Har mahsulot yonida o'z kategoriyasining o'rtacha narxi (`AVG OVER PARTITION`)
7. (dokon) Har mahsulot o'z kategoriyasi o'rtachasidan necha foiz qimmat/arzon
8. (kutubxona) Har janr ichida kitoblarni yil bo'yicha raqamlang

9. (kutubxona) Har janrning eng eski kitobi ($rn = 1$)
10. (kutubxona) Har kitob yonida o'sha muallifning jami kitoblari soni (COUNT OVER)
11. (kutubxona) Kitoblarni sahifa bo'yicha 3 guruhga bo'ling: NTILE(3) OVER (ORDER BY sahifa) — natijaga qarab qaysi kitob qaysi guruhga tushganini tushuntiring
12. (klinika) Har shifokor ichida qabullarni sana bo'yicha raqamlang — har shifokorning BIRINCHI qabuli qachon?
13. (klinika) Har bemorning OXIRGI tashrifi (PARTITION BY bemor_id ORDER BY sana DESC, $rn = 1$)
14. (klinika) Har qabul yonida: shu bemorning shu paytgacha jami to'lovi (PARTITION bilan running total)
15. (taksi) Har haydovchining eng qimmat safari
16. (taksi) Kunlik tushum + kechagi kun + farq (LAG misoli asosida)
17. (taksi) Kunlik tushumning yig'ilib boruvchi summasi (running total)
18. (taksi) Har safar yonida o'sha haydovchining o'rtacha safar narxi va shu safar undan qancha farq qilishi
19. (taksi) LEAD bilan: har safar yonida o'sha haydovchining KEYINGI safari sanasi — safarlar orasidagi tanaffusni hisoblang (TIMESTAMPDIFF(HOUR, ...))
20. (dokon) Murakkab: har mijozning birinchi va oxirgi buyurtmasi sanasi bitta qatorda (FIRST_VALUE va LAST_VALUE yoki MIN/MAX OVER — ikkala usulni sinang, LAST_VALUE'da frame tuzog'ini unutmang!)

17 — UPDATE va DELETE — chuqur

← Oldingi: 16 — Window funksiyalar · 🏠 README · Keyingi: 18 — ALTER, Constraint, Foreign Key →

Bu bobda: mavjud ma'lumotni o'zgartirish (UPDATE) va o'chirish (DELETE)ni chuqur o'rganamiz: WHERE unutilsa qanday falokat bo'lishini, "avval SELECT" xavfsiz protokolini, SQL_SAFE_UPDATES kamarini, NULL bilan bog'liq tuzoqni, soft delete (yumshoq o'chirish) degan professional yondashuvni va DELETE/TRUNCATE/DROP uchligining farqini ko'rib chiqamiz.

Shu paytgacha biz ma'lumotni asosan **o'qidik** (SELECT) va **qo'shdik** (INSERT). Endi qo'limizga "tuzatish qalami va o'chirg'ich" beriladi. Bu — kitobdagi eng ehtiyotkorlik talab qiladigan bob: SELECT'da xato qilsangiz, eng yomoni — noto'g'ri natija ko'rasiz, so'rovni qayta yozasiz, vassalom. UPDATE yoki DELETE'da xato qilsangiz — **ma'lumotning o'zi buziladi**, va oddiy "Ctrl+Z" yo'q (yagona qutqaruvchi — tranzaksiyalar, ularni 19-bobda o'rganamiz). Shuning uchun bu bobda buyruqlarning sintaksisidan ham ko'proq **xavfsiz ishlash odatiga** urg'u beramiz.

UPDATE — o'zgartirish

```
UPDATE jadval SET ustun = qiymat WHERE shart;
```

Uch qism — uch savol:

- `UPDATE jadval` — **qaysi jadvalda** ishlaymiz;
- `SET ustun = qiymat` — **nimani qanday** o'zgartiramiz (vergul bilan bir nechta ustunni birdan);
- `WHERE shart` — **qaysi qatorlarda**. Eng muhim qism — shu!

UPDATE anatomiyasi: WHERE — xavfsizlik kamaringiz

UPDATE mahsulotlar

qaysi jadvalda

SET narx = 2600000

nima qanday o'zgaradi

WHERE id = 3

qaysi qatorlarda

WHERE id = 3 bilan

id=1 · Samsung · o'zgarmadi

id=2 · iPhone · o'zgarmadi

id=3 · Redmi · narx: 2 600 000 ✓

id=4 · Lenovo · o'zgarmadi

Faqat kerakli qator o'zgardi

WHERE'siz (unutdingiz!)

id=1 · Samsung · narx: 0 X

id=2 · iPhone · narx: 0 X

id=3 · Redmi · narx: 0 X

id=4 · Lenovo · narx: 0 X

HAMMA qator o'zgarib ketdi!

Qoida: UPDATE/DELETE yozdingizmi — avval WHERE bormi, to'g'rimi deb tekshiring!

USE dokon;

-- Bitta mahsulot narxini o'zgartirish:

```
UPDATE mahsulotlar SET narx = 2600000 WHERE id = 3;
```

-- Bir nechta ustun birdan (vergul bilan):

```
UPDATE mahsulotlar SET narx = 2500000, soni = 30 WHERE id = 3;
```

-- Hisob bilan (yangi qiymat eski qiymatga asoslanadi):

```
UPDATE mahsulotlar SET narx = narx * 1.10 WHERE kategoriya_id = 1; --  
telefonlar 10% qimmatladi
```

-- Shartli guruh: 5-iyungacha 'yolda' bo'lganlar yetib bordi deylik

```
UPDATE buyurtmalar SET holat = 'yetkazildi' WHERE holat = 'yo'lda' AND  
sana < '2026-06-05';
```

`narx = narx * 1.10` qatoriga e'tibor bering: tenglikning o'ng tomonida ustunning **eski** qiymati turibdi. MySQL har bir mos qator uchun eski narxni oladi, 1.10 ga ko'paytiradi va natijani o'sha qatorga qaytarib yozadi. Shu usul bilan "hammaga 5% chegirma", "ombordagi sonni 1 taga kamaytir" kabi amallar bitta buyruqda bajariladi.

✦ MySQL'ning javobini o'qishni odat qiling:

Query OK, 3 rows affected

Rows matched: 3 Changed: 3 Warnings: 0

Rows matched — WHERE nechta qatorni topdi, Changed — nechta haqiqatan o'zgardi (yangi qiymat eskisiga teng bo'lsa, qator "matched" hisoblanadi, lekin "changed" emas). Bu sonlar — bepul nazoratchi: 3 qator kutgan edingiz, 300 chiqdi? Demak, WHERE'da nimadir noto'g'ri — to'xtab tekshiring.

⚠️ HAYOTIY QOIDA №1: UPDATE/DELETE'da WHERE'ni UNUTMANG!

```
UPDATE mahsulotlar SET narx = 0; -- ❌❌❌ HAMMA mahsulot 0 so'm bo'ldi!
```

WHERE'siz UPDATE — jadvaldagi **har bir** qatorga tegishli buyruq. MySQL e'tiroz bildirmaydi, "ishonchingiz komilmi?" deb so'ramaydi — jimgina hammasini o'zgartiradi. Bu xatoni har dasturchi hayotida bir marta qiladi (ko'pchilik — ish serverida). Sizning himoyangiz — **odat**:

1. Avval SELECT yozing: `SELECT * FROM mahsulotlar WHERE id = 3;`
2. Natija aynan siz kutgan qatorlar ekanini ko'ring
3. `SELECT *` qismini `UPDATE ... SET ...` ga almashtiring — **WHERE o'sha-o'sha qoladi**

Xavfsiz protokol: avval SELECT, keyin UPDATE

1-qadam: SELECT

```
SELECT *  
FROM mahsulotlar  
WHERE id = 3;
```

hali hech narsa o'zgarmadi

2-qadam: tekshiring

Kerakli qatorlarmi?
Soni to'g'rimi?

Noto'g'ri bo'lsa —
WHERE'ni tuzating

3-qadam: UPDATE

```
UPDATE mahsulotlar  
SET narx = 2600000  
WHERE id = 3;
```

WHERE o'sha-o'sha!

WHERE sharti SELECT'dan UPDATE'ga o'zgarishsiz ko'chadi

Odatga qo'shimcha ikkita "xavfsizlik kamari" ham bor:

```
SET SQL_SAFE_UPDATES = 1;
```

Bu rejim yoqilganda MySQL kalit ustun (masalan, `id`) ishtirok etmagan WHERE'li yoki umuman WHERE'siz UPDATE/DELETE'ni xato bilan to'xtatadi (Error 1175) — ikkala holatda ham faqat buyruqda LIMIT bo'lmasa: LIMIT'li buyruq (masalan, `DELETE FROM loglar LIMIT 5;`) bu rejimda ham o'taveradi, chunki LIMIT'ning o'zi cheklov hisoblanadi. MySQL Workbench'da bu rejim standart yoqilgan — endi nima uchunligini bilasiz. Ba'zan u halol so'rovingizni ham to'xtatadi (WHERE'da kalit ustun bo'lmasa), lekin o'rganish davrida yoqib qo'ygan ma'qul. O'chirish: `SET SQL_SAFE_UPDATES = 0;`

Ikkinchi kamar — **tranzaksiyalar**: `START TRANSACTION` bilan boshlasangiz, xato UPDATE'dan keyin `ROLLBACK` bilan hammasini orqaga qaytara olasiz. Bu alohida katta mavzu — 19-bobda batafsil.

NULL tuzog'i: `= NULL` ishlamaydi

8-bobdan eslang: NULL bilan oddiy `=` taqqoslab bo'lmaydi. Bu qoida UPDATE/DELETE'ning WHERE'ida ham xuddi shunday ishlaydi:

```
USE klinika; -- bemorlar jadvali klinika bazasida

UPDATE bemorlar SET telefon = '+998900000000' WHERE telefon = NULL; -
- ❌ 0 qator!
UPDATE bemorlar SET telefon = '+998900000000' WHERE telefon IS NULL; -
- ✅ to'g'ri
```

Eng yomoni — birinchi variant **xato bermaydi**: shunchaki `Rows matched: 0` deb jim qo'ya qoladi, siz esa "yangiladim" deb yuraverasiz. SET qismida esa NULL'ni bemalol yozish mumkin: `SET baho = NULL` qiymatni "bo'shatib" qo'yadi.

DELETE — o'chirish

```
DELETE FROM jadval WHERE shart;
```

DELETE'da SET yo'q — qator butunlay yo'qoladi, shuning uchun "nimani?" deb so'ralmaydi, faqat "qaysi qatorlarni?" (WHERE). Xavfsiz protokol esa aynan o'sha: avval SELECT, keyin DELETE.

```
-- Avval ko'ramiz, nima o'chadi:  
SELECT * FROM buyurtmalar WHERE holat = 'bekor' AND sana < '2026-01-01';  
  
-- Keyin o'chiramiz:  
DELETE FROM buyurtmalar WHERE holat = 'bekor' AND sana < '2026-01-01';
```

Bizning bazada 2026 yilgacha bekor qilingan buyurtma yo'q, shuning uchun bu buyruq `0 rows affected` qaytaradi — va bu normal holat: DELETE qator topolmasa, xato bermaydi, shunchaki hech narsani o'chirmaydi.

✳ Bog'langan jadvallarda **tartib** muhim: buyurtmani o'chirsangiz, uning `buyurtma_qatorlari` "egasiz" qolib ketadi. To'g'ri tartib — avval bola qatorlarni (`buyurtma_qatorlari`), keyin otasini (`buyurtmalar`) o'chirish. 18-bobda FOREIGN KEY bilan bazaning o'zi shu tartibni qattiq nazorat qilishini ko'ramiz.

ORDER BY va LIMIT — MySQL'ning qo'shimcha imkoniyati

MySQL'da UPDATE/DELETE'ga ORDER BY va LIMIT qo'shish mumkin (bu standart SQL emas, MySQL'ning o'z qo'shimchasi):

```
-- Eng eski 100 ta yozuvni o'chirish (masalan, log tozalash):  
DELETE FROM loglar ORDER BY sana ASC LIMIT 100;  
  
-- Deylik, sinov jadvalida bitta yozuv adashib IKKI marta kiritilgan —  
-- LIMIT 1 bilan dublikatning faqat bittasini o'chiramiz:  
DELETE FROM sinov_mahsulotlar WHERE nomi = 'Telefon chexol' LIMIT 1;
```

LIMIT bu yerda ham kamar vazifasini bajaradi: WHERE kutilmaganda ko'p qatorni topib qolsa ham, buyruq ko'pi bilan LIMIT'da ko'rsatilgan sondagi qatorga ta'sir qiladi.

Soft delete — professional yondashuv

Real tizimlarda ma'lumot kamdan-kam **chindan** o'chiriladi. O'rniga "o'chirilgan" deb belgilanadi:

```
-- ALTER TABLE (jadvalga yangi ustun qo'shish) bilan 18-bobda batafsil tanishamiz --
-- hozircha shunchaki nusxalab bajaring:
ALTER TABLE mijozlar ADD COLUMN ochirilgan_sana DATETIME NULL;

-- "O'chirish":
UPDATE mijozlar SET ochirilgan_sana = NOW() WHERE id = 6;

-- Faqat "tirik"larni ko'rsatish:
SELECT * FROM mijozlar WHERE ochirilgan_sana IS NULL;

-- "Tiklash":
UPDATE mijozlar SET ochirilgan_sana = NULL WHERE id = 6;
```

Nega? Mijoz "akkauntimni o'chirdim" deydi, bir hafta keyin "tiklang!" deydi. Buyurtma tarixi, hisobotlar — hammasi o'sha mijozga bog'liq edi. Hard delete = tarixni yo'qotish. Telegram'ning "akkauntga bir necha oy (standart sozlamada — 6 oy) kirmasangiz u o'chadi" siyosatini eslang — bu ham soft delete: avval belgi qo'yiladi, chindan o'chirish keyinroq (yoki umuman bo'lmaydi).

Yana bir nozik joy: nega ustun `ochirilgan TINYINT(0/1)` bayroq emas, `ochirilgan_sana DATETIME`? Chunki bitta ustun ikki savolga javob beradi: o'chirilganmi (`IS NULL` / `IS NOT NULL`) va **qachon** o'chirilgan. Keyinchalik hisobotda asqotadi: "o'tgan oyda nechta mijoz ketdi?"

DELETE vs TRUNCATE vs DROP

<code>DELETE FROM jadval;</code>	-- qatorlarni o'chiradi (sekinroq), jadval qoladi, id davom etadi	
<code>TRUNCATE TABLE jadval;</code>	-- hammasini tez tozalaydi, id yana 1 dan boshlanadi	
<code>DROP TABLE jadval;</code>	-- jadvalning O'ZI yo'qoladi	

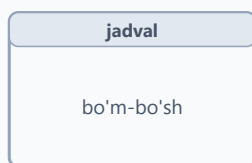
DELETE vs TRUNCATE vs DROP — nima yo'qoladi?

DELETE FROM jadval



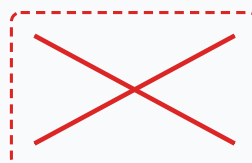
- WHERE bilan tanlaydi
- jadval qoladi
- id davom etadi
- ROLLBACK mumkin (19-bob)
- sekinroq (qatorma-qator)

TRUNCATE TABLE jadval



- HAMMA qator o'chadi
- jadval qoladi (bo'sh)
- id yana 1 dan boshlanadi
- ROLLBACK ishlamaydi
- juda tez

DROP TABLE jadval



jadvalning o'zi yo'q

- struktura ham, qator ham
- hech narsa qolmaydi
- qaytarib bo'lmaydi!
- juda tez

Tanlab o'chirish faqat DELETE'da — qolgan ikkisi "hammasi birdan" ishlaydi

Savol	DELETE	TRUNCATE	DROP
Nima o'chadi?	qatorlar	HAMMA qator	jadvalning o'zi
WHERE bilan tanlash	✓ bor	✗ yo'q	✗ yo'q
Jadval qoladimi?	✓ qoladi	✓ qoladi (bo'sh)	✗ yo'qoladi
AUTO_INCREMENT	davom etadi	1 dan boshlanadi	—
Tezligi	sekinroq (qatorma-qator)	juda tez	juda tez
ROLLBACK bo'ladimi? (19-bob)	✓ ha	✗ yo'q	✗ yo'q

Nega TRUNCATE bunchalik tez? Chunki u qatorlarni bitta-bitta o'chirib o'tirmaydi — jadvalni butunlay buzib, bo'sh holda qayta quradi. Aynan shu sababdan uni tranzaksiya ichida orqaga qaytarib bo'lmaydi, FOREIGN KEY bilan boshqa jadvaldan bog'lanib turgan jadvalga esa TRUNCATE umuman ishlamaydi (18-bobda ko'rasiz). Qoida sodda: kundalik ishda tanlab o'chirish — DELETE, sinov jadvalini "nol"dan boshlash — TRUNCATE, jadval umuman kerak bo'lmay qolsa — DROP.

17-bob masalalari

Har UPDATE/DELETE oldidan SELECT bilan tekshirish odatini shu yerda shakllantiring!

1. (dokon) iPhone 15 narxini 11 000 000 ga tushiring (avval SELECT!)
2. (dokon) Telefonlar kategoriyasidagi hamma narxni 5% oshiring (avval kategoriya_id'ni aniqlab oling)
3. (dokon) id=6 mahsulot (Acer) omborga keldi: soni'ni 10 ga o'rnating
4. (dokon) 'yolda'dagi hamma buyurtmani 'yetkazildi'ga o'tkazing
5. (dokon) Buyurtma #7 sotib olindi deylik: undagi mahsulotlar sonini ombordan ayiring (qo'lda, mahsulot id'sini topib: `soni = soni - 1`)
6. (kutubxona) Aziz Karimov 'Sariq devni minib'ni qaytardi: tegishli ijaraga bugungi sanani yozing (qaysi qator ekanini SELECT bilan toping!)
7. (kutubxona) 'Kichkina shahzoda'dan 2 nusxa yangi keldi: nusxa_soni'ni 2 taga oshiring (`nusxa_soni = nusxa_soni + 2`)
8. (kutubxona) Nilufar Rahimova telefon qo'ydi: '+998904445577' yozing
9. (kutubxona) Hamma she'riyat kitoblarining nusxa_soni'ni 1 taga oshiring (bazada janr 'sheriyat' deb yozilgan)
10. (klinika) Dr. Hakimov tajribasi 6 yil bo'ldi: yangilang
11. (klinika) Hamma shifokor qabul narxini 10 000 ga oshiring
12. (klinika) Telefoni NULL bemorlarga '+998900000000' vaqtinchalik raqam yozing (esingizdami: `IS NULL`!)
13. (taksi) Murod Aliyev reytingini qayta hisoblang: safarlardagi o'rtacha bahosini qo'lda hisoblab (AVG bilan SELECT), reyting ustuniga yozing
14. (taksi) Baho NULL safarlarga 5 qo'ying ("indamagan — rozi" siyosati :)
15. (dokon) Soft delete: mijozlarga `ochirilgan_sana` ustuni qo'shing (yuqoridagi `ALTER TABLE` misolidan nusxa oling — bu buyruq 18-bobda batafsil o'rgatiladi), bitta mijozni soft-delete qiling, "tiriklar" ro'yxatini chiqaring, keyin tiklang
16. (sinov bazasida!) Jadval yarating, 5 qator kiriting, WHERE'siz UPDATE qiling — oqibatni o'z ko'zingiz bilan ko'ring. Keyin TRUNCATE qilib tozalang
17. (sinov) DELETE bilan hammasini o'chiring, yangi qator kiriting — id nechadan boshlandi? TRUNCATE'dan keyin-chi? Solishtiring
18. (dokon) 'bekor' holatidagi buyurtmalarning qatorlarini (buyurtma_qatorlari) o'chiring — subquery bilan: `WHERE buyurtma_id IN (SELECT id FROM buyurtmalar WHERE holat='bekor')`

19. (taksi) 3 dan past baholi safarlarni o'chirishdan OLDIN ularni alohida jadvalga ko'chiring: `CREATE TABLE safarlar_arxiv AS SELECT * FROM safarlar WHERE baho < 3;` keyin DELETE — "arxivlash" patterni!
20. O'ylang va yozing: qaysi jadvallarda soft delete SHART (mijozlar? to'lovlar?), qaysilarida hard delete normal (loglar?)

18 — ALTER, Constraint, Foreign Key

◀ Oldingi: 17 — UPDATE va DELETE — chuqur · 🏠 README · Keyingi: 19 — Tranzaksiyalar ➡

Bu bobda: tayyor jadvalni buzmasdan o'zgartirishni (ALTER TABLE: ustun qo'shish, ta'rifini o'zgartirish, o'chirish), bazaning o'zi qo'riqlaydigan "qoidalar" — constraint'larni (NOT NULL, UNIQUE, DEFAULT, CHECK) va jadvallar orasidagi bog'lanish kafolati bo'lgan FOREIGN KEY'ni o'rganamiz; ota qator o'chirilganda nima bo'lishini ON DELETE (RESTRICT, CASCADE, SET NULL) bilan boshqarishni ham ko'ramiz.

Real loyihada jadval bir marta yaratilib, abadiy shu holicha qolmaydi: bugun mijozga email ustuni kerak bo'lib qoladi, ertaga telefon majburiy bo'lishi kerak, indinga bir ustun umuman ortiqcha chiqadi. Yaxshi yangilik — buning uchun jadvalni DROP qilib, ichidagi ma'lumot bilan birga yo'qotib, qaytadan qurish shart emas. Uyni buzib tashlamasdan ta'mirlaganday, jadvalni ham ishlab turgan holida o'zgartirish mumkin. Buning asbobi — **ALTER TABLE**.

ALTER — tayyor jadvalni o'zgartirish

```
ALTER TABLE mijozlar ADD COLUMN email VARCHAR(150) NULL; -- ☐
ustun qo'shish
ALTER TABLE mijozlar ADD COLUMN yosh INT AFTER ism; --
ma'lum joyga qo'shish
ALTER TABLE mijozlar MODIFY COLUMN telefon VARCHAR(25) NOT NULL; --
ta'rifini o'zgartirish
ALTER TABLE mijozlar RENAME COLUMN yosh TO yoshi; --
ustun nomini o'zgartirish
ALTER TABLE mijozlar DROP COLUMN yoshi; --
ustunni o'chirish
ALTER TABLE mijozlar RENAME TO xaridorlar; --
jadval nomini o'zgartirish
```

⚠ **Bu misollar** `dokon.mijozlar` jadvalini haqiqatan o'zgartiradi — ayniqsa oxirgi `RENAME TO xaridorlar` jadval nomini o'zgartirib yuboradi, shundan keyin `mijozlar` ga tayanadigan masalalar (shu bobdagi 11-masala ham!) va keyingi boblar ishlamay qoladi. Eng yaxshisi — ularni `sinov` bazasidagi alohida jadvalda

mashq qiling. `dokon` da sinab ko'rgan bo'lsangiz, oxirida hammasini asl holiga qaytarib qo'ying:

```
ALTER TABLE xaridorlar RENAME TO mijozlar;      -- nomni qaytarish
ALTER TABLE mijozlar DROP COLUMN email;         -- qo'shilgan ustunni olib tashlash
ALTER TABLE mijozlar MODIFY COLUMN telefon VARCHAR(20); -- asl ta'rifi qaytarish
```

✦ **MODIFY ustunning butun ta'rifini qaytadan yozadi.** Bu eng ko'p qoqiladigan joy: `telefon` avval `VARCHAR(20) NOT NULL` bo'lsa-yu, siz `MODIFY COLUMN telefon VARCHAR(25)` deb yozsangiz (`NOT NULL`'ni unutib), ustun endi `NULL`'ga ruxsat beradigan bo'lib qoladi. Shuning uchun `MODIFY` ishlatganda saqlab qolmoqchi bo'lgan hamma xususiyatni qayta sanab o'ting.

⚠ Ehtiyot bo'ladigan ikki holat: `DROP COLUMN` ustun bilan birga undagi **barcha ma'lumotni** qaytarib bo'lmas tarzda o'chiradi; turini "toraytirish" esa (masalan, `VARCHAR(150) → VARCHAR(20)`) ichida uzunroq qiymat bor bo'lsa xato beradi. Va yana: millionlab qatorli jadvalda `ALTER` bir necha soniya, hatto daqiqa olishi mumkin — bu normal, MySQL ba'zi o'zgarishlarda jadvalni qayta quradi.

O'zgartirgandan keyin natijani tekshirib qo'yish odat bo'lsin:

```
DESCRIBE mijozlar;      -- ustunlar ro'yxati: turi, NULL/NOT NULL, default
SHOW CREATE TABLE mijozlar; -- jadvalning to'liq ta'rifi (constraint'lar bilan)
```

Constraint — bazaning "qoidalari"

Constraint (cheklov) = "bu jadvalga noto'g'ri ma'lumot KIRMAYDI" kafolati. Buni binoning kirishidagi qo'riqchi deb tasavvur qiling: ruxsatnomasi yo'q odamni ichkariga qo'ymaydi. Tekshiruvni dasturga emas, bazaning o'ziga topshirishning katta afzalligi bor: dastur kodida tekshirishni unutib qo'yish mumkin (yoki bazaga uchta turli dastur ulanadi-yu, bittasida tekshiruv yo'q), baza esa hech qachon "unutmaydi" — qoida hamma uchun, doim ishlaydi:

Constraint	Vazifasi
NOT NULL	bo'sh bo'lmasin
UNIQUE	takrorlanmasin
PRIMARY KEY	NOT NULL + UNIQUE + asosiy identifikator
DEFAULT	berilmasa, shu qiymat
CHECK	shart bajarilsin (MySQL 8.0.16+)
FOREIGN KEY	boshqa jadvaldagi mavjud qiymatga ishora qilsin

Constraint — jadvalning "qoidalari"

NOT NULL

Bo'sh qiymat (NULL) kiritib bo'lmaydi

UNIQUE

Qiymat takrorlanmaydi (bir nechta NULL mumkin)

DEFAULT

Qiymat berilmasa, shu qiymat yoziladi

CHECK

Shart bajarilishi kerak
masalan: `narx > 0`
(MySQL 8.0.16+)

PRIMARY KEY

NOT NULL + UNIQUE
qatorning asosiy
identifikatori

FOREIGN KEY

Boshqa jadvaldagi
mavjud qiymatga
ishora qilishi shart

Dastur tekshirishni unutsa ham, baza unutmaydi — noto'g'ri ma'lumot KIRMAydi

```
ALTER TABLE klinika.shifokorlar ADD CONSTRAINT chk_tajriba CHECK
(tajriba_yil >= 0);
-- Endi tajriba_yil = -5 kiritib BO'LMAYDI

ALTER TABLE klinika.bemorlar ADD CONSTRAINT uq_telefon UNIQUE
(telefon);
-- Endi bitta telefon raqami ikki marta kirmaydi
```

✦ **Constraint'ga nom berish odati:** `chk_` (CHECK), `uq_` (UNIQUE), `fk_` (FOREIGN KEY) prefikslari bilan. Nom bersangiz, qoida buzilganda xato xabarida aynan qaysi constraint "ushlab qolgani" ko'rinadi, keyinchalik o'chirish ham oson bo'ladi:

```
ALTER TABLE klinika.shifokorlar DROP CHECK chk_tajriba; -- CHECK'ni o'chirish
ALTER TABLE klinika.bemorlar DROP INDEX uq_telefon; -- UNIQUE aslida indeks, shunday o'chiriladi
```

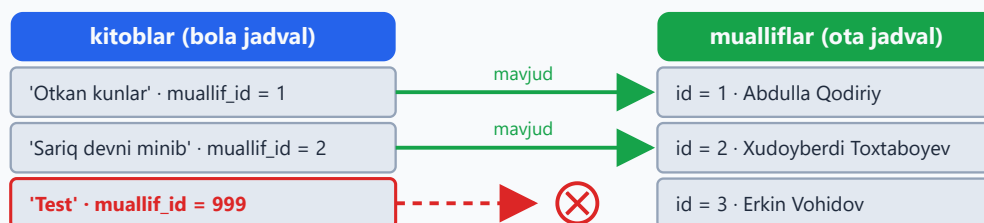
⚠ Tarixiy tuzoq: MySQL 8.0.16 dan oldingi versiyalar (jumladan, eski 5.7) CHECK'ni xatosiz **qabul qilardi, lekin tekshirmasdi** — qoida bordek ko'rinib, aslida ishlamasdi. MySQL 8 ning yangi versiyalarida bu tuzatilgan: CHECK haqiqatan qo'riqlaydi. Versiyangizni `SELECT VERSION();` bilan tekshirib oling.

Yana bir muhim nuans: **CHECK NULL'ni tekshirmaydi**. `CHECK (baho BETWEEN 1 AND 5)` qoidasi bor jadvalga `baho = NULL` bemalol kiradi, chunki NULL bilan solishtirish natijasi "noma'lum" — rad etish uchun yetarli emas. NULL ham kirmasin desangiz, ustunga alohida NOT NULL qo'ying.

FOREIGN KEY — bog'lanish kafolati

Hozir `kitoblar.muallif_id` ga 999 yozsangiz — MySQL indamaydi, garchi 999-muallif yo'q bo'lsa ham. Bu "yetim qator" (orphan row) — hisobotlarda yo'qolib qoladigan, JOIN'da chiqmay qoladigan, xatolar manbai bo'lgan ma'lumot. FOREIGN KEY shundan himoya qiladi: "bola" jadvaldagi ishora "ota" jadvalda haqiqatan mavjudligini har safar tekshiradi:

FOREIGN KEY: bola jadval otaga ishora qiladi



999 ota jadvalda YO'Q — MySQL bu qatorni RAD ETADI

Xato: Cannot add or update a child row: a foreign key constraint fails

FK kafolati: boladagi har bir ishora otada mavjud bo'lishi SHART

```
USE kutubxona;

ALTER TABLE kitoblar
ADD CONSTRAINT fk_kitob_muallif
FOREIGN KEY (muallif_id) REFERENCES mualliflar(id);
```

```
-- Endi:
INSERT INTO kitoblar (nomi, muallif_id) VALUES ('Test', 999);
-- ❌ Xato: Cannot add or update a child row – 999-muallif yo'q!

DELETE FROM mualliflar WHERE id = 1;
-- ❌ Xato: kitoblari bor muallifni o'chirib bo'lmaydi!
```

E'tibor bering: FK himoyasi **ikki tomonlama** ishlaydi. Bola jadvalga mavjud bo'lmagan ishorani kiritib bo'lmaydi, ota jadvaldan esa "bolalari" bor qatorni (default holatda) o'chirib bo'lmaydi.

FK qo'shishda ikkita texnik talab bor:

- **Turlar mos bo'lsin:** `muallif_id` INT bo'lsa, `mualliflar.id` ham INT bo'lishi kerak (ikkalasi ham signed yoki ikkalasi ham UNSIGNED).
- **Mavjud ma'lumot toza bo'lsin:** jadvalda allaqachon yetim qatorlar bo'lsa (masalan, `muallif_id = 999`), MySQL FK qo'shishga ruxsat bermaydi — avval ularni tuzating (UPDATE bilan to'g'rilang yoki NULL qiling).

✦ MySQL FK ustuniga avtomatik indeks ham qurib qo'yadi — bu tekshiruv tez ishlashi uchun kerak (indekslarni 21-bobda chuqur ko'ramiz). FK'ni o'chirish:

```
ALTER TABLE kitoblar DROP FOREIGN KEY fk_kitob_muallif;
```

ON DELETE — parent o'chsa nima bo'lsin?

FK qo'shayotganda "ota qator o'chirilsa, bolalari bilan nima qilamiz?" degan savolga oldindan javob berib qo'yish mumkin. Uchta variantdan **bittasini** tanlaysiz. Diqqat: `fk_kitob_muallif` nomli FK'ni yuqoridagi bo'limda allaqachon yaratgan edik — xuddi shu nom bilan ikkinchisini qo'shib bo'lmaydi (MySQL Duplicate foreign key constraint name xatosini beradi). Shuning uchun qaysi variantni sinamang, avval eskisini o'chiring:

```
-- Avval oldingi bo'limda yaratilgan FK'ni o'chiramiz (aks holda
-- "Duplicate foreign key constraint name" xatosi chiqadi):
ALTER TABLE kitoblar DROP FOREIGN KEY fk_kitob_muallif;

-- Variant 1: o'chirishga ruxsat YO'Q (default, eng xavfsiz)
ALTER TABLE kitoblar ADD CONSTRAINT fk_kitob_muallif
FOREIGN KEY (muallif_id) REFERENCES mualliflar(id) ON DELETE RESTRICT;

-- Variant 2: muallif o'chsa, kitoblari HAM o'chadi
ALTER TABLE kitoblar ADD CONSTRAINT fk_kitob_muallif
FOREIGN KEY (muallif_id) REFERENCES mualliflar(id) ON DELETE CASCADE;
```

```
-- Variant 3: muallif o'chsa, kitobda muallif_id = NULL bo'lib qoladi
ALTER TABLE kitoblar ADD CONSTRAINT fk_kitob_muallif
FOREIGN KEY (muallif_id) REFERENCES mualliflar(id) ON DELETE SET NULL;
```

ON DELETE: ota qator o'chirilsa nima bo'ladi?

Ssenariy: DELETE FROM mualliflar WHERE id = 2 — bu muallifning kitoblari bor

RESTRICT (default)

O'chirish RAD etiladi
Muallif ham, kitoblar
ham joyida qoladi

MySQL xato qaytaradi,
hech narsa o'zgarmaydi

Eng xavfsiz yo'l

CASCADE

Muallif o'chadi —
kitoblari HAM
birga o'chadi

Zanjir bo'ylab davom
etadi (nabiralar ham!)

Ehtiyot bo'ling!

SET NULL

Muallif o'chadi —
kitoblarda muallif_id
NULL bo'lib qoladi

(ustun NULL'ga ruxsat
berishi shart)

Ega noma'lum qoladi

Qachon qaysi?

Pul/tarix — RESTRICT · komment/layk — CASCADE · ega noma'lum qolsin — SET NULL

Qachon qaysi? **Pul/tarix bor joyda — RESTRICT.** Komment/layk kabi "arzon" bog'liqlarda — CASCADE. "Bola qator qolsin, lekin egasi noma'lum bo'lsin" — SET NULL.

⚠ CASCADE bilan ehtiyot bo'ling: u zanjir bo'ylab ishlaydi. Muallifni o'chirsangiz kitoblari o'chadi, kitoblarga CASCADE bilan bog'langan ijaralar bo'lsa — ular ham o'chadi. Bitta DELETE ko'zda tutilmagan minglab qatorni olib ketishi mumkin. SET NULL esa faqat ustun NULL'ga ruxsat bersagina ishlaydi — `NOT NULL` ustunga SET NULL qo'yib bo'lmaydi.

✳ ON DELETE'ning egizagi ham bor: **ON UPDATE** — ota qatorning id'si o'zgarsa nima bo'lishini belgilaydi. Amalda PK qiymatlari deyarli hech qachon o'zgartirilmaydi, shuning uchun ko'pincha yozilmaydi (default — RESTRICT).

18-bob masalalari

- (kutubxona) kitoblar ga `til VARCHAR(30) DEFAULT 'ozbek'` ustuni qo'shing
- (kutubxona) Yangi kitob kiriting (tilsiz) — DEFAULT ishladimi?
- (kutubxona) azolar.telefon ga UNIQUE qo'shing: `ALTER TABLE azolar ADD CONSTRAINT uq_telefon UNIQUE (telefon);` — NULL'lar bor-ku, ishladimi? (UNIQUE bir nechta NULL'ga ruxsat beradi!)

4. (kutubxona) Mavjud telefon bilan a'zo kiritib ko'ring — endi xato chiqishi kerak
5. (kutubxona) `kitoblar` ga FK qo'shing (yuqoridagi misol), 999-muallif bilan kitob kiritib ko'ring
6. (kutubxona) `ijaralar` ga 2 ta FK qo'shing: `kitob_id` → `kitoblar`, `azo_id` → `azolar`
7. (kutubxona) Kitobi bor muallifni DELETE qilib ko'ring — RESTRICT xatosini oling
8. (kutubxona) CHECK qo'shing: `nusxa_soni >= 0`. Keyin -3 kiritib ko'ring
9. (kutubxona) CHECK qo'shing: `yil BETWEEN 800 AND 2100`. 2500-yil bilan sinab ko'ring
10. (dokon) `mahsulotlar.kategoriya_id` ga FK qo'shing (kategoriyalar'ga)
11. (dokon) `buyurtmalar.mijoz_id` va `buyurtma_qatorlari` ning 2 ustuniga FK qo'shing
12. (dokon) CHECK: `narx > 0` va `soni >= 0` — ikkalasini qo'shib, buzib ko'ring
13. (dokon) Yangi jadval `izohlar` yarating: `id`, `mahsulot_id` FK **ON DELETE CASCADE** bilan, matn. Buyurtmalarda mavjud mahsulotni tanlang (masalan, `id = 1`), unga izoh kiriting, keyin mahsulotni o'chirib ko'ring — o'chira olmaysiz (`buyurtma_qatorlari` RESTRICT ushlab turadi) — ana shu zanjirli himoyani his qiling. (Diqqat: 6- va 11-mahsulotlar hech bir buyurtmada yo'q — ulardan birini tanlasangiz, RESTRICT aralashmaydi, mahsulot izohi bilan birga o'chib ketadi)
14. (klinika) `qabullar` ga 2 ta FK (bemor, shifokor) RESTRICT bilan
15. (klinika) CHECK: `tolov >= 0`; `tajriba_yil <= 70`
16. (taksi) `mashinalar.haydovchi_id` ga FK **ON DELETE SET NULL** bilan (ustun NULL'ga ruxsat berishini tekshiring — kerak bo'lsa MODIFY qiling). Test haydovchi yarating, mashina bering, haydovchini o'chiring — mashinada nima qoldi?
17. (taksi) CHECK: `baho BETWEEN 1 AND 5` (NULL ruxsat etiladi — CHECK NULL'ni tekshirmaydi, bu normal)
18. (sinov) Ataylab: bir-biriga FK bilan bog'langan 2 jadval yaratib, parent'ni DROP qilib ko'ring — nima dedi? (avval FK yoki child o'chiriladi)
19. (kutubxona) `azolar.telefon` ni NOT NULL qiling (MODIFY) — NULL qiymat bor-ku (Nilufar Rahimovanning telefoni NULL), nima bo'ladi? Xatoni o'z ko'zingiz bilan ko'ring, keyin UPDATE bilan to'ldirib, qayta urinib ko'ring
20. Hammasiga FK qo'yib chiqqaningizdan keyin: `SELECT * FROM information_schema.KEY_COLUMN_USAGE WHERE TABLE_SCHEMA='dokon' AND`

`REFERENCED_TABLE_NAME IS NOT NULL;` — bazadagi barcha FK'larni ko'ring

19 — Tranzaksiyalar

[← Oldingi: 18 — ALTER, Constraint, Foreign Key](#) · [🏠 README](#) · [Keyingi: 20 — Normalizatsiya — to'g'ri schema →](#)

Bu bobda: tranzaksiya nima va u bizni qanday falokatdan asrashini ("yo hammasi, yo hech narsa" tamoyili), `START TRANSACTION`, `COMMIT` va `ROLLBACK` buyruqlarini, autocommit rejimini, parallel ishlaganda `FOR UPDATE` qulfi qanday qutqarishini, deadlock degan "tiqilinch" nima ekanini va intervyularning sevimli savoli — ACID xossalarini o'rganamiz.

Muammo: yarim bajarilgan ish

Do'konda buyurtma ikki qadamdan iborat: (1) buyurtma yoziladi, (2) ombordan kamayadi. 1-qadam bo'ldi, 2-qadamda svet o'chdi. Natija: buyurtma bor, ombor kamaymagan — **ma'lumot yolg'on**.

Bank misoli battarroq: Aziz Malikaga 100 000 o'tkazdi. (1) Azizdan ayirildi, (2) Malikaga... server qulab tushdi. 100 000 **g'oyib bo'ldi**.

Bunday "yarim ish"ning eng xavfli tomoni — hech qanday xato xabari chiqmaydi. Baza ishlayveradi, dastur ishlayveradi, faqat raqamlar yolg'on gapiradi. Bunday nosozlikni oradan hafta o'tib topish — pichan g'aramidan igna izlash bilan barobar.

Pul o'tkazish: nega ikkala UPDATE birga bo'lishi shart?



Tranzaksiya — bo'linmas paket: yo ikkala UPDATE birga, yo hech biri.

Yechim: tranzaksiya — "yo hammasi, yo hech narsa"

Tranzaksiya — bir nechta buyruqni bitta **bo'linmas paket**ga bog'lash. Paket yoki to'liq bajariladi, yoki umuman bajarilmaydi — o'rtasi yo'q. (`hisoblar` jadvalini hali yaratmadik — bob oxiridagi 4-masalada `sinov` bazasida o'zingiz qurasiz, hozir g'oyaga e'tibor bering.)

```
START TRANSACTION;  
  
UPDATE hisoblar SET balans = balans - 100000 WHERE egasi = 'Aziz';  
UPDATE hisoblar SET balans = balans + 100000 WHERE egasi = 'Malika';  
  
COMMIT;      -- ikkalasi ham muvaffaqiyatli — endi saqlansin
```

Agar orada xato bo'lsa:

```
ROLLBACK;    -- hammasi BEKOR, boshlang'ich holatga qaytadi
```

COMMIT'gacha o'zgarishlar "qoralama"da — boshqa foydalanuvchilar ko'rmaydi. Buni xat yozishga o'xshating: "yuborish" tugmasini bosmaguningizcha xatni xohlagancha tahrirlaysiz yoki butunlay o'chirib tashlaysiz. COMMIT — "yuborish", ROLLBACK — "o'chirib tashlash".

Tranzaksiya oqimi: yo COMMIT, yo ROLLBACK



ROLLBACK — baza tranzaksiya boshlanishidagi holatiga qaytadi

✦ Tranzaksiyalar jadvalning InnoDB "dvigateli"da ishlaydi. MySQL 8 da har bir yangi jadval standart holatda InnoDB bo'lib yaratiladi, shuning uchun qo'shimcha hech narsa sozlash shart emas — jadvallaringiz allaqachon tayyor.

Qo'lda sinab ko'rish

```
USE dokon;
```

```
START TRANSACTION;
UPDATE mahsulotlar SET soni = 0 WHERE id = 1;
SELECT soni FROM mahsulotlar WHERE id = 1; -- 0 ko'rinadi (sizga)
ROLLBACK;
SELECT soni FROM mahsulotlar WHERE id = 1; -- eski qiymat qaytdi!
```

Eslatma: MySQL'da har bir alohida buyruq o'z-o'zidan kichik tranzaksiya — yozdingizmi, darhol saqlanadi. Bu rejim **autocommit** deyiladi. `START TRANSACTION` esa "to'xta, men o'zim COMMIT demagunimcha saqlama" degani: bir nechta buyruqni bitta paketga bog'laydi.

Yana ikkita muhim qoidani bilib qo'ying:

- **Xato avtomatik ROLLBACK qilmaydi.** Tranzaksiya ichida bitta buyruq xato bersa (masalan, yo'q jadvalga INSERT), MySQL odatda faqat o'sha buyruqni rad etadi — tranzaksiya ochiq qolaveradi. COMMIT yoki ROLLBACK qarori sizda (buni 7-masalada o'zingiz sinaysiz).

- **DDL buyruqlar tranzaksiyani yopib yuboradi.** `CREATE TABLE`, `ALTER`, `DROP` kabi buyruqlar tranzaksiya o'rtasida kelsa, MySQL avval ungacha qilingan ishlarni **jimgina COMMIT qilib yuboradi** — `ROLLBACK`'ka qaytib bo'lmaydi. Shuning uchun tranzaksiya ichida faqat ma'lumot bilan ishlang: `INSERT`, `UPDATE`, `DELETE`, `SELECT`.

Real misol: buyurtma rasmiylashtirish

```
START TRANSACTION;

INSERT INTO buyurtmalar (mijoz_id, sana, holat) VALUES (2, NOW(),
'yangi');
SET @buyurtma = LAST_INSERT_ID();

INSERT INTO buyurtma_qatorlari (buyurtma_id, mahsulot_id, soni, narx)
VALUES (@buyurtma, 3, 1, 2800000);

UPDATE mahsulotlar SET soni = soni - 1 WHERE id = 3 AND soni >= 1;

COMMIT;
```

`@buyurtma` — sessiya o'zgaruvchisi (ulanish ochiq turguncha yashaydi): birinchi `INSERT` bergan id'ni eslab qolib, ikkinchisida ishlatdik. `AND soni >= 1` — omborda yo'q narsani sotib yubormaslik himoyasi.

Real dasturda `COMMIT`'dan oldin yana bir tekshiruv qilinadi: `SELECT ROW_COUNT()`; — oxirgi `UPDATE` nechta qatorni o'zgartirganini aytadi. `0` chiqdimi — demak shart bajarilmagan (ombor bo'sh ekan), butun buyurtmani `ROLLBACK` qilamiz; `1` bo'lsa — bema'lol `COMMIT`.

Parallellik: FOR UPDATE qulfi

Omborda oxirgi bitta iPhone qoldi deylik (`soni = 1`). Ikki kassir BIR VAQTDA sotmoqchi: ikkalasi ham `SELECT` bilan qaraydi, ikkalasi ham `soni = 1` ni ko'radi, "bor ekan!" deb ikkalasi ham sotadi → `soni = -1`. Mijozlardan biriga yo'q telefonni sotib qo'ydik.

Muammo — `SELECT` bilan `UPDATE` orasidagi vaqtda: siz "qancha bor?" deb qaragan paytdan "kamaytir" degan paytgacha boshqa odam ham ulgurib qolishi mumkin. Yechim — qatorni o'qiyotgandayoq qulflab qo'yish:

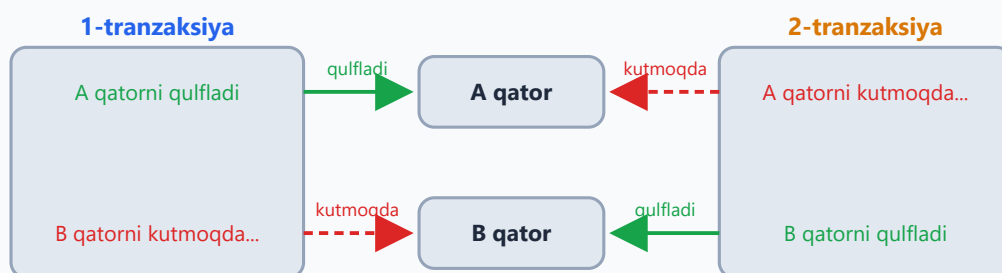
```
START TRANSACTION;
SELECT soni FROM mahsulotlar WHERE id = 2 FOR UPDATE;
-- FOR UPDATE = bu qator QULFLANDI. 2-kassir xuddi shu joyda KUTIB
TURADI,
-- toki biz COMMIT qilmagunimizcha. Keyin u yangi (kamaygan) sonni
ko'radi.
UPDATE mahsulotlar SET soni = soni - 1 WHERE id = 2;
COMMIT;
```

✦ FOR UPDATE faqat tranzaksiya ichida ma'noga ega: qulf COMMIT yoki ROLLBACK bo'lganda bo'shaydi. Shuning uchun tranzaksiyani iloji boricha qisqa tuting — qulf turgan paytda boshqalar navbatda kutib turadi.

Deadlock: ikki tranzaksiya bir-birini kutib qoladi

Qulf — kuchli qurol, lekin uning "yon ta'siri" bor. Tasavvur qiling: 1-tranzaksiya A qatorni qulflab olib, endi B qatorni so'rayapti. Xuddi shu payt 2-tranzaksiya B ni qulflab olib, A ni so'rayapti. Ikkalasi ham bir-birini kutadi — tor ko'chada ikki mashina yuzma-yuz kelib qolganday: hech biri orqaga yurmasa, ikkalasi ham abadiy qotib qoladi. Bu holat **deadlock** (o'zaro qulf) deyiladi.

Deadlock: ikki tranzaksiya bir-birini kutadi



Ikkalasi ham abadiy kutishi mumkin edi, lekin MySQL buni o'zi payqaydi:
birini tanlab ROLLBACK qiladi — "Deadlock found..." xatosi.

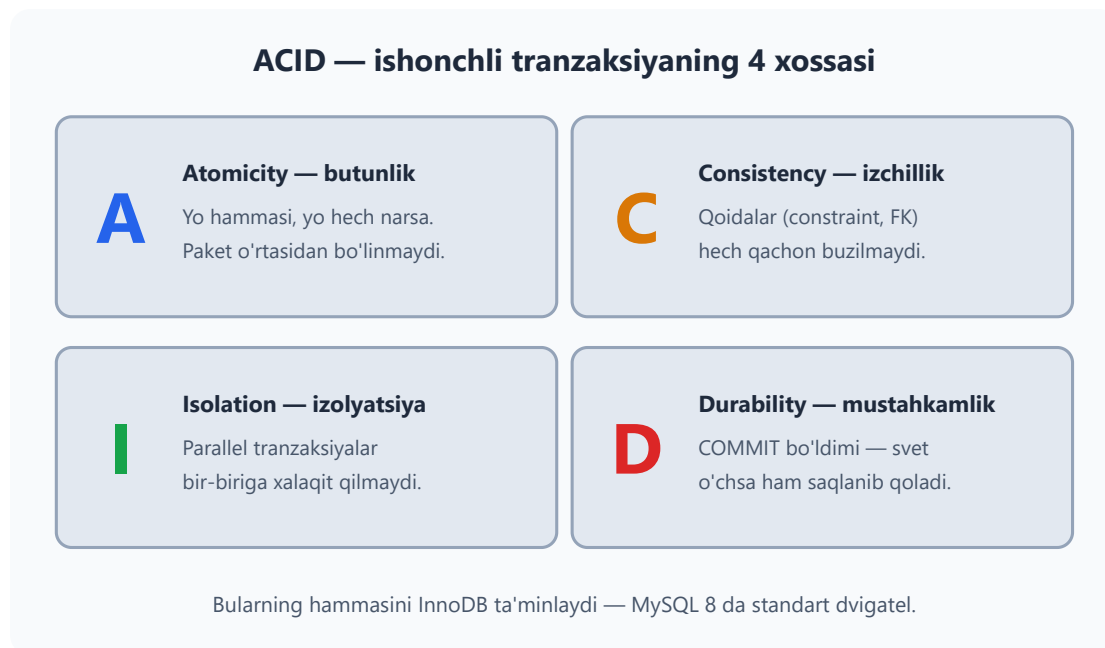
Oldini olish: qatorlarni doim bir xil tartibda qulflang, tranzaksiyani qisqa tuting.

Yaxshi yangilik: MySQL buni o'zi payqaydi va "qurbon" sifatida bittasini tanlab ROLLBACK qilib yuboradi — o'sha tranzaksiya `Deadlock found when trying to get lock` xatosini oladi. Dastur bu xatoni ushlab, tranzaksiyani qaytadan urinishi kerak. Oldini olish retsepti esa oddiy: qatorlarni hamma joyda **bir xil tartibda**

qulflang (masalan, doim kichik id'dan kattasiga qarab) va tranzaksiyani qisqa tuting.

ACID — bilib qo'ying (intervyu savoli)

Tranzaksiya haqida gap ochilsa, intervyuda albatta ACID so'raladi. Bu — ishonchli tranzaksiyaning 4 xossasi:



- **Atomicity (butunlik):** yo hammasi, yo hech narsa — paket o'rtasidan bo'linmaydi. Pul o'tkazmaning "yarmi" bajarilib qolishi mumkin emas.
- **Consistency (izchillik):** tranzaksiya bazani bir to'g'ri holatdan boshqa to'g'ri holatga o'tkazadi — qoidalar (constraint, FOREIGN KEY) hech qachon buzilmaydi.
- **Isolation (izolyatsiya):** parallel tranzaksiyalar bir-biriga xalaqit qilmaydi — har biri o'zini "bazada yolg'iz ishlayapman" deb his qiladi.
- **Durability (mustahkamlik):** COMMIT bo'ldimi — svet o'chsa ham, server qulab tushsa ham ma'lumot diskda saqlanib qolgan.

19-bob masalalari

1. (dokon) Yuqoridagi "qo'lda sinash"ni bajaring: TRANSACTION → UPDATE → ROLLBACK → tekshiring

2. (dokon) Xuddi shuni COMMIT bilan: o'zgarish saqlanib qoldimi?
3. (dokon) ROLLBACK'dan keyin ROLLBACK qilib ko'ring — nima bo'ladi? (hech nima, tranzaksiya allaqachon tugagan)
4. (sinov) `hisoblar` jadvali yarating (egasi VARCHAR(100), balans DECIMAL), Aziz=500000, Malika=200000 kiriting
5. (sinov) Azizdan Malikaga 100 000 o'tkazing — to'liq tranzaksiya bilan. Oxirida ikkala balansni tekshiring (jami 700 000 bo'lishi kerak!)
6. (sinov) O'tkazma qiling, lekin COMMIT o'rniga ROLLBACK — balanslar joyidami?
7. (sinov) Tranzaksiya oching, avval bitta UPDATE bajaring (masalan, Aziz balansiga 1 qo'shing), keyin ataylab xato buyruq yozing (mavjud bo'lmagan jadvalga INSERT) — xato chiqadi, lekin tranzaksiya ochiq qoladi. Endi ROLLBACK qiling. Birinchi UPDATE ham bekor bo'ldimi, tekshiring
8. (dokon) "Buyurtma rasmiylashtirish" misolini o'zingiz uchun takrorlang: yangi buyurtma + 2 ta qator + ombor kamaytirish, hammasi bitta tranzaksiyada
9. (dokon) `UPDATE ... SET soni = soni - 1 WHERE id = 6 AND soni >= 1` — Acer (soni=0 yoki siz qilgan qiymat) uchun bajaring. `ROW_COUNT()` funksiyasi bilan nechta qator o'zgarganini tekshiring: `SELECT ROW_COUNT();` — 0 chiqsa "sotib bo'lmadi" degani
10. (kutubxona) Kitob berish tranzaksiyasi: `ijaralar` ga INSERT + `kitoblar` da `nusxa_soni - 1` (faqat nusxa bor bo'lsa!)
11. (kutubxona) Kitob qaytarish tranzaksiyasi: `ijaralar` da `qaytarilgan_sana` ni to'ldirish + `kitoblar` da `nusxa_soni + 1`
12. (klinika) Qabul yozish tranzaksiyasi: `qabullar` ga INSERT (sana NOW(), tolav shifokorning `qabul_narxi` ga teng — avval SELECT bilan oling yoki subquery ishlatib)
13. FOR UPDATE'ni HIS QILISH (2 ta oyna kerak!): 2 ta terminal oching. 1-oykada: START TRANSACTION; SELECT ... FOR UPDATE. 2-oykada xuddi shu SELECT ... FOR UPDATE — u KUTIB qoladi! 1-oykada COMMIT qiling — 2-oyka davom etadi. Buni ko'rgan odam qulflarni unutmaydi
14. 13-mashqni FOR UPDATE'siz takrorlang — 2-oyka kutmaydi, darrov o'qiydi. Farqni yozib oling
15. (taksi) Safar yakunlash tranzaksiyasi: `safarlar` ga INSERT + haydovchi reytingini yangilash (o'rtacha baho asosida)
16. Savol-javob: nega SELECT'larga odatda tranzaksiya kerak emas? (o'zgartirmaydi). Qachon kerak? (izchil "surat" kerak bo'lganda)
17. (sinov) Autocommit'ni his qiling: `SET autocommit = 0;` qiling, UPDATE bajaring, boshqa oynadan qarang (ko'rinmaydi), COMMIT qiling (ko'rinadi).

Oxirida `SET autocommit = 1;` ga qaytaring

18. (dokon) Tranzaksiya ichida 3 ta UPDATE, 2-sidan keyin `SAVEPOINT nuqta1;` qo'ying, 3-UPDATE'dan keyin `ROLLBACK TO nuqta1;` — faqat 3-si bekor bo'ladi. Sinang!
19. O'ylang: ombor kamaytirishda nega `WHERE soni >= 1` sharti qulfdan tashqari YANA kerak? (himoya qatlamlari)
20. Yozing: o'z hayotingizdan 3 ta "tranzaksiya bo'lishi SHART" ssenariy (masalan: Click to'lov, aviabilet bron...)

20 — Normalizatsiya — to'g'ri schema

← Oldingi: 19 — Tranzaksiyalar · 🏠 README · Keyingi: 21 — Indekslar →

Bu bobda: "hammasi bitta jadvalda" dizayni qanday kasalliklarga olib kelishini (takrorlash, yangilash, o'chirish va kiritish anomaliyalari), normalizatsiyaning 3 ta sodda qoidasini (1NF, 2NF, 3NF — oddiy tilda), jadvallar orasidagi bog'lanish turlarini (1:1, 1:N, N:M) va qachon ataylab takrorlash to'g'ri ekanini (snapshot, denormalizatsiya) o'rganamiz.

Yomon jadval qanday ko'rinadi

Shu paytgacha tayyor, to'g'ri loyihalangan bazalar bilan ishladik. Endi eng muhim savolga keldik: **nega** do'kon bazasi 5 ta jadval? Bitta katta jadval qulayroq emasmi — hammasi bir joyda-ku? Javobni his qilish uchun avval o'sha "qulay" variantni ko'raylik. Tasavvur qiling, do'kon hamma narsani BITTA jadvalga yozadi:

buyurtmalar_HAMMASI:

mijoz_ism	mijoz_tel	mahsulotlar	summa
Davron	+99891123...	Telefon, Chexol	2850000
Davron	+99891123...	Noutbuk	6500000

Birinchi qarashda sodda ko'rinadi. Lekin bu jadval kamida 5 ta kasallik bilan og'rigan:

1. **Takrorlash:** Davronning telefoni har qatorda qayta yoziladi. 100 buyurtma — 100 nusxa. Joy isrofi-ku mayli, asl xavf keyingi badda
2. **Yangilash anomaliyasi:** Davron raqamini o'zgartirsa — 100 joyda yangilash kerak. Bittasi qolib ketsa, bazada ikki xil "haqiqat" paydo bo'ladi: qaysi raqam to'g'ri? Endi hech kim bilmaydi
3. **Bitta katakda ko'p qiymat:** "Telefon, Chexol" — bu SQL uchun shunchaki matn, ro'yxat emas. "Chexol jami nechta sotilgan?" deb so'rab bo'lmaydi — alohida mahsulotni qidirish/sanash azobga aylanadi
4. **O'chirish anomaliyasi:** Davronning yagona buyurtmasini o'chirsak — Davron haqidagi ma'lumot (ismi, telefoni) ham birga yo'qoladi
5. **Kiritish anomaliyasi:** hali buyurtma qilmagan mijozni qo'shib bo'lmaydi — mahsulot va summa kataklariga nima yozasiz?

Bitta katta jadvalning kasalliklari

buyurtmalar_HAMMASI			
mijoz_ism	mijoz_tel	mahsulotlar	summa
Davron	+998911234567	Telefon, Chexol	2 850 000
Davron	+998911234567	Noutbuk	6 500 000
takror!		bitta katakda 2 qiymat!	

1. Takrorlash

Bir xil ma'lumot yuzlab nusxada —
joy isrofi va chalkashlik xavfi

2. Yangilash anomaliyasi

Telefon o'zgarsa — yuzlab joyda
yangilash kerak, bittasi qolib ketadi

3. Bitta katakda ko'p qiymat

"Telefon, Chexol" — SQL uchun bu
ro'yxat emas, matn: qidirish/sanash azob

4. O'chirish anomaliyasi

Yagona buyurtmani o'chirsak —
mijoz haqidagi ma'lumot ham yo'qoladi

5. Kiritish anomaliyasi

Hali buyurtma qilmagan mijozni
jadvalga qo'shishning iloji yo'q

Davo — normalizatsiya: har fakt o'z jadvalida, bir marta

Bunday muammolarning rasmiy nomi bor: **anomaliya** — jadval tuzilishi noto'g'ri bo'lgani uchun kelib chiqadigan "g'alati" holatlar. Davo esa bitta: **normalizatsiya** — ma'lumotni takrorsiz, tartibli jadvallarga ajratish.

Davo: 3 ta sodda qoida

Normalizatsiya — qo'rqinchli akademik so'z, lekin mag'zi uchta oddiy qoidaga sig'adi:

1-qoida (1NF): bitta katak — bitta qiymat. "Telefon, Chexol" ✗ → har mahsulot alohida qatorda (buyurtma_qatorlari jadvali!). Bitta buyurtma — bir nechta qator, har qatorda bitta mahsulot.

2-qoida (2NF/3NF soddalashtirilgan): har fakt — O'Z jadvalida, BIR marta.
Mijoz telefoni — mijozlar da. Mahsulot narxi — mahsulotlar da. Buyurtma sanasi — buyurtmalar da. Telefon o'zgarsa — BIR joyda yangilanadi, vassalom: yangilash anomaliyasi yo'qoldi.

3-qoida: jadvallar id orqali bog'lanadi (FK). Buyurtma mijozning ismini emas, mijoz_id sini saqlaydi. Ism kerak bo'lsa — JOIN qilamiz (12-bob), bog'lanish buzilmasligini esa FOREIGN KEY qo'riqlaydi (18-bob).

Normalizatsiya qadamlari — bitta misolda

1NF

Bitta katak — bitta qiymat

Avval: mahsulotlar katagida 'Telefon, Chexol' — ikki qiymat

Keyin: har mahsulot alohida qatorda (buyurtma_qatorlari)

2NF

Har fakt o'z jadvalida, bir marta

Avval: mijoz telefoni har buyurtma qatorida takrorlanadi

Keyin: telefon faqat mijozlar'da, buyurtmada — mijoz_id (FK)

3NF

Oddiy ustun boshqa oddiy ustunga bog'liq bo'lmasin

Avval: kategoriya_nomi har mahsulot qatorida takrorlanadi

Keyin: kategoriyalar jadvali, mahsulotda — kategoriya_id (FK)

Uchala qadam ham bitta g'oya: takrorni yo'qotish — har faktga bitta "uy"

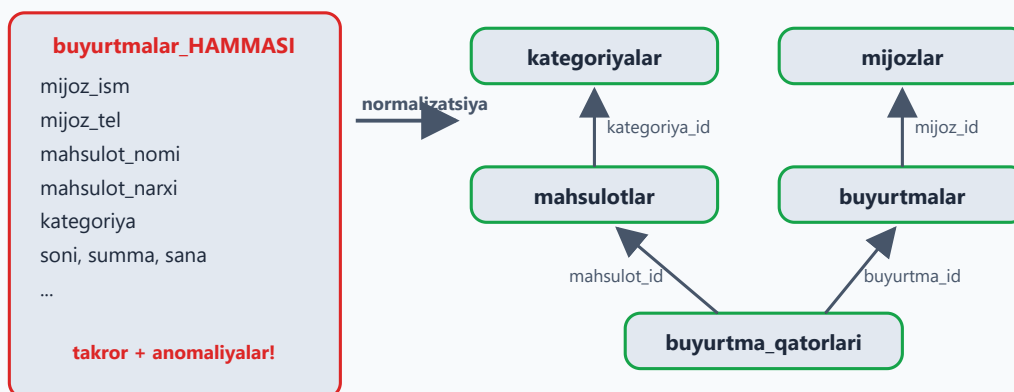
✦ Rasmiy atamalar: bu qoidalar **normal formalar** (1NF, 2NF, 3NF) deb ataladi. Akademik kitoblarda ularning qat'iy ta'riflari (va davomi: BCNF, 4NF, 5NF...) bor, lekin amaliyotda yuqoridagi uch qoidaga amal qilsangiz, bazangiz taxminan 3NF darajasida bo'ladi — bu mutlaq ko'pchilik loyihaga yetib ortadi.

Endi `buyurtmalar_HAMMASI` ni shu qoidalar bilan "davolaylik". Natijada nima chiqadi?

```
mijozlar:          id, ism, telefon
kategoriyalar:     id, nomi
mahsulotlar:       id, nomi, kategoriya_id, narx, ...
buyurtmalar:       id, mijoz_id, sana, holat
buyurtma_qatorlari: id, buyurtma_id, mahsulot_id, soni, narx
```

Tanish-a? Natija — bizning `dokon` bazasi! 5 jadval: kategoriyalar, mahsulotlar, mijozlar, buyurtmalar, buyurtma_qatorlari. Siz 3-bobdan beri NORMALLASHGAN baza bilan ishlayapsiz :) Endi nega aynan shunday loyihalanganini ham bilasiz.

Normalizatsiyadan keyin: 5 ta bog'langan jadval



Har fakt bitta joyda — jadvallar id (FOREIGN KEY) orqali bog'lanadi

Bu — siz 3-bobdan beri ishlatayotgan dkon bazasi!

Bog'lanish turlari

Jadvallarni ajratdik — endi ular orasidagi bog'lanishlar qanday turga bo'linishini ko'raylik. Hammasi bo'lib 3 xil:

Tur	Misol	Qanday quriladi
1:1	odam — pasport	FK + UNIQUE
1:N	muallif — kitoblar	child'da FK (kitoblar.muallif_id)
N:M	kitob — a'zo (ijara)	oraliq jadval (ijaralar)

1:N — eng keng tarqalgani. Bitta muallifning ko'p kitobi bor, lekin har kitobning bitta muallifi. FK doim "ko'p" tomonda turadi: `kitoblar.muallif_id`. Eslab qolish oson: "bola ota-onasining id'sini saqlaydi".

1:1 kamroq uchraydi. Uni odatda katta jadvalning kam ishlatiladigan yoki maxfiy qismini alohida saqlash uchun qilamiz. FK ustuniga `UNIQUE` qo'yilmasa, bog'lanish bilinmay 1:N bo'lib qoladi — bitta haydovchiga ikkita hujjat qatori kirib ketishi mumkin:

```
-- taksi bazasida (10-masala uchun namuna)
CREATE TABLE haydovchi_hujjatlari (
  id INT AUTO_INCREMENT PRIMARY KEY,
```



```

    haydovchi_id INT NOT NULL UNIQUE,    -- UNIQUE: har haydovchiga
    ko'pi bilan bitta qator
    litsenziya_raqami VARCHAR(30),
    tibbiy_korik_sanasi DATE,
    FOREIGN KEY (haydovchi_id) REFERENCES haydovchilar(id)
);

```

N:M — bitta kitobni ko'p a'zo olgan, bitta a'zo ko'p kitob olgan. N:M ni hech qachon to'g'ridan-to'g'ri qilib bo'lmaydi — doim o'rtada jadval: ijaralar, buyurtma_qatorlari, qabullar... Oraliq jadvalning har bir qatori — bitta "uchrashuv" fakti: "kim qaysi kitobni qachon oldi". E'tibor bering: bizning hamma "bog'lovchi" jadvallar aynan shu!

Qachon ataylab takrorlaymiz (snapshot)

3-bobda savol bergandik: `buyurtma_qatorlari.narx` nega bor — mahsulotda narx bor-ku? Bir qarashda bu normalizatsiyaga zid: bitta fakt ikki joyda turibdi!

Javob: mahsulot narxi **ertaga o'zgaradi**. Lekin mijoz **o'sha kungi** narxda olgan! Buyurtmaga narx **nusxalanadi** — bu tarixiy haqiqat (snapshot). Bu normalizatsiya buzilishi emas: `mahsulotlar.narx` — "hozir qancha turadi" degan fakt, `buyurtma_qatorlari.narx` — "o'sha kuni qancha to'langan" degan ALOHIDA fakt. Ikki xil fakt — ikki joy. Hayotiy misol — do'kon cheki: ertaga narx oshsa ham, qo'lingizdagi chekda eski narx turaveradi.



Oddiy qoida:

Savol	Yechim
Hozirgi qiymat kerakmi?	saqlamang — JOIN bilan asl jadvaldan oling
O'sha paytdagi qiymat kerakmi?	nusxalang — bu snapshot, tarixiy fakt

💡 Ba'zan tezlik uchun ham ataylab takrorlanadi — bu **denormalizatsiya** deyiladi. Masalan, har safar layklarni `COUNT` bilan sanamaslik uchun postga tayyor `layklar_soni` ustunini qo'shish. Lekin bu — "keyin o'ylanadigan" optimizatsiya: endi kesh ustunini asl ma'lumot bilan sinxron ushlash sizning zimmangizda. Qoida shunday: **avval normallang, keyin zarurat tug'ilsagina ongli ravishda buzing.**

20-bob masalalari

1. Yuqoridagi `buyurtmalar_HAMMASI` jadvalini qog'ozda normallang: qanday jadvallar chiqadi?
2. Quyidagi jadvalni normallang (qog'ozda): `talaba(ism, guruh_nomi, guruh_xonasi, fan1, fan2, fan3, oqituvchi1, oqituvchi2)` — qaysi qoidalar buzilganini ham ayting
3. Sportzal tizimini loyihalang: a'zolar, trenerlar, mashg'ulotlar, qatnashuvlar — jadvallar + ustunlar + FK'lar (qog'ozda chizing, keyin SQL'da yarating)
4. Mehmonxona: xonalar, mehmonlar, bronlar. Bron — qaysi bog'lanish turi? (N:M, vaqt bilan)
5. Instagram'ning soddalashtirilgan modeli: userlar, postlar, layklar, kommentlar, obunalar (kim kimga) — chizing va yarating. Obunalar jadvalida 2 ta FK BITTA jadvalga (userlar) ishora qiladi!
6. 5-masala bazasiga test data kiriting (har jadvalga 5+ qator)
7. 5-masala: "har postning layklari soni" query'sini yozing — normallangan schema buni osonlashtirdimi?
8. Taksi tizimimizda kamchilik bor: `safarlar.mijoz_ism` — oddiy matn! Mijoz alohida jadval bo'lishi kerakmidi? Qachon shart, qachon shart emas? (mijoz tarixi/akkaunti keraklimi?) Fikringizni yozing
9. Kamchilikni tuzating: `mijozlar` jadvalini yarating, `safarlar` ga `mijoz_id` qo'shing, ismlardan mijozlar yasab ko'chiring (`INSERT INTO mijozlar (ism) SELECT DISTINCT mijoz_ism FROM safari;`)

10. 1:1 misolini quring: `haydovchilar` + `haydovchi_hujjatlari` (litsenziya raqami, tibbiy ko'rik sanasi) — FK UNIQUE bilan
11. Nega telefon raqamlarini `azo`lar ichida vergul bilan emas (`' +9989... , +9989... '`), alohida `azo_telefonlari` jadvalida saqlash kerak? Yarating va 2 raqamli a'zo qo'shing
12. "Yetim qator" yasang. Diqqat: 18-bob 10-masalasida `mahsulotlar.kategoriya_id` ga FK qo'shgansiz — u bunday kiritishni darhol rad etadi (1452-xato). Shuning uchun FK'SIZ test-jadval yarating: `CREATE TABLE test_mahsulotlar (id INT AUTO_INCREMENT PRIMARY KEY, nomi VARCHAR(100), kategoriya_id INT);` — va unga mavjud bo'lmagan `kategoriya_id` bilan mahsulot kiriting (xohlasangiz, buning o'rniga `mahsulotlar` dagi FK'ni vaqtincha `DROP FOREIGN KEY` qilib, tajribadan keyin qaytarib qo'yishingiz ham mumkin). Endi `kategoriyalar` bilan JOIN qiling — mahsulot natijadan yo'qoldi (INNER) yoki kategoriyasi NULL (LEFT). FK shuning oldini olardi!
13. Kutubxonada kitobning 2 muallifi bo'lsa-chi (hammualliflik)? Hozirgi schema buni qo'llab-quvvatlaydimi? Qanday o'zgartirish kerak? (`kitob_muallif` oraliq jadvali!) Yarating
14. Maktab davomat tizimi: o'quvchilar, sinflar, darslar, davomat — loyihalang. Davomatda sana + holat (keldi/kelmadi/kechikdi)
15. Snapshot kerak joylarni toping: klinika qabulida shifokor narxi o'zgarsa, eski qabullardagi `tolov` o'zgaradimi? (yo'q — `tolov` allaqachon snapshot!) Yana 2 ta snapshot misolini o'ylab toping
16. Anti-misol quring: hamma narsani BITTA jadvalga yozadigan `kutubxona_hammasi` yarating, 5 qator kiriting, keyin "muallif davlatini o'zgartirish" qancha mashaqqat ekanini his qiling
17. Dorixona tizimi: dorilar, yetkazib beruvchilar, kirimlar, sotuvlar — loyihalang (narx snapshot'larini unutmang!)
18. O'zingiz ishlatadigan biror ilovani (Click/Payme) oling: 6–8 jadvalini taxmin qilib chizing
19. ER-diagramma: DBeaver'da bazangizni oching → ER Diagram ko'ring (jadvallar va bog'lanishlar rasmi) — o'z ishingizni vizual ko'rish juda foydali
20. Yakuniy test: do'stingizga (yoki o'zingizga) tushuntiring: "nega 5 ta jadval, bitta katta jadval emas?" — 3 ta sabab ayta olsangiz, bob o'zlashtirilgan!

21 — Indekslar

◀ Oldingi: 20 — Normalizatsiya — to'g'ri schema · 🏠 README · Keyingi: 22 — VIEW, Stored Procedure, Trigger, Event ▶

Bu bobda: indeks nima ekanini kitob oxiridagi alfavit ko'rsatkich o'xshatishi bilan tushunamiz, 1 million qatorli jadval yasab full scan va indeksli qidiruv farqini o'z qo'limiz bilan o'lchaymiz, indeks ichidagi B-tree daraxtini, indeks ishlamay qoladigan tuzoqlarni, kompozit indeks va chap prefiks qoidasini hamda indeksning narxini o'rganamiz.

Indeks nima?

500 betlik kitobdan "fotosintez" so'zini qidiraysiz. 2 yo'l: 1. Har sahifani o'qib chiqish — yarim kun 2. Kitob oxiridagi **alfavit ko'rsatkich**ga qarash: "fotosintez — 217-bet" — 10 soniya

Indeks — jadval ustuni uchun ana shunday "ko'rsatkich". Usiz MySQL `WHERE` shartiga mos qatorni topish uchun **har bir qatorni** birma-bir tekshiradi (bu **full table scan** deyiladi), indeks bilan esa — to'g'ridan-to'g'ri kerakli qatorga boradi.



Yupqa broshyurada ko'rsatkichning keragi yo'q — varaqlash ham tez. Xuddi shunday, kichik jadvalda indeks farqi sezilmaydi: indeksning kuchi **yuz minglab va millionlab** qatorda ko'rinadi. Shu bobda buni o'zimiz o'lchab ko'ramiz.

Yaratish va ko'rish

```
CREATE INDEX idx_kitoblar_janr ON kutubxona.kitoblar(janr);  
SHOW INDEX FROM kutubxona.kitoblar;  
DROP INDEX idx_kitoblar_janr ON kutubxona.kitoblar;
```

Nomlashda keng tarqalgan odat — `idx_jadval_ustun` ko'rinishi: keyin `SHOW INDEX` ro'yxatida qaysi indeks nima uchunligini darrov tushunasiz.

✦ Ba'zi indekslar **avtomatik** paydo bo'ladi: PRIMARY KEY va UNIQUE — har doim indeks. FOREIGN KEY ustuniga ham MySQL (InnoDB) o'zi indeks qo'yib qo'yadi. Shuning uchun yangi indeks qo'shishdan oldin `SHOW INDEX` bilan tekshiring — balki allaqachon bordir.

Indeks ichida nima bor? — B-tree

MySQL indeksni **B-tree** (balanced tree — muvozanatli daraxt) ko'rinishida saqlaydi: qiymatlar saralangan holda daraxt shoxlariga joylanadi. Qidiruvda MySQL ildizdan boshlaydi va har qadamda "kichikmi-kattami?" deb solishtirib, kerakli shoxga buriladi:



Eng qizig'i — bu daraxt juda "yassi": million qatorli jadvalda ham ildizdan barggacha bor-yo'g'i 3-4 qadam yetadi. Har qadamda ma'lumotning katta qismi

chetda qolib ketadi — shuning uchun indeks shunchalik tez.

Farqni O'LCHAB ko'ramiz (eng muhim mashq!)

Kichik jadvalda farq sezilmaydi. Katta jadval yasaymiz:

```
CREATE DATABASE katta;
USE katta;

CREATE TABLE foydalanuvchilar (
  id INT AUTO_INCREMENT PRIMARY KEY,
  ism VARCHAR(50),
  yosh INT,
  shahar VARCHAR(50)
);

-- 1 million qator generatsiya (recursive CTE — 15-bobdan tanish!):
SET cte_max_recursion_depth = 1000000; -- standart chegara 1000 edi,
ko'taramiz

INSERT INTO foydalanuvchilar (ism, yosh, shahar)
WITH RECURSIVE seq AS (
  SELECT 1 AS n
  UNION ALL
  SELECT n + 1 FROM seq WHERE n < 1000000
)
SELECT
  CONCAT('User', n),
  FLOOR(18 + RAND() * 50),
  ELT(1 + FLOOR(RAND() * 5), 'Toshkent', 'Samarqand', 'Buxoro',
  'Andijon', 'Nukus')
FROM seq;
-- 10-30 soniya kutib turing
```

Endi solishtiramiz:

```
SELECT * FROM foydalanuvchilar WHERE ism = 'User777777';
-- vaqtga qarang: ~0.3-0.5 sekund (full scan, million qator
tekshirildi)

CREATE INDEX idx_ism ON foydalanuvchilar(ism); -- bir necha soniya

SELECT * FROM foydalanuvchilar WHERE ism = 'User777777';
-- ~0.001 sekund. YUZLAB BARAVAR TEZ!
```

1 000 000 qator: full scan va indeksli qidiruv

Full scan (indekssiz)



Indeks orqali (idx_ism)



Farq — yuzlab baravar. Buni hozir o'zingiz o'lchab ko'rasiz!

💡 MySQL indeksdan foydalandimi-yo'qmi — taxmin qilib o'tirmang, query oldiga `EXPLAIN` so'zini qo'shing:

```
EXPLAIN SELECT * FROM foydalanuvchilar WHERE ism = 'User777777';  
-- type: ALL → full scan (katta jadvalda yomon belgi)  
-- type: ref → indeks orqali (idx_ism ishladi)
```

`EXPLAIN` ni 23-bobda chuqur o'rganamiz, hozircha `type` va `rows` ustunlariga qarash yetarli.

Indeks ISHLAMAYDIGAN holatlar

Indeks bor-u, lekin MySQL undan foydalana olmaydigan vaziyatlar ham bor — eng ko'p uchraydigan tuzoqlar:

```
WHERE ism LIKE 'User77%'      -- ✅ ishlaydi (boshi aniq)  
WHERE ism LIKE '%7777'      -- ❌ ishlamaydi (boshi noma'lum —  
    lug'atda "oxiri ...ov so'zlar"ni qidirish kabi)  
WHERE YEAR(sana) = 2026      -- ❌ ustunga funksiya — indeks o'chadi  
WHERE sana >= '2026-01-01' AND sana < '2027-01-01' -- ✅ to'g'ri  
    yo'l
```

Nega funksiya indeksni "o'chiradi"? Indeks `sana` qiymatlari bo'yicha saralangan, `YEAR(sana)` natijalari bo'yicha emas. Shartni tekshirish uchun MySQL har bir

qatorda funksiyani hisoblashga majbur — bu yana full scan. Qoida oddiy: **shart ichida ustunni "toza" qoldiring**, hisob-kitobni taqqoslashning narigi tomoniga o'tkazing.

✦ MySQL 8.0.13+ da **funksional indeks** ham bor: `CREATE INDEX idx_yil ON jadval ((YEAR(sana)))`; — qo'sh qavsga e'tibor bering. Lekin ko'p hollarda shartni to'g'ri yozishning o'zi (yuqoridagi ☒ varianti) yetarli va soddaroq.

Kompozit indeks

Bitta indeks bir nechta ustunni qamrashi mumkin:

```
CREATE INDEX idx_shahar_yosh ON foydalanuvchilar(shahar, yosh);
```

Bu "shahar, ichida yosh bo'yicha saralangan" ko'rsatkich. Qoida — **chapdan boshlab ishlaydi** (chap prefiks qoidasi):

```
WHERE shahar = 'Toshkent' -- ☒
WHERE shahar = 'Toshkent' AND yosh = 25 -- ☒ (eng zo'r)
WHERE yosh = 25 -- ☒ (faqat yosh — chap qism yo'q)
```

Kompozit indeks (shahar, yosh): chap prefiks qoidasi

Indeks (saralangan)

shahar	yosh
Andijon	19
Andijon	25
Buxoro	22
Buxoro	31
Toshkent	25
Toshkent	25
Toshkent	40

avval shahar bo'yicha,
ichida yosh bo'yicha saralangan

- WHERE shahar = 'Toshkent'
☒ **ishlaydi — chap ustun bor**
- WHERE shahar = 'Toshkent' AND yosh = 25
☒ **eng zo'r — ikkala ustun ham indeksdan**
- WHERE yosh = 25
☒ **ishlamaydi — chap qism (shahar) yo'q**

Telefon kitobchasi ham shunday: familiya+ism bo'yicha saralangan — faqat ismni bilib qidirsangiz, hammasini varaqlashga to'g'ri keladi.
(shahar, yosh) va (yosh, shahar) — ikki xil indeks: tartib muhim!

Telefon kitobi analogiyasi: familiya+ism bo'yicha saralangan. Familiyani bilsangiz topasiz; faqat ismni bilsangiz — hammasini varaqlaysiz.

💡 Shu sababli ustunlar **tartibi** muhim: (shahar, yosh) va (yosh, shahar) — ikki xil indeks. Qaysi ustun shartlarda yolg'iz ham tez-tez kelsa, o'shani chap tomonga qo'ying.

Indeksning narxi

Indeks **tekinga kelmaydi**:

Narxi	Sababi
Yozish sekinlashadi	har INSERT/UPDATE/DELETE'da indeks(lar) ham yangilanadi
Disk joy oladi	indeks — alohida saqlanadigan tuzilma (ba'zan jadvalning o'zidan ham katta)
Foyda har doim ham yo'q	kam xil qiymatli ustunda indeks deyarli yordam bermaydi

Oxirgi qator **selektivlik** tushunchasi: ustunda qancha ko'p xil qiymat bo'lsa, indeks shuncha foydali. `ism` (million xil qiymat) — zo'r nomzod; `jins` (2 xil qiymat) — indeks bo'lsa ham million qatorning yarmi baribir o'qiladi, foydasi kam.

Shuning uchun: **WHERE/JOIN/ORDER BY'da tez-tez ishlatiladigan ustunlarga** qo'ying, "har ehtimolga" emas.

21-bob masalalari

- `katta` bazasini yarating va 1 mln qator generatsiya qiling (yuqoridagi skript)
- Indekssiz qidiruv: `WHERE ism = 'User5000000'` — vaqtini yozib oling
- `idx_ism` yarating, qidiruvni takrorlang — necha baravar tezlashdi?
- `WHERE yosh = 30` — indekssiz vaqti? `idx_yosh` yarating, takrorlang
- `WHERE shahar = 'Buxoro' AND yosh = 25` — indekssiz (yosh indeksini DROP qilib), keyin kompozit (`shahar, yosh`) bilan solishtiring
- Kompozit indeks bilan `WHERE yosh = 25` (shaharsiz) — tezmi? Nega sekin? (chap qism yo'q!)

7. `WHERE ism LIKE 'User9999%'` va `WHERE ism LIKE '%9999'` — vaqtlarini solishtiring, sababini yozing
8. `SHOW INDEX FROM foydalanuvchilar;` — nechta indeks bor? PRIMARY ham ko'rinyaptimi?
9. Indeks hajmi: `SELECT table_name, ROUND(data_length/1024/1024) AS data_mb, ROUND(index_length/1024/1024) AS index_mb FROM information_schema.tables WHERE table_schema='katta';` — indekslar necha MB?
10. Yozish narxi: hamma qo'shimcha indeksni DROP qiling, 100 000 qator INSERT vaqtini o'lchang; 3 ta indeks yaratib yana 100 000 INSERT qiling — farq sezildimi? (eslatma: yangi sessiyada `cte_max_recursion_depth` standart 1000 ga qaytadi — INSERT'dan oldin uni qayta `SET` qiling; ismlar mavjud `User1..User1000000` bilan takrorlanib qolmasligi uchun `n` ni 1000001 dan boshlang: `SELECT 1000001 AS n` va `WHERE n < 1100000` — shunda 14-masaladagi UNIQUE tajribasi ham toza qoladi)
11. (kutubxona) `ijaralar(azo_id)` va `ijaralar(kitob_id)` ga indeks qo'ying (FK qo'ygan bo'lsangiz allaqachon bor — `SHOW INDEX` bilan tekshiring)
12. (dokon) Qaysi ustunlarga indeks kerak? O'ylab ro'yxat yozing (maslahat: `buyurtmalar.mijoz_id`, `buyurtmalar.sana`, `mahsulotlar.kategoriya_id...`) va qo'ying
13. (taksi) `safarlar(haydovchi_id, sana)` kompozit indeks qo'ying — "falon haydovchining falon kungi safarlari" query'si uchun ideal
14. UNIQUE indeks: `foydalanuvchilar` da `ism` ni UNIQUE qilib bo'ladimi? Sinab ko'ring (duplikatlar bormi — `SELECT ism, COUNT(*) ... HAVING COUNT(*)>1`)
15. `WHERE FLOOR(yosh/10) = 2` (20-29 yosh) — indeks ishlamaydi! Xuddi shu mantiqni `WHERE yosh BETWEEN 20 AND 29` deb yozing — endi tez. Ikkala vaqtni o'lchang
16. ORDER BY ham indeksdan foydalanadi: `ORDER BY ism LIMIT 10` — indeksli va indekssiz vaqtini solishtiring (`idx_ism`'ni DROP/CREATE qilib)
17. `COUNT(*)` katta jadvalda: `SELECT COUNT(*) FROM foydalanuvchilar WHERE shahar='Toshkent';` — indeksli va indekssiz farqi?
18. O'ylang: `jins ENUM('erkak', 'ayol')` ustuniga indeks foyda beradimi? (kam — million qatorning yarmi baribir o'qiladi; bu "selektivlik" tushunchasi)
19. (dokon) Telefon bo'yicha mijoz qidirish real ssenariy: `mijozlar.telefon` da UNIQUE indeks borligini tekshiring — login/qidiruv ustunlari doim indeksli bo'lsin
20. Xulosa yozing: qaysi 3 turdagi ustunga DOIM indeks kerak? (FK ustunlari; WHERE'da tez-tez ishlatiladiganlar; UNIQUE bo'lishi shart bo'lganlar)

22 — VIEW, Stored Procedure, Trigger, Event

◀ Oldingi: 21 — Indekslar · 🏠 README · Keyingi: 23 — EXPLAIN va optimizatsiya ▶

Bu bobda: bazaning "avtomatika qurollari"ni o'rganamiz: VIEW bilan uzun query'ga nom qo'yib, uni oddiy jadvaldek so'rashni; Stored Procedure bilan bir necha buyruqni bitta `CALL` ga jamlashni; Trigger bilan jadvaldagi har bir o'zgarishga bazaning o'zi avtomatik javob berishini va Event — bazaning ichki "budilniki" bilan vazifalarni belgilangan vaqtda o'zi bajarishini ko'rib chiqamiz.

VIEW — saqlangan query

Har safar 4 jadvalli JOIN yozish charchatadi. Bir marta yozib, NOM bering:

```
USE kutubxona;

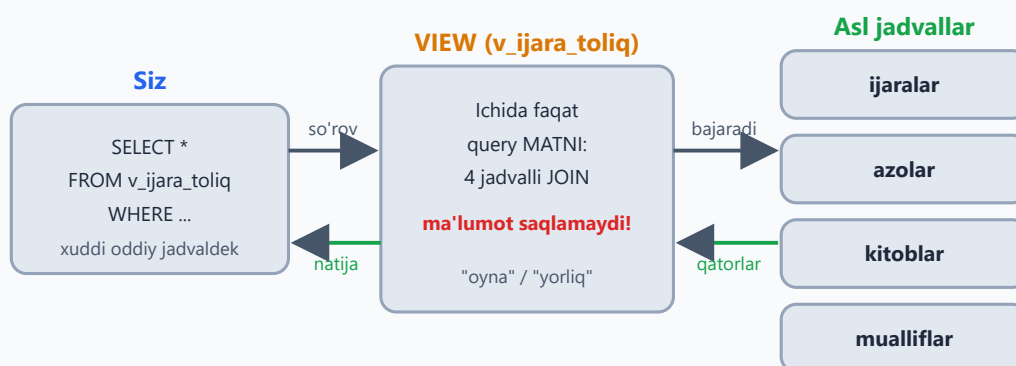
CREATE VIEW v_ijara_toliq AS
SELECT i.id, a.ism AS azo, k.nomi AS kitob, m.ism AS muallif,
       i.olingan_sana, i.qaytarilgan_sana
FROM   ijaralar i
JOIN   azolar a ON a.id = i.azo_id
JOIN   kitoblar k ON k.id = i.kitob_id
JOIN   mualliflar m ON m.id = k.muallif_id;

-- Endi oddiy jadvaldek:
SELECT * FROM v_ijara_toliq WHERE qaytarilgan_sana IS NULL;
```

View ma'lumot SAQLAMAYDI — har murojaatda asl query qayta bajariladi. Bu "yorliq/shortcut": asl jadvallarga yangi qator qo'shilsa, view'da ham darhol ko'rinadi, alohida "yangilash" kerak emas. Shu sababli view tezlik QO'SHMAYDI — qulaylik beradi, xolos: sekin JOIN'ni view qilib qo'ysangiz, u view ichida ham sekinligicha qoladi (davosi — 21-bobdagi indekslar).

✳ Ba'zi bazalarda (PostgreSQL, Oracle) natijani diskka saqlab qo'yadigan **materialized view** ham bor — MySQL'da bunday tur yo'q, view har doim "jonli".

VIEW — saqlangan query'ga qo'yilgan nom



Har murojaatda asl query QAYTA bajariladi — view nusxa emas

Jadvallarga yangi qator qo'shilsa, view'da darhol ko'rinadi

💡 View'ning yana bir foydasi — **xavfsizlik**: hamkasbingizga jadvalning o'zini emas, faqat kerakli ustunlarni ko'rsatadigan view'ni ochib berasiz (masalan, xodimlar jadvalini maosh ustunisiz). Ruxsatlarni 24-bobda GRANT bilan amalda ko'ramiz.

View bilan ishlashning qolgan buyruqlari (masalalarda kerak bo'ladi):

```
CREATE OR REPLACE VIEW v_ijara_toliq AS ...; -- bor bo'lsa, yangisi bilan almashtiradi
SHOW FULL TABLES WHERE Table_type = 'VIEW'; -- bazadagi view'lar ro'yxati
DROP VIEW v_ijara_toliq; -- view o'chadi, asl jadvallarga tegmaydi!
```

✳️ View'ning bitta cheklovi bor: u parametr qabul qilmaydi — `v_ijara_toliq(5)` deb chaqirib bo'lmaydi. "Parametrli saqlangan kod" kerak bo'lsa, navbatdagi mavzu aynan shu.

Stored Procedure — bazada saqlangan "funksiya"

Dasturlash tillaridagi funktsiyani eslang: bir necha amalni bitta nomga jamlab, kerak bo'lganda chaqirasiz. Stored procedure — xuddi shu, faqat bazaning o'zida yashaydi:

```
DELIMITER //
CREATE PROCEDURE kitob_berish(IN p_kitob_id INT, IN p_azo_id INT)
```

```

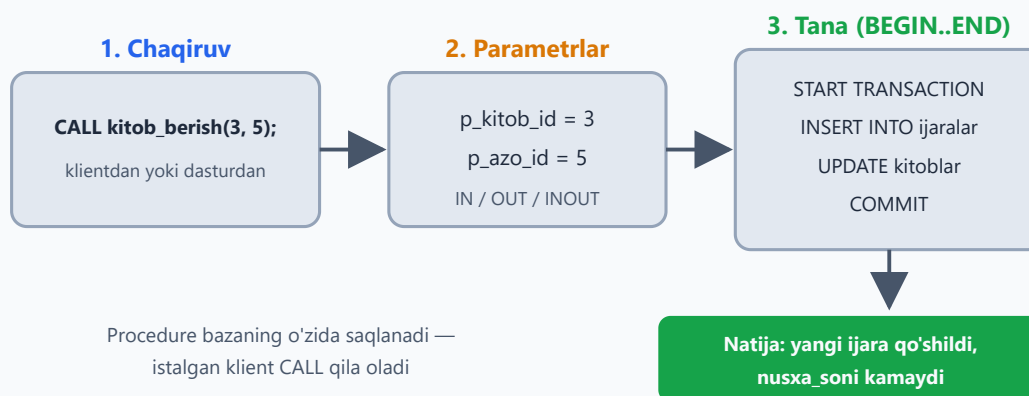
BEGIN
  START TRANSACTION;
  INSERT INTO ijaralar (kitob_id, azo_id, olingan_sana)
  VALUES (p_kitob_id, p_azo_id, CURDATE());
  UPDATE kitoblar SET nusxa_soni = nusxa_soni - 1
  WHERE id = p_kitob_id AND nusxa_soni > 0;
  COMMIT;
END //
DELIMITER ;

CALL kitob_berish(3, 5);

```

DELIMITER // — vaqtincha "buyruq tugashi" belgisini o'zgartiramiz, chunki procedure ichida ; ko'p: usiz klient birinchi ; ni ko'riboq "buyruq tugadi" deb o'ylaydi. Aytgancha, DELIMITER — SQL buyrug'i emas, mysql klienti va Workbench'ning o'z buyrug'i; ba'zi GUI vositalarida u shart ham emas.

Stored procedure: CALL'dan natijagacha



Bitta CALL = bir nechta SQL buyruq + tranzaksiya bitta joyda

Parametrlar 3 xil bo'ladi:

Turi	Ma'nosi	Misol
IN	qiymat ichkariga kiradi (eng ko'p ishlatiladi)	CALL kitob_berish(3, 5);
OUT	procedure natijani shu o'zgaruvchiga yozib qaytaradi	CALL kitob_soni('roman', @soni);
INOUT	ham kiradi, ham qaytadi	kamdan-kam kerak bo'ladi

OUT ga kichik misol:

```
DELIMITER //
CREATE PROCEDURE kitob_soni(IN p_janr VARCHAR(50), OUT p_soni INT)
BEGIN
    SELECT COUNT(*) INTO p_soni FROM kitoblar WHERE janr = p_janr;
END //
DELIMITER ;

CALL kitob_soni('roman', @soni);
SELECT @soni; -- 4 - bazamizda janri 'roman' bo'lgan kitoblar soni
```

@soni — sessiya o'zgaruvchisi (19-bobdagi @buyurtma ni eslang): procedure tugagach ham qiymat unda saqlanib turadi, keyingi query'da bimalol ishlataverasiz.

⚠ Yuqoridagi kitob_berish da bitta nuqson bor: nusxa qolmagan bo'lsa (nusxa_soni = 0), UPDATE hech narsani o'zgartirmaydi, lekin INSERT baribir o'tib ketadi — "yo'q kitob" berilgan bo'lib qoladi. To'g'ri yo'l — avval tekshirish:

```
DELIMITER //
CREATE PROCEDURE kitob_berish_xavfsiz(IN p_kitob_id INT, IN p_azo_id INT)
BEGIN
    DECLARE v_soni INT;

    START TRANSACTION;
    SELECT nusxa_soni INTO v_soni
    FROM kitoblar WHERE id = p_kitob_id FOR UPDATE; -- 19-bobdagi qulf!

    IF v_soni > 0 THEN
        INSERT INTO ijaralar (kitob_id, azo_id, olingan_sana)
        VALUES (p_kitob_id, p_azo_id, CURDATE());
        UPDATE kitoblar SET nusxa_soni = nusxa_soni - 1 WHERE id = p_kitob_id;
        COMMIT;
    ELSE
        ROLLBACK;
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Kitob nusxasi qolmagan!';
    END IF;
END //
DELIMITER ;
```

DECLARE — procedure ichida lokal o'zgaruvchi e'lon qiladi (faqat BEGIN dan keyin, eng boshida turishi shart), SELECT ... INTO unga qiymat yozadi. SIGNAL esa

o'zimizning xato xabarimizni "otadi" — CALL qilgan tomon uni oddiy MySQL xatosi kabi ko'radi va tranzaksiya bekor bo'lganini darrov biladi. Yana bir nozik joy: `p_kitob_id` bazada umuman yo'q bo'lsa, `SELECT ... INTO` hech narsa topmaydi va `v_soni` `NULL` bo'lib qoladi — `NULL > 0` rost emas, shuning uchun bu holat ham xavfsiz tarzda `ELSE` tarmog'iga tushadi.

Aytgancha, MySQL'da `CREATE FUNCTION` ham bor: u bitta qiymat **qaytaradi** va `SELECT` ichida xuddi `NOW()` kabi ishlatiladi; procedure esa `CALL` bilan chaqiriladi va bir nechta amalni ketma-ket bajarishga mo'ljallangan. Procedure'larni boshqarish buyruqlari:

```
SHOW PROCEDURE STATUS WHERE Db = 'kutubxona'; -- bazadagi
procedure'lar ro'yxati
DROP PROCEDURE kitob_berish;                -- o'chirish
```

Trigger — avtomatik reaksiya

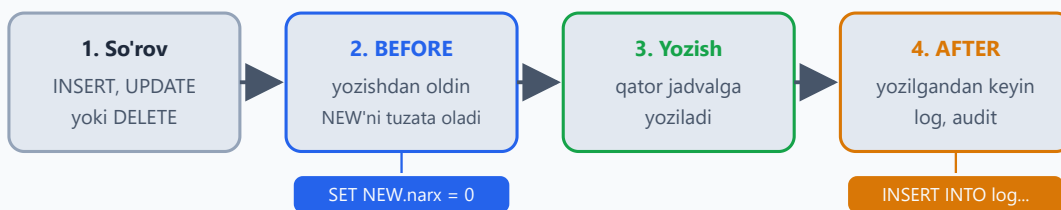
Trigger — jadvalga "soqchi" qo'yish: kimdir qator qo'shsa, o'zgartirsa yoki o'chirsa, siz oldindan yozib qo'ygan kod O'ZI ishga tushadi. Masalan, kitob nusxalari sonini kuzatib boramiz:

```
CREATE TABLE kitob_tarixi (
    id INT AUTO_INCREMENT PRIMARY KEY,
    kitob_id INT, eski_soni INT, yangi_soni INT, vaqt DATETIME
);

DELIMITER //
CREATE TRIGGER trg_nusxa_kuzatuv
AFTER UPDATE ON kitoblar
FOR EACH ROW
BEGIN
    IF OLD.nusxa_soni != NEW.nusxa_soni THEN
        INSERT INTO kitob_tarixi (kitob_id, eski_soni, yangi_soni,
        vaqt)
        VALUES (NEW.id, OLD.nusxa_soni, NEW.nusxa_soni, NOW());
    END IF;
END //
DELIMITER ;
```

Endi nusxa_soni har o'zgarganda tarix O'ZI yoziladi — hech kim "log yozishni unutib" qo'ymaydi. `OLD` — eski qiymat, `NEW` — yangi.

Trigger qachon ishga tushadi?



OLD va NEW kimda bor?

INSERT	OLD yo'q · NEW — kiritilayotgan qator
UPDATE	OLD — eski qiymat · NEW — yangi qiymat
DELETE	OLD — o'chirilayotgan qator · NEW yo'q

Trigger'ni hech kim chaqirmaydi — hodisaning o'zi ishga tushiradi

2 vaqt (BEFORE/AFTER) × 3 hodisa (INSERT/UPDATE/DELETE) = jami 6 xil trigger

Trigger jami 6 xil bo'ladi: 2 vaqt (BEFORE / AFTER) × 3 hodisa

(INSERT / UPDATE / DELETE). BEFORE — qator yozilishidan OLDIN ishga tushadi va NEW ni o'zgartira oladi (17-masala shunga asoslangan: SET NEW.narx = 0). AFTER — qator yozilgandan KEYIN, log/audit uchun qulay. OLD va NEW esa hodisaga qarab bor yoki yo'q:

Hodisa	OLD	NEW
INSERT	yo'q	kiritilayotgan qator
UPDATE	eski qiymat	yangi qiymat
DELETE	o'chirilayotgan qator	yo'q

Trigger'larni ko'rish va o'chirish:

```
SHOW TRIGGERS; -- bazadagi trigger'lar ro'yxati
DROP TRIGGER trg_nusxa_kuzatuv; -- o'chirish
```

⚠ Bitta muhim cheklov: trigger O'ZI osilgan jadvalni UPDATE / INSERT / DELETE qila olmaydi — kitoblar dagi trigger ichida UPDATE kitoblar ... yozsangiz, MySQL 1442-xato beradi (bu cheksiz aylanishning oldini oladi). O'z qatorini tuzatish kerak bo'lsa, BEFORE trigger'da SET NEW.ustun = ... ishlatiladi; BOSHQA jadvalga yozish esa bimalol — yuqoridagi misolda kitob_tarixi ga yozdik.

Ogohlantirish: trigger — "ko'rinmas sehr". Ko'payib ketsa, "bu qiymat qayerdan o'zgardi?!" deb soatlab qidirasiz. Audit/log uchun yaxshi, biznes mantiq uchun ehtiyot bo'ling.

Event — bazaning ichki budilniki

Event — MySQL'ning o'z "rejalashtiruvchisi": belgilangan vaqtda yoki har N daqiqa/soat/kunda query'ni o'zi bajaradi. Avval rejalashtiruvchi yoqilganini tekshirib olamiz:

```
SHOW VARIABLES LIKE 'event_scheduler'; -- MySQL 8 da odatda ON
SET GLOBAL event_scheduler = ON;       -- OFF bo'lsa yoqamiz (admin
huquq kerak)
```

```
CREATE EVENT ev_eski_tarix_tozalash
ON SCHEDULE EVERY 1 DAY
DO DELETE FROM kitob_tarixi WHERE vaqt < NOW() - INTERVAL 90 DAY;
```

Har kuni bir marta 90 kundan eski tarix yozuvlari o'chiriladi — qo'lingiz tegmasdan. Bir martalik ish uchun `EVERY` o'rniga `AT` yoziladi: `ON SCHEDULE AT NOW() + INTERVAL 1 HOUR` — bir soatdan keyin bir marta bajaradi-da, event o'z-o'zidan o'chib ketadi. Eventlarni ko'rish va o'chirish:

```
SHOW EVENTS;
DROP EVENT ev_eski_tarix_tozalash;
```

22-bob masalalari

1. (kutubxona) `v_ijara_toliq` view'ini yarating va ishlating
2. (kutubxona) `v_qarzdorlar` view: qaytarilmagan + 15 kundan oshgan ijaralar (a'zo ismi, kitob, necha kun)
3. (kutubxona) `v_kitob_statistika`: har kitob — nomi, muallifi, necha marta ijara qilingan
4. (dokon) `v_buyurtma_summalari`: buyurtma id, mijoz ismi, sana, jami summa

5. (dokon) `v_ombor_holati`: mahsulot, kategoriya, soni, holati (CASE: tugagan/oz/yetarli)
6. (klinika) `v_qabullar_toliq`: bemor + shifokor + sana + tashxis + tolov
7. (taksi) `v_haydovchi_statistika`: ism, safarlar soni, jami daromad, o'rtacha baho
8. View ustida view: `v_qarzdorlar` dan faqat 30+ kunliklarni oladigan `v_jiddiy_qarzdorlar` yarating
9. View'ni o'zgartirish: `CREATE OR REPLACE VIEW ...` bilan 5-masala view'iga narx ustunini qo'shing
10. `SHOW FULL TABLES WHERE Table_type = 'VIEW';` — bazadagi view'larni ko'ring; `DROP VIEW` bilan bittasini o'chiring
11. (kutubxona) `kitob_berish` procedure'ini yarating va 2 marta CALL qiling
12. (kutubxona) `kitob_qaytarish(p_ijara_id)` procedure yozing: sana qo'yadi + nusxa qaytaradi
13. (dokon) `mahsulot_sotish(p_mahsulot_id, p_soni)` procedure: ombordan ayiradi (yetarli bo'lsa! — bobdagi `kitob_berish_xavfsiz` namunasiga qarang)
14. (klinika) `qabul_yozish(p_bemor, p_shifokor)` procedure: NOW() sana, tolov = shifokor qabul_narxi (SELECT INTO o'zgaruvchi bilan: `SELECT qabul_narxi INTO @narx FROM ...`)
15. `SHOW PROCEDURE STATUS WHERE Db = 'kutubxona';` — procedure'laringizni ko'ring; `DROP PROCEDURE` sinang
16. (kutubxona) Yuqoridagi trigger'ni yarating, nusxa_soni'ni o'zgartiring, `kitob_tarixi` ni tekshiring — yozildimi?
17. (dokon) BEFORE INSERT trigger: mahsulot narxi 0 dan kichik kiritilsa, 0 ga to'g'rilasin (`SET NEW.narx = 0`)
18. (taksi) AFTER INSERT trigger safarlar'ga: yangi safar qo'shilganda haydovchining reytingini avtomatik qayta hisoblasin (`UPDATE haydovchilar SET reyting = (SELECT AVG(baho) FROM safarlar WHERE ...) WHERE id = NEW.haydovchi_id;` — `AVG NULL` baholarni o'zi tashlab ketadi)
19. Event yarating: har 1 MINUTE'da test jadvalga NOW() yozsin. 3 daqiqa kutib tekshiring, keyin DROP EVENT qiling
20. Fikrlang va yozing: katta loyihada biznes-mantiq qayerda turgani ma'qul — bazadami (procedure/trigger) yoki dastur kodidami? Har tarafga 2 tadan argument toping (javob: ko'pincha kodda — versiyalash, test, debug oson; baza darajasida — audit, ma'lumot butunligi)

23 — EXPLAIN va optimizatsiya

◀ Oldingi: 22 — VIEW, Stored Procedure, Trigger, Event · 🏠 README · Keyingi: 24 — Xavfsizlik va administratsiya ▶

Bu bobda: sekin query'ning sababini taxmin bilan emas, dalil bilan topishni o'rganamiz: EXPLAIN rejasidagi muhim ustunlarni (type, key, rows, Extra) o'qishni, type shkalasini (const'dan ALL'gacha), EXPLAIN ANALYZE bilan real vaqtlarni ko'rishni, optimizatsiyaning 5 qadamli siklini va eng ko'p uchraydigan 4 ta "kasallik"ni (ustunga funksiya, SELECT *, katta OFFSET, indeksiz JOIN) davolari bilan ko'rib chiqamiz.

Muammo: query ishlayapti, lekin sekin. Sababini qayerdan bilamiz?

21-bobda indeks qidiruvni yuzlab baravar tezlashtirishini o'z ko'zimiz bilan ko'rdik. Lekin real loyihada savol boshqacha turadi: o'nlab query bor — qaysi biri sekin va NEGA sekin?

"Menimcha, indeks yetishmayapti", "balki JOIN aybdor" — bular taxmin. Yaxshi shifokor "qorningiz og'riyaptimi — demak, oshqozon" demaydi, rentgenga yuboradi. MySQL'da ham ana shunday rentgen bor va u bepul: `EXPLAIN`.

EXPLAIN — query'ning rentgeni

Query OLDIGA `EXPLAIN` so'zini qo'ying — MySQL "men buni QANDAY bajarmoqchiman" degan rejasini ko'rsatadi. Muhim jihati: query'ning o'zi bajarilmAYDI, faqat reja chiqadi — million qatorli jadvalda ham bir zumda:

```
USE katta;
EXPLAIN SELECT * FROM foydalanuvchilar WHERE yosh = 25;
```

yosh da indeks bo'lmasa, natija taxminan bunday (muhim ustunlarnigina qoldirdik; 21-bobdan `idx_yosh` qolgan bo'lsa, "kasal" holatni ko'rish uchun avval `DROP INDEX idx_yosh ON foydalanuvchilar;` qiling):

table	type	possible_keys	key	rows	Extra
foydalanuvchilar	ALL	NULL	NULL	≈996000	Using where

O'qilishi: "mos indeks topmadim (`key = NULL`), shuning uchun jadvalni boshidan oxirigacha o'qiyman (`type = ALL`), taxminan 996 ming qatorni tekshiraman". Bitta savol uchun million qator — mana sizga sekinlikning hujjatlashtirilgan sababi, taxminsiz.

Indeks qo'yib, rentgenni takrorlaymiz:

```
CREATE INDEX idx_yosh ON foydalanuvchilar(yosh);
EXPLAIN SELECT * FROM foydalanuvchilar WHERE yosh = 25;
```

table	type	possible_keys	key	rows	Extra
foydalanuvchilar	ref	idx_yosh	idx_yosh	≈20000	Using index

Reja butunlay o'zgardi: `type=ref` (indeks bo'yicha tenglik), `key=idx_yosh` (indeks REAL ishladi), `rows` million emas — 20 ming atrofida (faqat 25 yoshlilar).

Asosiy ustunlar:

Ustun	Nima	Nimaga qarash kerak
<code>type</code>	qidiruv usuli	<code>const / ref / range</code> — yaxshi; <code>ALL</code> — full scan, YOMON
<code>possible_keys</code>	nazariy mos keladigan indekslar	bo'sh bo'lsa — mos indeksning o'zi yo'q
<code>key</code>	real ishlatilgan indeks	<code>NULL</code> = indeksiz ishlayapti
<code>rows</code>	taxminan nechta qator o'qiladi	kam = yaxshi
Extra	qo'shimcha belgilar	<code>Using index</code> — zo'r; <code>Using filesort</code> — saralash indeksiz; <code>Using temporary</code> —

Ustun	Nima	Nimaga qarash kerak
		vaqtinchalik jadval kerak bo'ldi



✳️ `rows` — aniq son emas, MySQL statistikasiga asoslangan **baho** (shuning uchun sizda biroz boshqacha raqam chiqishi mumkin). Lekin "20 ming yoki million" farqini ko'rsatishga bu bemaolol yetadi.

💡 `Using filesort` nomiga aldanmang: bu "diskdagi fayl bilan ishladi" degani emas, "saralashni indeksiz, alohida bosqichda qildim" degani. O'nta qator uchun bu arzon ish, million qator uchun — qimmat.

type — eng muhim ustun: shkala

`type` MySQL qancha mehnat qilishini bir so'zda aytadi. Qiymatlarini yaxshidan yomonga tersak, shkala hosil bo'ladi:

type	Ma'nosi	Qachon chiqadi
const	bitta qator, darrov topiladi	PK yoki UNIQUE bo'yicha: <code>WHERE id = 500</code>
eq_ref	JOIN'da har qator uchun PK/UNIQUE orqali bittadan	FK → PK ulanishlari
ref	indeks bo'yicha tenglik	<code>WHERE ism = 'User500'</code> (idx_ism bilan)

type	Ma'nosi	Qachon chiqadi
range	indeksning bir oralig'i	BETWEEN , > , < , LIKE 'User5%'
index	BUTUN indeksni o'qish	full scan'dan yengilroq, lekin baribir og'ir
ALL	butun jadvalni o'qish — full scan	mos indeks yo'q. Qizil bayroq!



💡 Maqsad — har query'dan `const` undirish emas. Katta jadvalda `ref` yoki `range` ko'rsangiz — bu yaxshi natija. Xavotir `ALL` dan boshlanadi. Kichik jadvalda esa `ALL` ham normal: 50 qatorni to'g'ridan-to'g'ri o'qish indeksga borib kelishdan arzon, MySQL buni o'zi hisoblab tanlaydi.

EXPLAIN ANALYZE — reja emas, fakt

Oddiy EXPLAIN — "rejam shunday". `EXPLAIN ANALYZE` (MySQL 8.0.18+) esa query'ni **haqiqatdan bajarib**, har bosqichning REAL vaqti va REAL qator sonini ko'rsatadi:

```
EXPLAIN ANALYZE SELECT * FROM foydalanuvchilar WHERE yosh = 25;
```

Indeks qo'yishdan oldingi holatda natija taxminan bunday (qisqartirilgan):

```
-> Filter: (foydalanuvchilar.yosh = 25) (cost=... rows=99633)
      (actual time=0.07..0.350 rows=20102 loops=1)
```

```
-> Table scan on foydalanuvchilar (cost=... rows=996357)
(actual time=0.06..290 rows=1000000 loops=1)
```

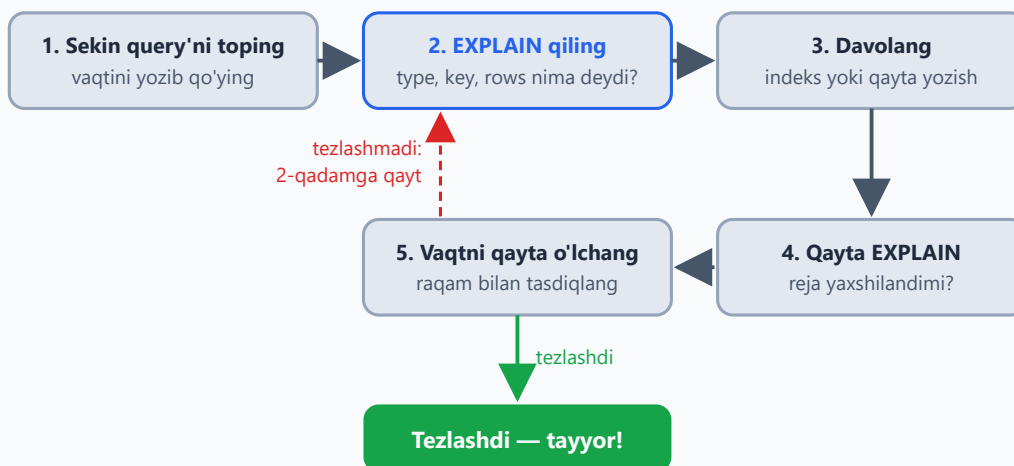
O'qilishi: `actual time=0.07..350` — birinchi qator 0.07 ms'da, oxirgisi 350 ms'da kelgan; `rows=20102` — real topilgan qatorlar. Reja `rows=99633` degan edi — baho besh baravar adashibdi, bu normal: reja — bashorat, ANALYZE — fakt.

⚠ Esda tuting: ANALYZE query'ni chindan ishga tushiradi. SELECT bilan bemalol, lekin yarim soat ishlaydigan og'ir query'ga ANALYZE qo'ysangiz — yarim soat kutasiz. Va eng muhimi: `EXPLAIN ANALYZE` ni UPDATE yoki DELETE'ga qo'llasangiz, ma'lumot CHINDAN o'zgaradi — oddiy `EXPLAIN` esa hech narsani bajarmaydi, xavfsiz.

Optimizatsiya sikli (har doim shu ketma-ketlik!)

1. **Sekin query'ni toping** va hozirgi vaqtini yozib qo'ying (real loyihada buni slow query log topib beradi — 14-masalada sinaysiz)
2. **EXPLAIN qiling** — type, key, rows nima deydi?
3. **Davolang**: `type=ALL` yoki katta `rows` ko'rsangiz — WHERE/JOIN ustunlariga indeks qo'ying yoki query'ni qayta yozing
4. **Qayta EXPLAIN** — reja yaxshilandimi?
5. **Vaqtni qayta o'lchang** — raqam bilan tasdiqlang. Tezlashmagan bo'lsa — 2-qadamga qayting

Optimizatsiya sikli — 5 qadam



5-qadamni tashlamang: foydalanuvchiga reja emas, soniya muhim.

✦ 5-qadamni tashlab ketmang! "EXPLAIN chiroyli bo'ldi" — hali natija emas: foydalanuvchiga reja emas, soniya muhim.

Tipik kasalliklar va davolari

KASALLIK 1: ustunga funksiya. Indeks `ism` qiymatlari bo'yicha saralangan — `UPPER(ism)` natijalari bo'yicha emas. Shartda ustunga funksiya qo'ysangiz, MySQL uni har qatorda hisoblashga majbur — indeks o'chadi:

```
EXPLAIN SELECT * FROM foydalanuvchilar WHERE UPPER(ism) = 'USER5'; --
type: ALL
-- DAVO: shartni "toza" ustunga yozing:
EXPLAIN SELECT * FROM foydalanuvchilar WHERE ism = 'User5'; --
type: ref
```

(Aytgancha, MySQL'ning standart sozlamasida matn taqqoslash katta-kichik harfni baribir farqlamaydi — `UPPER` bu yerda umuman ortiqcha edi.) Sanalarda ham xuddi shu hikoya: `YEAR(sana) = 2026` o'rniga `sana >= '2026-01-01' AND sana < '2027-01-01'` yozing (21-bobdan tanish).

KASALLIK 2: SELECT * (keraksiz ustunlar). Faqat kerakli ustunlarni so'rang. Bonus: so'ralgan hamma ustun indeksning o'zida bo'lsa, MySQL jadvalga umuman tegmaydi — Extra'da `Using index` chiqadi (bu **covering index** deyiladi):

```
EXPLAIN SELECT ism FROM foydalanuvchilar WHERE ism LIKE 'User1%'; --
Extra: Using where; Using index
EXPLAIN SELECT * FROM foydalanuvchilar WHERE ism LIKE 'User1%'; --
jadvalga ham boradi
```

KASALLIK 3: katta OFFSET. `OFFSET` "sakrab o'tish" emas — MySQL baribir hamma qatorni o'qib, keraksizlarini tashlab yuboradi:

```
SELECT * FROM foydalanuvchilar ORDER BY id LIMIT 10 OFFSET 900000;
-- 900 010 qator o'qiladi, 900 000 tasi axlatga!

-- DAVO (keyset pagination): "oxirgi ko'rgan id'dan keyingi 10 tasini ber":
SELECT * FROM foydalanuvchilar WHERE id > 900000 ORDER BY id LIMIT 10;
-- darrov!
```

"Keyingi sahifa" tugmasi bosilganda oxirgi qatorning id'sini eslab qolib shartga qo'yasiz — PK bo'yicha range qidiruv minginchi sahifada ham birinchisidek tez ishlaydi.

KASALLIK 4: indekssez JOIN. JOIN'da MySQL birinchi jadvalning HAR qatori uchun ikkinchisidan mosini qidiradi. Ulanish ustuni indekssez bo'lsa, eski MySQL'larda har bir qidiruv full scan'ga aylanardi: $\text{ming} \times \text{million} = \text{falokat}$. MySQL 8 esa bunday holatda **hash join** ishlatib falokatni ancha yumshatadi, lekin indeksli ulanish baribir odatda tezroq va xotirani tejaydi. DAVO: `ON` dagi ustunlar (ayniqsa FK) indeksli bo'lsin. FOREIGN KEY constraint qo'ygan bo'lsangiz, indeks avtomatik bor (18-bob); qo'ymagan bo'lsangiz — o'zingiz qo'ying.

23-bob masalalari

katta bazasida (1 mln qator) ishlang — kichik jadvalda EXPLAIN doim "hammasi yaxshi" deyveradi, chunki 50 qatorga indeksning o'zi shart emas. Boshlashdan oldin `SHOW INDEX FROM foydalanuvchilar;` bilan **hamma** indekslarni ko'rib chiqing: 21-bobdagi `idx_ism` va bob matnida yaratgan `idx_yosh` turgan bo'lsin; qolgan "meros" indekslarni esa o'chirib tashlang — xususan 21-bob matnidagi `idx_shahar_yosh` kompozitini: `DROP INDEX idx_shahar_yosh ON foydalanuvchilar;` U turaversa, 10/13/15/16-masalalar natijasi "oldindan berilgan" bo'lib chiqadi va tajribalardan hech narsa o'rganmaysiz.

1. `EXPLAIN SELECT * FROM foydalanuvchilar WHERE id = 500;` — type nima? (const — eng zo'r, PK bo'yicha)
2. `EXPLAIN ... WHERE ism = 'User500';` — `idx_ism` bor bo'lsa `type=ref`, key ustunida `idx_ism` ko'rinadi. Indeksni DROP qilib qayta EXPLAIN — `type=ALL`! Farqni yozib oling (oxirida indeksni qaytarib qo'ying)
3. `EXPLAIN ... WHERE yosh BETWEEN 25 AND 30;` — indeksli holatda `type=range` bo'lishi kerak
4. `EXPLAIN ... WHERE UPPER(ism) = 'USER500';` — indeks bor bo'lsa ham `type=ALL`. Funksiyasiz versiya bilan solishtiring
5. `EXPLAIN ... WHERE ism LIKE 'User5%';` vs `LIKE '%500';` — type farqini ko'ring (range vs ALL — sababi 21-bobdagi lug'at o'xshatishida)
6. rows ustuni: 3 xil query'da (PK bo'yicha, indeksli ustun bo'yicha, indekssez ustun bo'yicha) rows qiymatlarini solishtirib jadval tuzing
7. `EXPLAIN ANALYZE` bilan 2-masalani takrorlang — actual time'ni o'qing (indeksli va indekssez farqi necha baravar?)

8. OFFSET kasalligi: `LIMIT 10 OFFSET 900000` vaqtini o'lchang, keyset versiyasi (`WHERE id > 900000`) bilan solishtiring
9. `EXPLAIN SELECT ism FROM foydalanuvchilar WHERE ism LIKE 'User1%';` — Extra'da `Using index` bormi? (faqat indeksdagi ustun so'raldi — jadvalga tegilmadi!) `SELECT *` bilan solishtiring
10. filesort: avval 21-bobdan `idx_shahar_yosh` qolgan bo'lsa, o'chiring: `DROP INDEX idx_shahar_yosh ON foydalanuvchilar;` — kompozit chap prefiksi (`shahar`) orqali saralashga ham xizmat qiladi va filesort umuman chiqmaydi. Endi `EXPLAIN SELECT * FROM foydalanuvchilar ORDER BY shahar LIMIT 10;` — Extra'da `Using filesort`. `shahar` ga indeks qo'yib qayta EXPLAIN — filesort ketdimi?
11. (dokon) Hamma asosiy query'laringizdan 3 tasini EXPLAIN qiling — type'lar qanday?
12. (kutubxona) 12-bobdagi 4 jadvalli JOIN'ni EXPLAIN qiling — natijada har jadval alohida qatorda chiqadi; har birida key ustunini tekshiring (FK indeksleri ishlayaptimi?)
13. Sekin query "yasang": `SELECT * FROM foydalanuvchilar WHERE shahar='Toshkent' OR yosh=25;` — EXPLAIN qiling (ikkala ustun ham indeksli bo'lsa, type'da `index_merge` chiqishi mumkin — MySQL ikki indeksni o'zi birlashtirgani). Keyin uni UNION bilan ikkita indeksli query'ga bo'lib yozing (UNION ALL emas: ikkala shartga birdek mos qatorlar ikki marta chiqib qoladi) — tezroqmi?
14. Slow log yoqing: `SET GLOBAL slow_query_log=ON; SET GLOBAL long_query_time=0.5;` (GLOBAL faqat yangi ulanishlarga ta'sir qiladi — joriy oynada `SET SESSION long_query_time=0.5;` ham bajaring). Sekin query bajarib, log fayl'ni toping (`SHOW VARIABLES LIKE 'slow_query_log_file';`)
15. COUNT solishtiruvi: `COUNT(*)` butun jadvalda vs `WHERE shahar='Nukus'` bilan (indeksli) — vaqtlar?
16. 2 ustunli saralash: `ORDER BY shahar, yosh LIMIT 10` — (`shahar`) indeksli yetadimi yoki (`shahar, yosh`) kompoziti kerakmi? Tajriba toza chiqishi uchun avval `shahar` ga aloqador HAMMA indeksni o'chiring: 10-masalada yaratgan `shahar` indeksini (nomini `SHOW INDEX` da ko'rib DROP qiling) va, hali turgan bo'lsa, 21-bobdan qolgan `idx_shahar_yosh` ni ham — ikkalasi birga tursa, EXPLAIN qaysi birini tanlagani noma'lum bo'lib, isbot buziladi. So'ng FAQAT (`shahar`) indeksini yaratib EXPLAIN qiling, keyin uni DROP qilib (`shahar, yosh`) kompozitini yarating va qayta EXPLAIN — filesort bilan isbotlang
17. JOIN tartibi: 5 qatorli kichik jadval yarating (masalan, `shaharlar(nomi, viloyat)`) va `foydalanuvchilar` bilan shahar bo'yicha JOIN qilib EXPLAIN qiling — MySQL qaysi jadvaldan boshlayapti? (Tartibni o'zi tanlaydi — odatda kichigidan)

18. O'zingiz "eng yomon query" yozing (ustunga funksiya + % bilan boshlanadigan LIKE + indeksiz ustun bo'yicha ORDER BY) va EXPLAIN'da hamma "qizil bayroq"larni ko'rsating
19. Keyin o'sha query'ni bosqichma-bosqich davolang — har qadamda EXPLAIN va vaqt o'zgarishini jadvalga yozing (bu portfolio'bop mashq!)
20. Xulosa konspekt yozing: "Sekin query ko'rsam, 5 qadamim: ..." — o'z so'zingiz bilan

24 — Xavfsizlik va administratsiya

← Oldingi: 23 — EXPLAIN va optimizatsiya · 🏠 README · Keyingi: 25 — Yakuniy loyihalar →

Bu bobda: bazani himoya qilishni o'rganamiz: SQL injection hujumi qanday ishlashini va prepared statement bilan undan qutulishni, har ilovaga alohida user yaratib minimal huquq berishni (CREATE USER, GRANT, REVOKE), mysqldump bilan backup olish va tiklashni, oxirida server holatini ko'rsatadigan diagnostika buyruqlarini ko'rib chiqamiz.

SQL Injection — №1 xavf

Shu paytgacha query'larni o'zimiz yozdik. Real ilovada esa query ko'pincha **foydalanuvchi kiritgan qiymat** asosida tuziladi: login formasi, qidiruv maydoni, URL'dagi id. Aynan shu yerda web tarixidagi eng mashhur hujum yotadi — **SQL injection**: foydalanuvchi ma'lumot o'rniga SQL KOD yuboradi va sizning query'ingiz uning query'siga aylanib qoladi.

Dastur kodida (PHP misolida) query'ni matn yopishtirish bilan yasash:

```
// XAVFLI KOD:  
$query = "SELECT * FROM userlar WHERE login = '" . $_POST['login'] .  
        '";
```

Foydalanuvchi login o'rniga yozadi: ' OR '1'='1'. Query bo'ladi:

```
SELECT * FROM userlar WHERE login = '' OR '1'='1'  
-- '1'='1' doim TRUE → HAMMA userlar qaytadi → parolsiz kirish!
```

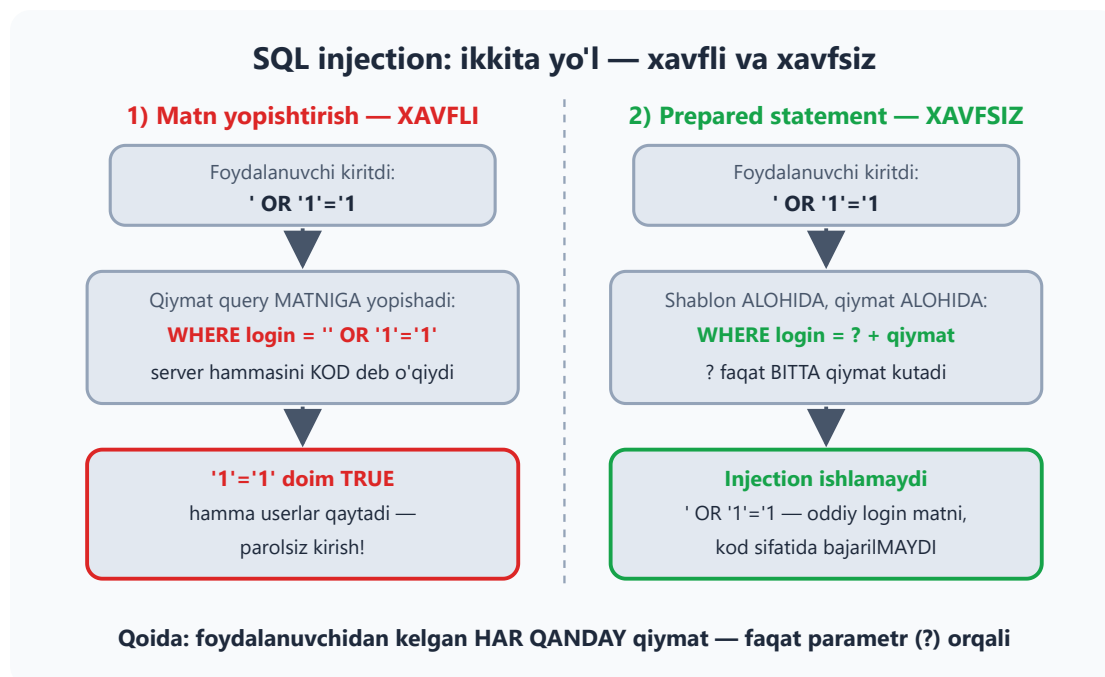
Yoki battarroq'i: '; DROP TABLE userlar; -- → jadval yo'qoladi. Oxiridagi -- — komment belgisi (MySQL'da undan keyin bo'sh joy bo'lishi shart): query'ning qolgan qismini "o'chirib" yuboradi, shunda payload sintaksis xatosiz o'tadi. Muammoning ildizi shunda: server uchun yopishtirilgan satr — bitta butun query. U qayeri sizning kodingiz-u qayeri foydalanuvchi qiymati ekanini ajrata olmaydi.

✦ Adolat uchun: mysqli kabi ba'zi drayverlar (PHP'da `mysqli->query()`, Python'da `mysql-connector` standart rejimda) bitta so'rovda IKKITA statement bajarishga ruxsat bermaydi, shuning uchun `DROP TABLE` payload har doim ham o'tavermaydi. Lekin hammasi bunday emas: masalan, PHP PDO standart sozlamada ko'p statement'li satrni bimalol bajaradi — aynan shu sabab PDO bilan yozilgan zaif kod bunday hujumga ochiq. Xulosa: drayverga suyanmang — `OR '1'='1'` kabi injection umuman BITTA statement ichida ishlaydi va hech qanday to'siqqa uchramaydi.

Yechim — parametrli query (prepared statement): qiymat query MATNI bilan emas, ALOHIDA yuboriladi:

```
// XAVFSIZ:  
$stmt = $pdo->prepare("SELECT * FROM userlar WHERE login = ?");  
$stmt->execute([$POST['login']]);
```

Server avval shablonni oladi va tushunib qo'yadi: "login ustunini BITTA qiymat bilan solishtiraman". Keyin kelgan qiymat nima bo'lishidan qat'i nazar — faqat qiymat. Endi `' OR '1'='1'` — shunchaki g'alati login MATNI bo'lib qoladi, kod sifatida bajarilmaydi. **Qoida: foydalanuvchidan kelgan har qanday qiymat — faqat parametr orqali. Hech qanday istisno.**



✦ Bu PHP'ga xos muammo emas: Python, Java, JavaScript, C# — hammasida prepared statement bor (nomi har xil: parameterized query, placeholder). Qaysi tilda yozmang, qoida o'sha-o'sha.

Userlar va huquqlar

Mehmonxonada farroshga hamma eshikni ochadigan universal kalit berilmaydi — har xodimga faqat o'z eshiklarining kaliti. Bazada bu **eng kam huquq tamoyili** deyiladi: har userga ishi uchun yetarli minimal huquq, ortig'i — yo'q.

Shuning uchun dastur hech qachon `root` bilan ulanmasin! `root` — hamma narsaga qodir administrator; ilova kodida xato yo xakerlik bo'lsa, butun server xavf ostida qoladi. Har ilovaga — o'z useri, minimal huquq:

```
CREATE USER 'dokon_app'@'localhost' IDENTIFIED BY 'Kuchli_Parol_123!';

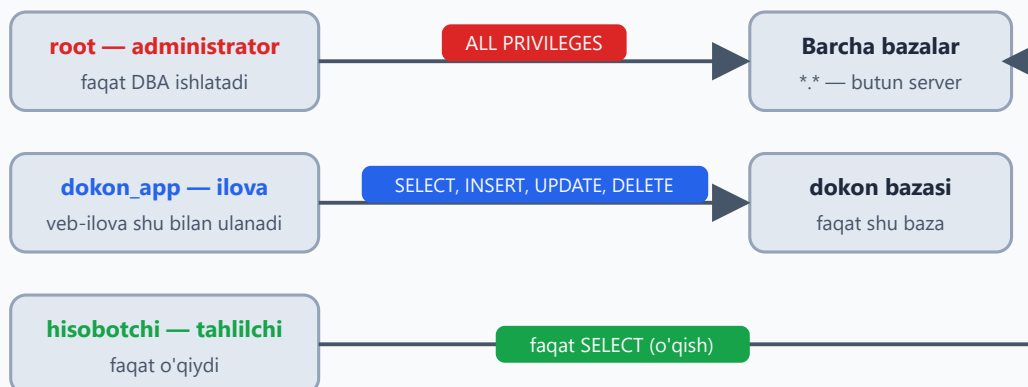
GRANT SELECT, INSERT, UPDATE, DELETE ON dokon.* TO
'dokon_app'@'localhost';
-- DROP, ALTER bermadik — ilova jadval o'chira olmaydi!

SHOW GRANTS FOR 'dokon_app'@'localhost';           -- huquqlarni
ko'rish
REVOKE DELETE ON dokon.* FROM 'dokon_app'@'localhost'; -- huquqni
qaytarib olish
ALTER USER 'dokon_app'@'localhost' IDENTIFIED BY 'Yangi_Parol_456!'; -
- parolni almashtirish
DROP USER 'dokon_app'@'localhost';                 -- userni
o'chirish
```

Eng ko'p beriladigan huquqlar:

Huquq	Nima beradi	Ilovaga kerakmi?
SELECT	o'qish	ha
INSERT, UPDATE, DELETE	qator qo'shish / yangilash / o'chirish	odatda ha
CREATE, ALTER, DROP	jadval STRUKTURASINI o'zgartirish	yo'q!
ALL PRIVILEGES	hammasi	faqat administratorga

Eng kam huquq tamoyili: kimga qancha ruxsat?



Har userga — ishi uchun yetarli MINIMAL huquq, ortig'i yo'q

Ilova hech qachon root bilan ulanmaydi: kod xatosi yoki xakerlik butun serverni xavf ostiga qo'ymasin

💡 `'dokon_app'@'localhost'` — bu nom + manzil juftligi. `localhost` — user faqat server turgan kompyuterning o'zidan ulana oladi; `'%'` — istalgan joydan (kerak bo'lmasa, ochmang!). MySQL uchun `'app'@'localhost'` va `'app'@'%'` — ikkita ALOHIDA user.

✚ Internetdagi eski qo'llanmalarda har `GRANT` 'dan keyin `FLUSH PRIVILEGES;` yozishadi — kerak emas: `CREATE USER`, `GRANT`, `REVOKE` o'zgarishni darhol qo'llaydi. `FLUSH PRIVILEGES` faqat huquq jadvalarini qo'lda (to'g'ridan-to'g'ri `INSERT/UPDATE` bilan) o'zgartirganda kerak bo'ladi — bunday ish esa sizga umuman kerak bo'lmaydi.

💡 Userlar ko'payganda MySQL 8 ning **ROLE** mexanizmi qo'l keladi: huquqlar to'plamiga nom berib, bir nechta userga birdek "kiydirasiz". `CREATE ROLE 'oquvchi'; GRANT SELECT ON dokon.* TO 'oquvchi'; GRANT 'oquvchi' TO 'ali'@'localhost';` — endi o'nlab userning huquqini bitta joydan boshqarasiz (rol kirishda avtomatik yoqilishi uchun yana `SET DEFAULT ROLE ALL TO 'ali'@'localhost';` deyiladi).

Backup — mysqldump

Disk kuyishi, xato `DELETE`, xakerlik — baza bir kunda yo'q bo'lishi mumkin. Yagona haqiqiy himoya — **backup** (zaxira nusxa). MySQL'da buning klassik quroli — `mysqldump`: bazani bitta `.sql` faylga "to'kib" beradi. Fayl ichi — oddiy matn: jadvallarni qayta yaratadigan `CREATE TABLE` va ma'lumotni qaytaradigan `INSERT` buyruqlari. Tiklash — shu faylni qaytadan bajarish, xolos.

```
# Backup (terminalda, mysql ichida emas!):
mysqldump -u root -p dokon > dokon_backup.sql

# Ishlayotgan serverda izchil nusxa (InnoDB uchun tavsiya):
mysqldump -u root -p --single-transaction dokon > dokon_backup.sql

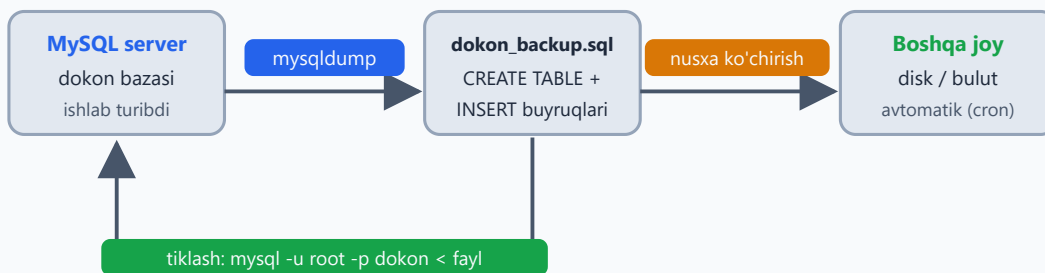
# Hamma bazalar:
mysqldump -u root -p --all-databases > hammasi.sql

# Tiklash:
mysql -u root -p dokon < dokon_backup.sql

# --all-databases faylini tiklash (baza nomi shart emas — fayl ichida
CREATE DATABASE va USE bor):
mysql -u root -p < hammasi.sql
```

✦ E'tibor bering: bitta baza dump qilinganda fayl ichida `CREATE DATABASE` BO'LMAYDI — tiklashdan oldin bo'sh bazani o'zingiz yaratasiz (`CREATE DATABASE dokon;`). `--databases` yoki `--all-databases` bilan olingan faylda esa bu buyruq bor, shuning uchun to'g'ridan-to'g'ri tiklayverasiz.

Backup va tiklash oqimi (mysqldump)



Temir qoida: tiklab KO'RILMAGAN backup — backup emas!

Nusxani server bilan BIR joyda qoldirmang — server kuysa, backup ham birga kuyadi

Temir qoida: tiklab KO'RILMAGAN backup — backup emas. Faylning borligi yetmaydi — tiklanishini tekshiring.

✦ Backupni baza turgan serverning o'zida qoldirmang: server kuysa, backup ham birga kuyadi. Nusxani boshqa diskka, boshqa serverga yoki bulutga ko'chiring — va buni qo'lda emas, avtomatik (cron) qiling.

Foydali diagnostika

Server "qornida" nima bo'layotganini ko'rish uchun bir nechta buyruq yetadi:

```
SHOW PROCESSLIST;           -- hozir kim nima qilyapti
KILL 42;                     -- osilib qolgan query'ni
to'xtatish (id - PROCESSLIST'dan)
SHOW STATUS LIKE 'Threads_connected'; -- hozir nechta ulanish bor
SHOW VARIABLES LIKE 'max_connections'; -- ruxsat etilgan maksimum
SELECT table_schema, ROUND(SUM(data_length+index_length)/1024/1024) AS
mb
FROM information_schema.tables GROUP BY table_schema; -- baza
hajmlari (MB)
```

`SHOW PROCESSLIST` — serverning "kim bor?" ro'yxati: har bir ulanish qaysi bazada, qancha vaqtdan beri qaysi query'ni bajarayotganini ko'rsatadi. Sayt sekinlashganda birinchi qaraladigan joy — shu. Uzun query matni ... bilan kesilib ko'rinsa, `SHOW FULL PROCESSLIST`; to'liq ko'rsatadi. Osilib qolgan query'ni `KILL <id>` bilan to'xtatasiz — buni 15-masalada o'zingiz sinab ko'rasiz. Nozik farq: `KILL 42` butun ulanishni uzadi, `KILL QUERY 42` esa ulanishni saqlab faqat hozirgi query'ni to'xtatadi.

24-bob masalalari

1. Qog'ozda: `login = ' OR '1'='1` qanday ishlashini qadam-baqadam yozing (query qanday ko'rinishga keladi?)
2. `'; DROP TABLE userlar; --` payloadini tahlil qiling: `-- SQL'da nima?` (komment — qolgan qismni o'chiradi; MySQL'da `--` dan keyin bo'sh joy bo'lishi shartligini ham eslang)
3. SQL darajasida prepared statement sinang: `PREPARE st FROM 'SELECT * FROM dokon.mijozlar WHERE id = ?'; SET @id = 1; EXECUTE st USING @id;`
4. 3-masalada `SET @id = '1 OR 1=1';` qilib `EXECUTE` qiling — injection ishladimi? (yo'q — butun matn bitta qiymat sifatida qaraldi!)
5. `dokon_app` userini yarating (SELECT, INSERT, UPDATE huquqlari bilan, DELETE'siz). Bobni o'qib borib allaqachon yaratgan bo'lsangiz, avval `DROP USER` qiling
6. Yangi terminalda `mysql -u dokon_app -p` bilan kiring, SELECT qiling (ishlaydi), DELETE qiling (xato!) — "DELETE command denied" xatosini o'z ko'zingiz bilan ko'ring

7. O'sha user bilan `kutubxona` ga kirib ko'ring — kira olmaysiz (huquq faqat dokon'ga edi)
8. Faqat O'QIY oladigan user yarating: `hisobotchi` — faqat SELECT, hamma bazaga (`*.*`)
9. `SHOW GRANTS` ni ikkala user uchun bajaring va farqini yozing
10. REVOKE bilan dokon_app'dan UPDATE'ni oling, tekshiring, qaytarib bering
11. `dokon` ni mysqldump bilan backup qiling — fayl hajmini ko'ring, ichini matn muharririda oching (oddiy SQL buyruqlar!)
12. Bazani DROP qiling (ha, jasorat!) va backup'dan to'liq tiklang: `mysql -u root -p < ...` (fayl ichida CREATE DATABASE bo'lmasa, avval yaratib `mysql ... dokon < fayl`). Hammasi joyidami — COUNT bilan tekshiring
13. 4 ta bazani bitta faylga backup qiling (`--databases kutubxona dokon klinika taksi`)
14. Cron g'oyasi: har kuni avtomatik backup buyrug'ini yozing (bajarish shart emas, buyruqni tuzing): sana nomli fayl + gzip
15. `SHOW PROCESSLIST;` — o'z ulanishingizni toping. 2-terminal oching, unda uzun query bajaring (`SELECT SLEEP(30);`), 1-terminaldan PROCESSLIST'da ko'rib, `KILL <id>;` bilan to'xtating!
16. Baza hajmlari query'sini bajaring — qaysi baza eng katta? (katta bazasi, albatta)
17. Parol siyosati: `IDENTIFIED BY '123'` bilan user yaratib ko'ring — MySQL 8 e'tiroz bildirishi mumkin (validate_password). Xabarni o'qing
18. O'ylang: nega ilova uchun DROP/ALTER huquqlari berilmaydi? Qanday xavf? (xakerlik yoki kod xatosi jadval strukturasi buzmasin)
19. O'ylang: backup fayl ni qayerda saqlash kerak — o'sha serverdami? (yo'q! server kuysa backup ham kuyadi — boshqa joyga nusxa)
20. Mini-audit: o'z 4 bazangiz bo'yicha "xavfsizlik cheklisti" yozing: userlar to'g'rimi, parollar kuchlimi, backup bormi, FK'lar joyidami

25 — Yakuniy loyihalar

← Oldingi: 24 — Xavfsizlik va administratsiya · 🏠 README

Bu bobda: kitob davomida o'rganganlarimizning hammasini jamlab, 3 ta to'liq tizimni noldan quramiz — kinoteatr, onlayn kurslar platformasi va yetkazib berish xizmati. Har birida yo'l bir xil: talablar → schema → CREATE → test ma'lumot → query'lar → hisobotlar. Oxirida kitobdan keyingi yo'l xaritasi va 25-bob masalalari — intervyuga tayyorgarlik uchun 20 ta klassik savol bor.

Bilim sinovdan o'tadi. 3 ta loyiha — har biri "noldan oxirigacha". Hech qayerdan ko'chirmang, o'zingiz yozing. Qiynalsangiz — tegishli bobga qayting: bu mag'lubiyat emas, aynan shunday o'rganiladi. Bu bobda ataylab tayyor kod yo'q — kitob davomida ko'chirib yozdingiz, endi o'zingiz yaratasisiz.

Loyihani qanday qurish kerak — 6 qadam

Uchala loyihada ham ish tartibi bir xil. Real ishda ham xuddi shunday:



Qiynalsangiz — tegishli bobga qayting. Real ishda ham yo'l xuddi shunday.

1. **Talablar.** Tizim nimani saqlashi va qanday savollarga javob berishi kerak? Bir varaq qog'ozga yozib chiqing.
2. **Schema — avval qog'ozda.** Jadvallar, ustunlar, bog'lanishlar (1:N, N:M) — chizing (3 va 20-boblar). Klaviaturaga shoshilmang: qog'ozdagi xato 1 daqiqada tuzatiladi, to'la bazadagi xato — bir soatda.

3. **CREATE.** Jadvallarni FK, CHECK, UNIQUE bilan yarating (4 va 18-boblar). Tartib muhim: avval "ota" jadvallar, keyin ularga FK bilan bog'lanadigan "bola"lar.
4. **Test ma'lumot.** INSERT bilan boy va xilma-xil data kiriting (6-bob). NULL'lar, bekor qilingan buyurtmalar, sotuvsiz filmlar ham bo'lsin — "ideal" data hech narsani sinamaydi.
5. **Query'lar.** Topshiriqlardagi savollarga javob yozing (7–16-boblar). GROUP BY'da SELECT'ga faqat guruhlangan ustunlar va aggregate'lar yozilishini unutmang — MySQL 8 buni qattiq tekshiradi (ONLY_FULL_GROUP_BY, 11-bob).
6. **Hisobot va jilo.** VIEW, PROCEDURE, TRIGGER, indeks va EXPLAIN (21–23-boblar) — loyihani "mahsulot" darajasiga ko'taring.

✦ Schema tavsiflarida ustun nomlari apostrofsiz yozilgan (o'rin emas, orin_raqami) — SQL identifikatorlarida apostrof ishlatilmaydi, jadval yaratganingizda siz ham shunday yozing.

Uchala loyiha bir qarashda — jadvallar soni va murakkablik A dan C tomon o'sib boradi:



LOYIHA A: Kinoteatr (oson-o'rta)

Tanish mavzu, 4 jadval — qizishish uchun ideal. Asosiy "tuz" — bitta o'rin ikki marta sotilmasligini bazaning o'zi kafolatlashi.

Schema (o'zingiz yarating): filmlar (nomi, janr, davomiyligi, yil), zallar (nomi, sigimi), seanslar (film, zal, vaqt, narx), chiptalar (seans, orin_raqami, mijoz_ism, sotilgan_vaqt).

Topshiriqlar:

1. 4 jadvalni FK'lar bilan yarating (chiptada seans+o'rin UNIQUE bo'lsin — bir o'rin ikki marta sotilmasin!)
2. 5 film, 3 zal, 10 seans, 25 chipta kiriting
3. Bugungi seanslar afishasi: film + zal + vaqt + narx
4. Har film nechta chipta sotgan (sotuvsizlar 0 bilan)
5. Har seansning to'liqlik foizi: $\text{chiptalar} / \text{zal sig'imi} * 100$
6. Eng daromadli film (chiptalar narxidan)
7. Birorta ham seansi yo'q filmlar (anti-join)
8. Chipta sotish PROCEDURE'i: o'rin bandligini tekshirib INSERT (band bo'lsa — hech narsa)
9. `v_afisha` view yarating
10. Har janr bo'yicha o'rtacha chipta narxi va jami daromad

💡 Maslahatlar: 4-topshiriq — LEFT JOIN + COUNT (12-bob); 5-da natijani ROUND bilan chiroyli qiling; 7 — anti-join qolipi (12-bob); 8-procedure ichida IF + EXISTS tekshiruvi bo'ladi (22-bob).

LOYIHA B: Onlayn kurslar platformasi (o'rta)

Endi 5 jadval va eng muhim yangi tushuncha — talabaning **progressi**: ikki xil hisobning (ko'rilgan darslar va jami darslar) nisbati.

Schema: kurslar (nomi, narx, daraja), talabalar, yozilishlar (talaba, kurs, sana, tolangan_summa — snapshot, 20-bob!), darslar (kurs, nomi, tartib_raqami), korishlar (talaba, dars, sana).

Topshiriqlar:

1. Schema + FK + CHECK'lar ($\text{narx} \geq 0$, $\text{tartib_raqami} > 0$)
2. Test data: 5 kurs, 10 talaba, 20 yozilish, 25 dars, 60+ ko'rish
3. Har kursning talabalari soni va jami tushumi
4. Har talabaning progressi: $\text{kursdagi ko'rgan darslari} / \text{jami darslar} * 100$ (eng qiyin query — JOIN + GROUP BY + subquery!)

5. Eng mashhur 3 kurs; birorta talabasi yo'q kurslar
6. Oxirgi 7 kunda faol bo'lgan (dars ko'rgan) talabalar
7. "Tashlab qo'yganlar": yozilgan, lekin 14+ kun dars ko'rmaganlar
8. Window: har kurs ichida talabalarni progress bo'yicha reytinglang
9. Oylik tushum + o'tgan oyga nisbatan o'sish (LAG)
10. Trigger: yangi ko'rish yozilganda yozilishlar jadvalidagi `oxirgi_faollik` ustuni yangilansin (ustunni o'zingiz qo'shing)

💡 4-topshiriqda ikki xil hisobni BITTA query'da aralashtirmang — alohida hisoblang-da, keyin birlashtiring: subquery (14-bob) yoki CTE (15-bob) bilan bo'laklang. 6 va 7 uchun sana funksiyalari (10-bob), 8–9 uchun window funksiyalar (16-bob) kerak bo'ladi.

LOYIHA C: Yetkazib berish xizmati (murakkab — portfolio darajasi!)

6 jadval, tranzaksiya, indeks auditi va backup — bu loyihani tugatsangiz, GitHub'ga qo'yib intervyuda ko'rsatsa bo'ladi.

Schema: restoranlar, taomlar (restoran, nomi, narx), kuryerlar (ism, telefon, faolmi), mijozlar, buyurtmalar (mijoz, restoran, kuryer, holat: yangi→tayyorlanmoqda→yolda→yetkazildi/bekor, yaratilgan_vaqt, yetkazilgan_vaqt), buyurtma_taomlari (buyurtma, taom, soni, narx — snapshot).

Topshiriqlar:

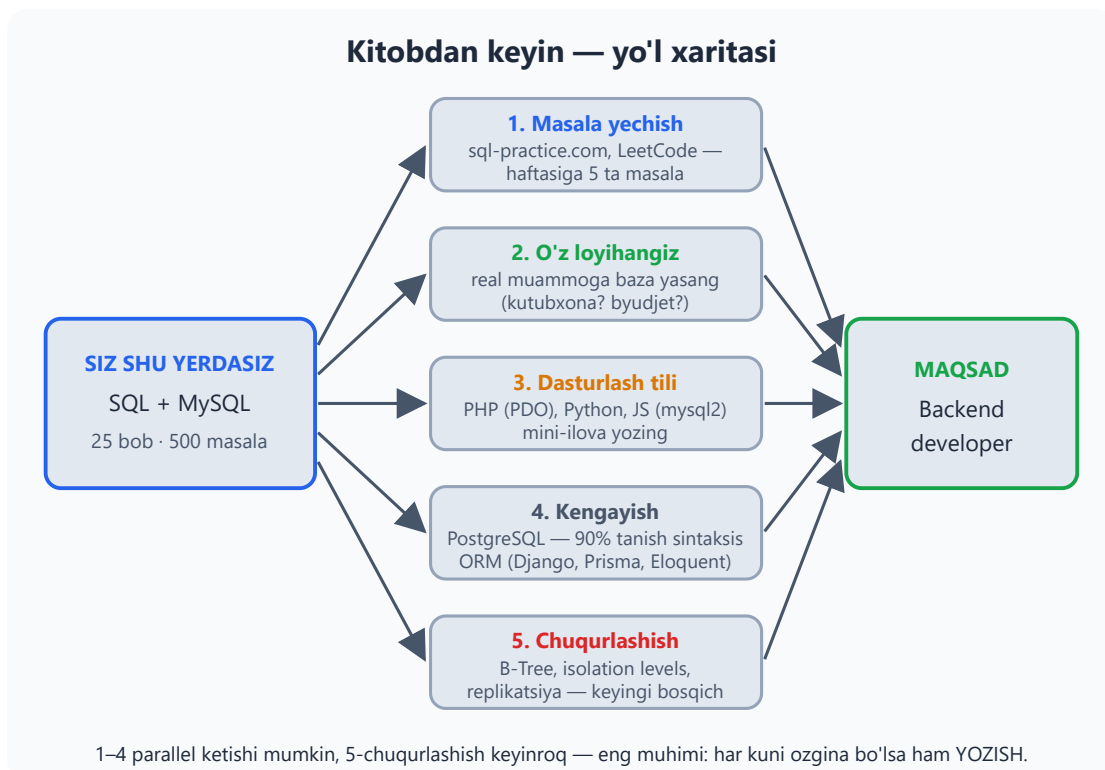
1. To'liq schema: FK, CHECK, kerakli indekslar (qaysi ustunlarga — o'zingiz asoslang!)
2. Boy test data: 5 restoran, 25 taom, 6 kuryer, 10 mijoz, 30 buyurtma har xil holatda
3. Buyurtma tranzaksiyasi (procedure): buyurtma + taomlar + jami hisoblash — atomar
4. Har restoranning daromadi (faqat 'yetkazildi'lar), kamayish tartibida
5. Har kuryer: yetkazgan buyurtmalari, o'rtacha yetkazish vaqti (TIMESTAMPDIFF(MINUTE, yaratilgan, yetkazilgan))
6. Eng ko'p buyurtma qilingan 5 taom
7. Har mijozning LTV'si (jami xaridi) + birinchi va oxirgi buyurtma sanasi (window yoki MIN/MAX)

8. Soatlar bo'yicha buyurtmalar taqsimoti (HOUR + GROUP BY) — "qaysi soat eng gavjum?"
9. Kunlik tushum + yig'ilib boruvchi summa (running total) + 3 kunlik o'rtacha
10. `v_boshqaruv_paneli` view: bugungi buyurtmalar, tushum, faol kuryerlar, o'rtacha yetkazish vaqti — bitta view'da
11. EXPLAIN audit: 5 ta asosiy query'ngizni tekshirib, kerakli indekslarni isbot bilan qo'ying
12. To'liq backup + tiklash sinovi

💡 Maslahatlar: `holat` ustuni uchun ENUM mos keladi (5-bob); 3-topshiriqda START TRANSACTION + COMMIT/ROLLBACK (19-bob); 9-dagi 3 kunlik o'rtacha — window frame: ROWS BETWEEN 2 PRECEDING AND CURRENT ROW (16-bob); 12-da mysqldump (24-bob). `yetkazilgan_vaq` faqat 'yetkazildi' holatida to'ladi — qolganlarida NULL bo'lishi tabiiy.

Loyihalardan keyin — keyingi qadamlar

Kitob tugadi, lekin yo'l endi boshlanadi. Mana xarita:



1. **Masala yechish:** sql-practice.com (bepul, brauzerda), LeetCode Database bo'limi, HackerRank SQL — haftasiga 5 ta masala

2. **O'z loyihangiz:** atrofingizdagi real muammoga baza yasang (mahalla kutubxonasi? oilaviy byudjet?)
3. **Dasturlash tiliga ulash:** PHP (PDO), Python (mysql-connector) yoki JavaScript (mysql2) bilan bazaga ulanib, mini-ilova yozing — SQL bilimingiz "jonlanadi"
4. **Kengayish:** PostgreSQL bilan tanishning — sintaksisning 90 foizi sizga tanish bo'ladi; ORM nima ekanini ko'ring (Django ORM, Prisma, Eloquent) — lekin ORM ostida baribir SQL yotganini unutmang
5. **Chuqurlashish:** indekslarning B-Tree tuzilishi, isolation levels, replikasiya — bular keyingi bosqich

25-bob masalalari (intervyuga tayyorgarlik — 20 ta klassik savol)

Bu bobning 20 masalasi — intervyu formatida. Har savolga avval o'zingiz **yozma** javob bering (og'zaki "bilaman" hisobga o'tmaydi!), keyin quyidagi jadval orqali tegishli bobdan tekshiring:

1. PRIMARY KEY va UNIQUE farqi? (PK — bitta, NULL yo'q; UNIQUE — ko'p bo'lishi mumkin, NULL mumkin)
2. INNER JOIN va LEFT JOIN farqi?
3. WHERE va HAVING farqi?
4. COUNT(*) va COUNT(ustun) farqi?
5. NULL bilan = nega ishlamaydi?
6. DELETE, TRUNCATE, DROP farqlari?
7. Indeks nima, qachon ishlamaydi?
8. Kompozit indeksning "chap qism" qoidasi?
9. Tranzaksiya nima? ACID?
10. SQL Injection nima, qanday himoyalanasiz?
11. Normalizatsiya nima? 3 ta anomaliya?
12. N:M bog'lanish qanday quriladi?
13. Subquery va JOIN — qachon qaysi?
14. GROUP BY'da SELECT'ga qanday ustunlar yozish mumkin?
15. Har guruhdagi eng katta yozuvni qanday topasiz? (window: PARTITION + ROW_NUMBER)

- 16. View nima, ma'lumot saqlaydimi?
- 17. EXPLAIN'da type=ALL nimani anglatadi?
- 18. Soft delete nima, nega kerak?
- 19. FOR UPDATE qachon kerak? (parallel yozishda race condition)
- 20. Katta OFFSET nega sekin, yechimi? (keyset pagination)

Hammasiga bu kitobda javob bor — bilmaganingizga duch kelsangiz, tegishli bobga qayting:

Savol	Bob	Savol	Bob
1	04, 18	11, 12	20
2	12	13	14
3, 4, 14	11	15	16
5	08	16	22
6, 18	17	17	23
7, 8	21	19	19
9	19	20	23
10	24		

💡 Intervyuda "yodlab kelgan ta'rif"dan ko'ra kichik misol kuchliroq ta'sir qiladi: "NULL bilan = ishlamaydi, chunki NULL — noma'lum; WHERE telefon = NULL hech narsa qaytarmaydi, IS NULL kerak" — mana shunday javob bering.

So'nggi so'z

Siz 25 bob va 500 ta masalani bosib o'tdingiz. Endi siz:

- Noldan baza loyihalay olasiz
- Istalgan murakkablikdagi query yoza olasiz
- Sekin query'ni topib, davolay olasiz

- Ma'lumotni xavfsiz saqlay olasiz

SQL 1970-yillarda tug'ilgan — 50 yildan beri asosi o'zgargani yo'q va yana o'nlab yillar xizmat qiladi. Framework'lar keladi-ketadi, SQL qoladi.

Eng katta sirni oxirida aytaman: bu kitobdagi eng kuchli o'quvchi — masalalarning hammasini YOZGAN o'quvchi. O'qigan emas. Yozgan. Omad! 🚀